US012411695B2

(12) **United States Patent**
**Ould-Ahmed-Vall et al.**

(10) **Patent No.: US 12,411,695 B2**
(45) **Date of Patent: Sep. 9, 2025**

(54) **MULTICORE PROCESSOR WITH EACH CORE HAVING INDEPENDENT FLOATING POINT DATAPATH AND INTEGER DATAPATH**

(71) Applicant: **Intel Corporation**, Santa Clara, CA (US)

(72) Inventors: **Elmoustapha Ould-Ahmed-Vall**, Chandler, AZ (US); **Barath Lakshmanan**, Chandler, AZ (US); **Tatiana Shpeisman**, Menlo Park, CA (US); **Joydeep Ray**, Folsom, CA (US); **Ping T. Tang**, Edison, NJ (US); **Michael Strickland**, Sunnyvale, CA (US); **Xiaoming Chen**, Shanghai (CN); **Anbang Yao**, Beijing (CN); **Ben J. Ashbaugh**, Folsom, CA (US); **Linda L Hurd**, Cool, CA (US); **Liwei Ma**, Beijing (CN)

(73) Assignee: **Intel Corporation**, Santa Clara, CA (US)

( * ) Notice: Subject to any disclaimer, the term of this patent is extended or adjusted under 35 U.S.C. 154(b) by 0 days.

(21) Appl. No.: **18/312,079**

(22) Filed: **May 4, 2023**

(65) **Prior Publication Data**

US 2023/0315481 A1 Oct. 5, 2023

**Related U.S. Application Data**

(63) Continuation of application No. 17/839,856, filed on Jun. 14, 2022, now Pat. No. 12,175,252, which is a (Continued)

(51) **Int. Cl.**
***G06F 9/30*** (2018.01)
***G06F 9/38*** (2018.01)
(Continued)

(52) **U.S. Cl.**
CPC ..... ***G06F 9/3887*** (2013.01); ***G06F 9/3001*** (2013.01); ***G06F 9/30014*** (2013.01);
(Continued)

(58) **Field of Classification Search**
CPC .. G06F 9/3887; G06F 9/3001; G06F 9/30014; G06F 9/30036; G06F 9/30094;
(Continued)

(56) **References Cited**

U.S. PATENT DOCUMENTS

3,872,442 A 3/1975 Boles et al.
4,476,523 A 10/1984 Beauchamp
(Continued)

FOREIGN PATENT DOCUMENTS

CN 101131674 2/2008
CN 101133393 A 2/2008
(Continued)

OTHER PUBLICATIONS

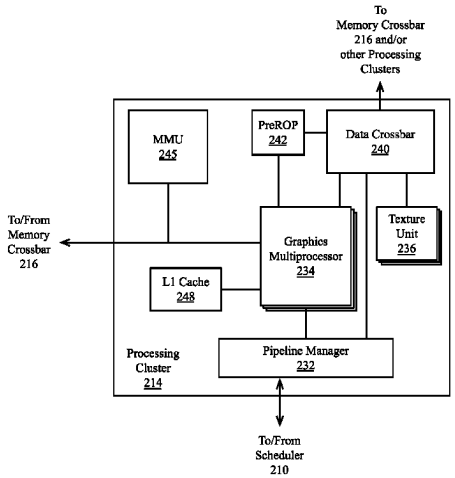
Bruintjes, T., "Sabrewing: A lightweight architecture for combined floating-point and integer arithmetic." ACM Transactions on Architecture and Code Optimization (TACO) 8.4(12): 1-22, 2012.
(Continued)

*Primary Examiner* — Shawn Doman
(74) *Attorney, Agent, or Firm* — Jaffery Watson Hamilton & DeSanctis LLP

(57) **ABSTRACT**

Described herein is a general-purpose graphics processing unit including a multiprocessor having a single instruction, multiple thread, SIMT, architecture. The multiprocessor comprises multiple sets of compute units each having a first logic unit configured to perform floating-point operations and a second logic unit configured to perform integer operations, with a thread of the floating-point instruction being executed in parallel with a thread of the integer instruction.

**21 Claims, 43 Drawing Sheets**



**Related U.S. Application Data**

continuation of application No. 15/819,167, filed on Nov. 21, 2017, now Pat. No. 11,409,537, which is a continuation of application No. 15/494,773, filed on Apr. 24, 2017, now Pat. No. 10,409,614.

(51) **Int. Cl.**

| | |
|---|---|
| ***G06F 9/50*** | (2006.01) |
| ***G06F 13/40*** | (2006.01) |
| ***G06F 13/42*** | (2006.01) |
| ***G06F 15/80*** | (2006.01) |
| ***G06F 16/242*** | (2019.01) |
| ***G06N 3/00*** | (2023.01) |
| ***G06N 3/044*** | (2023.01) |
| ***G06N 3/045*** | (2023.01) |
| ***G06N 3/063*** | (2023.01) |
| ***G06N 3/084*** | (2023.01) |
| ***G06N 20/00*** | (2019.01) |
| ***G06N 20/10*** | (2019.01) |
| ***G06T 1/20*** | (2006.01) |

(52) **U.S. Cl.**

CPC ..... ***G06F 9/30036*** (2013.01); ***G06F 9/30094*** (2013.01); ***G06F 9/30109*** (2013.01); ***G06F 9/30112*** (2013.01); ***G06F 9/3016*** (2013.01); ***G06F 9/3851*** (2013.01); ***G06F 9/3888*** (2023.08); ***G06F 9/38885*** (2023.08); ***G06F 9/3891*** (2013.01); ***G06F 9/50*** (2013.01); ***G06F 13/4068*** (2013.01); ***G06F 13/4282*** (2013.01); ***G06F 15/80*** (2013.01); ***G06N 3/00*** (2013.01); ***G06N 3/044*** (2023.01); ***G06N 3/045*** (2023.01); ***G06N 3/063*** (2013.01); ***G06N 3/084*** (2013.01); ***G06N 20/00*** (2019.01); ***G06N 20/10*** (2019.01); ***G06T 1/20*** (2013.01); *G06F 2213/0026* (2013.01)

(58) **Field of Classification Search**

CPC ..... G06F 9/30109; G06F 9/30112; G06F 9/3016; G06F 9/3851; G06F 9/3891; G06F 9/50; G06F 13/4068; G06F 13/4282; G06F 15/80; G06F 2213/0026; G06N 3/00; G06N 3/044; G06N 3/045; G06N 3/063; G06N 3/084; G06N 20/00; G06N 20/10; G06T 1/20; Y02D 10/00

See application file for complete search history.

(56) **References Cited**

U.S. PATENT DOCUMENTS


| | | | | |
|---|---|---|---|---|
| 4,823,252 | A | 4/1989 | Horst et al. | |
| 5,182,801 | A | 1/1993 | Asfour | |
| 5,381,539 | A | 1/1995 | Yanai et al. | |
| 5,450,607 | A | 9/1995 | Kowalczyk et al. | |
| 5,469,552 | A | 11/1995 | Suzuki et al. | |
| 5,471,593 | A | 11/1995 | Branigin | |
| 5,502,827 | A | 3/1996 | Yoshida | |
| 5,574,928 | A | 11/1996 | White et al. | |
| 5,627,985 | A | 5/1997 | Fetterman et al. | |
| 5,673,407 | A | 9/1997 | Poland et al. | |
| 5,737,752 | A | 4/1998 | Hilditch | |
| 5,777,629 | A | 7/1998 | Baldwin | |
| 5,805,475 | A | 9/1998 | Putrino et al. | |
| 5,822,767 | A | 10/1998 | MacWilliams et al. | |
| 5,890,211 | A | 3/1999 | Sokolov et al. | |
| 5,917,741 | A | 6/1999 | Ng | |
| 5,926,406 | A | 7/1999 | Tucker et al. | |
| 5,940,311 | A | 8/1999 | Dao et al. | |
| 5,943,687 | A | 8/1999 | Liedberg | |
| 6,038,582 | A | 3/2000 | Arakawa et al. | |
| 6,049,865 | A | 4/2000 | Smith | |
| 6,078,940 | A | 6/2000 | Scales | |
| 6,260,008 | B1 | 7/2001 | Sanfilippo | |
| 6,412,046 | B1 | 6/2002 | Sharma et al. | |
| 6,480,872 | B1 | 11/2002 | Choquette | |
| 6,513,099 | B1 | 1/2003 | Smith et al. | |
| 6,529,928 | B1 | 3/2003 | Resnick et al. | |
| 6,578,102 | B1 | 6/2003 | Batchelor et al. | |
| 6,598,120 | B1 | 7/2003 | Berg et al. | |
| 6,678,806 | B1 | 1/2004 | Redford | |
| 6,788,738 | B1 | 9/2004 | New | |
| 6,856,320 | B1 | 2/2005 | Rubinstein et al. | |
| 6,947,049 | B2 | 9/2005 | Spitzer et al. | |
| 6,963,954 | B1 | 11/2005 | Trehus et al. | |
| 7,102,646 | B1 | 9/2006 | Rubinstein et al. | |
| 7,127,482 | B2 | 10/2006 | Hou et al. | |
| 7,197,605 | B2 | 3/2007 | Schmisseur et al. | |
| 7,327,289 | B1 | 2/2008 | Lippincott | |
| 7,346,741 | B1 | 3/2008 | Keish et al. | |
| 7,373,369 | B2 | 5/2008 | Gerwig et al. | |
| 7,483,031 | B2 | 1/2009 | Williams et al. | |
| 7,567,252 | B2 | 7/2009 | Buck et al. | |
| 7,616,206 | B1 | 11/2009 | Danilak | |
| 7,620,793 | B1 | 11/2009 | Edmondson et al. | |
| 7,873,812 | B1 | 1/2011 | Mimar | |
| 7,913,041 | B2 | 3/2011 | Shen et al. | |
| 7,958,558 | B1 | 6/2011 | Leake et al. | |
| 8,253,750 | B1 | 8/2012 | Huang et al. | |
| 8,340,280 | B2 | 12/2012 | Gueron et al. | |
| 8,429,351 | B1 | 4/2013 | Yu et al. | |
| 8,458,354 | B2 | 6/2013 | Bremler-Barr et al. | |
| 8,488,055 | B2 | 7/2013 | Côtéet al. | |
| 8,645,634 | B1 | 2/2014 | Cox et al. | |
| 8,669,990 | B2 * | 3/2014 | Sprangle | G06F 9/3877 712/205 |
| 8,847,965 | B2 | 9/2014 | Chandak et al. | |
| 8,990,505 | B1 | 3/2015 | Jamil et al. | |
| 9,104,633 | B2 | 8/2015 | Moloney | |
| 9,170,955 | B2 | 10/2015 | Forsyth et al. | |
| 9,304,835 | B1 | 4/2016 | Ekanadham et al. | |
| 9,317,251 | B2 | 4/2016 | Tsen et al. | |
| 9,461,667 | B2 | 10/2016 | Quinnell | |
| 9,483,293 | B2 * | 11/2016 | McNairy | G06F 12/0893 |
| 9,491,112 | B1 | 11/2016 | Patel et al. | |
| 9,501,392 | B1 | 11/2016 | Weingarten | |
| 9,558,156 | B1 | 1/2017 | Bekas et al. | |
| 9,690,696 | B1 | 6/2017 | Hefner et al. | |
| 9,727,337 | B2 | 8/2017 | Gschwind et al. | |
| 9,804,666 | B2 | 10/2017 | Jiao | |
| 9,811,468 | B2 | 11/2017 | Hooker et al. | |
| 9,960,917 | B2 | 5/2018 | Gopal et al. | |
| 10,002,045 | B2 | 6/2018 | Chung et al. | |
| 10,049,322 | B2 | 8/2018 | Ross | |
| 10,102,015 | B1 | 10/2018 | Gordon et al. | |
| 10,146,738 | B2 | 12/2018 | Nurvitadhi et al. | |
| 10,229,470 | B2 | 3/2019 | Benthin et al. | |
| 10,353,706 | B2 | 7/2019 | Kaul et al. | |
| 10,379,865 | B1 | 8/2019 | Menezes et al. | |
| 10,409,614 | B2 | 9/2019 | Ould-Ahmed-Vall et al. | |
| 10,409,887 | B1 | 9/2019 | Gauria et al. | |
| 10,474,458 | B2 | 11/2019 | Kaul et al. | |
| 10,528,864 | B2 | 1/2020 | Dally et al. | |
| 10,572,409 | B1 | 2/2020 | Zejda et al. | |
| 10,678,508 | B2 | 6/2020 | Vantrease et al. | |
| 10,755,201 | B2 | 8/2020 | Sika | |
| 10,762,137 | B1 | 9/2020 | Volpe et al. | |
| 10,762,164 | B2 | 9/2020 | Chen et al. | |
| 10,769,750 | B1 | 9/2020 | Yoon | |
| 10,860,316 | B2 | 12/2020 | Zhi et al. | |
| 10,860,922 | B2 | 12/2020 | Dally et al. | |
| 10,891,538 | B2 | 1/2021 | Dally et al. | |
| 10,896,045 | B2 | 1/2021 | Sodani et al. | |
| 11,080,046 | B2 | 8/2021 | Kaul et al. | |
| 11,113,784 | B2 | 9/2021 | Ray et al. | |
| 11,169,799 | B2 | 11/2021 | Kaul et al. | |
| 11,250,108 | B2 | 2/2022 | Dong et al. | |
| 11,360,767 | B2 | 6/2022 | Kaul et al. | |
| 11,361,496 | B2 | 6/2022 | Maiyuran et al. | |
| 11,409,537 | B2 | 8/2022 | Ould-Ahmed-Vall et al. | |
| 11,461,107 | B2 | 10/2022 | Ould-Ahmed-Vall et al. | |

(56) **References Cited**

U.S. PATENT DOCUMENTS

11,550,971 B1 1/2023 Lu et al.
11,620,256 B2 4/2023 Koker et al.
2001/0042194 A1 11/2001 Elliott et al.
2002/0152361 A1 10/2002 Dean et al.
2002/0156979 A1 10/2002 Rodriguez
2002/0188808 A1 12/2002 Rowlands et al.
2003/0204840 A1 10/2003 Wu
2004/0054841 A1 3/2004 Callison et al.
2005/0080834 A1 4/2005 Belluomini et al.
2005/0125631 A1 6/2005 Symes et al.
2005/0169463 A1 8/2005 Ahn et al.
2005/0219253 A1 10/2005 Piazza et al.
2006/0037003 A1 2/2006 Long et al.
2006/0083489 A1 4/2006 Aridome et al.
2006/0101244 A1 5/2006 Siu et al.
2006/0143396 A1 6/2006 Cabot
2006/0149803 A1 7/2006 Siu et al.
2006/0179092 A1 8/2006 Schmookler
2006/0248279 A1 11/2006 Al-Sukhni et al.
2006/0265576 A1 11/2006 Davis et al.
2006/0277365 A1 12/2006 Pong
2006/0282620 A1 12/2006 Kashyap et al.
2007/0030277 A1 2/2007 Prokopenko et al.
2007/0030279 A1 2/2007 Paltashev et al.
2007/0074008 A1 3/2007 Donofrio
2007/0083742 A1 4/2007 Abernathy et al.
2007/0115291 A1 5/2007 Chen et al.
2007/0130432 A1 6/2007 Aigo
2007/0211064 A1 9/2007 Buck et al.
2007/0273698 A1 11/2007 Du et al.
2007/0294682 A1 12/2007 Demetriou et al.
2008/0028152 A1 1/2008 Du et al.
2008/0030510 A1 2/2008 Wan et al.
2008/0052466 A1 2/2008 Zulauf
2008/0071851 A1 3/2008 Zohar et al.
2008/0086598 A1 4/2008 Maron et al.
2008/0147992 A1 6/2008 Raikin et al.
2008/0189487 A1 8/2008 Craske
2008/0288746 A1 11/2008 Inglett et al.
2008/0307207 A1 12/2008 Khailany et al.
2009/0003593 A1 1/2009 Gopal et al.
2009/0019253 A1 1/2009 Stecher et al.
2009/0030960 A1 1/2009 Geraghty et al.
2009/0113170 A1 4/2009 Abdallah
2009/0150654 A1 6/2009 Oberman et al.
2009/0157972 A1 6/2009 Byers et al.
2009/0182942 A1 7/2009 Greiner et al.
2009/0189898 A1 7/2009 Dammertz et al.
2009/0190432 A1 7/2009 Bilger et al.
2009/0254733 A1 10/2009 Chen et al.
2009/0307472 A1 12/2009 Essick, IV et al.
2009/0309896 A1 12/2009 DeLaurier et al.
2010/0053162 A1 3/2010 Dammertz et al.
2010/0082906 A1 4/2010 Hinton et al.
2010/0082910 A1 4/2010 Raikar et al.
2010/0162247 A1 6/2010 Welc et al.
2010/0185816 A1 7/2010 Sauber et al.
2010/0228941 A1 9/2010 Koob et al.
2010/0281235 A1 11/2010 Vorbach et al.
2010/0293334 A1 11/2010 Xun et al.
2010/0299656 A1 11/2010 Shah et al.
2010/0312944 A1 12/2010 Walker
2010/0332775 A1 12/2010 Kapil et al.
2011/0040744 A1 2/2011 Haas et al.
2011/0040822 A1* 2/2011 Eichenberger ..... G06F 9/30038 708/607
2011/0060879 A1 3/2011 Rogers et al.
2011/0078226 A1 3/2011 Baskaran et al.
2011/0119446 A1 5/2011 Blumrich et al.
2011/0153707 A1 6/2011 Ginzburg et al.
2011/0157195 A1 6/2011 Sprangle et al.
2011/0238934 A1 9/2011 Xu et al.
2012/0011182 A1 1/2012 Raafat et al.
2012/0042130 A1 2/2012 Peapell
2012/0059983 A1 3/2012 Nellans et al.
2012/0072631 A1 3/2012 Chirca et al.
2012/0075319 A1 3/2012 Dally
2012/0124115 A1 5/2012 Ollmann
2012/0154411 A1 6/2012 Cheng
2012/0233444 A1 9/2012 Stephens et al.
2012/0268909 A1 10/2012 Emma et al.
2012/0278376 A1 11/2012 Bakos
2013/0013864 A1 1/2013 Chung et al.
2013/0031328 A1 1/2013 Kelleher et al.
2013/0073599 A1 3/2013 Maloney
2013/0099946 A1 4/2013 Dickie et al.
2013/0111136 A1 5/2013 Bell, Jr. et al.
2013/0138795 A1 5/2013 Field et al.
2013/0141442 A1 6/2013 Brothers et al.
2013/0148947 A1 6/2013 Glen et al.
2013/0185515 A1 7/2013 Sassone et al.
2013/0218938 A1 8/2013 Dockser et al.
2013/0219088 A1 8/2013 Rawe et al.
2013/0235925 A1 9/2013 Nguyen et al.
2013/0254491 A1 9/2013 Coleman et al.
2013/0283135 A1 10/2013 Kiyoshi et al.
2013/0293544 A1 11/2013 Schreyer et al.
2013/0297906 A1 11/2013 Loh et al.
2014/0006753 A1 1/2014 Gopal et al.
2014/0032818 A1 1/2014 Chang et al.
2014/0040552 A1 2/2014 Rychlik et al.
2014/0052928 A1 2/2014 Shimoi
2014/0059328 A1 2/2014 Gonion
2014/0068197 A1 3/2014 Joshi et al.
2014/0075060 A1 3/2014 Sharp et al.
2014/0075163 A1 3/2014 Loewenstein et al.
2014/0082322 A1 3/2014 Loh et al.
2014/0089371 A1 3/2014 Dinechin et al.
2014/0089697 A1 3/2014 Kim et al.
2014/0095796 A1 4/2014 Bell, Jr. et al.
2014/0108481 A1 4/2014 Davis et al.
2014/0122555 A1 5/2014 Hickmann et al.
2014/0129807 A1 5/2014 Tannenbaum et al.
2014/0146607 A1 5/2014 Nagai et al.
2014/0173203 A1 6/2014 Forsyth
2014/0173207 A1 6/2014 Wang et al.
2014/0181487 A1 6/2014 Sasanka
2014/0188963 A1 7/2014 Tsen et al.
2014/0188966 A1 7/2014 Galal et al.
2014/0208069 A1 7/2014 Wegener
2014/0223096 A1 8/2014 Zhe Yang et al.
2014/0223131 A1 8/2014 Agarwal et al.
2014/0229713 A1 8/2014 Muff et al.
2014/0258622 A1 9/2014 Lacourba et al.
2014/0266417 A1 9/2014 Dally et al.
2014/0267232 A1 9/2014 Lum et al.
2014/0281008 A1 9/2014 Muthiah et al.
2014/0281110 A1 9/2014 Duluk, Jr. et al.
2014/0281261 A1 9/2014 Vera et al.
2014/0281274 A1* 9/2014 Pokam ..... G06F 9/321 711/145
2014/0281299 A1 9/2014 Duluk, Jr. et al.
2014/0317388 A1 10/2014 Chung et al.
2014/0348431 A1 11/2014 Brick et al.
2014/0365715 A1 12/2014 Lee
2014/0379987 A1 12/2014 Aggarwal et al.
2015/0039661 A1 2/2015 Blomgren et al.
2015/0046655 A1 2/2015 Nystad et al.
2015/0046662 A1 2/2015 Heinrich et al.
2015/0067259 A1 3/2015 Wang et al.
2015/0095588 A1 4/2015 Abdallah
2015/0106613 A1 4/2015 Amann et al.
2015/0160872 A1 6/2015 Chen
2015/0193358 A1 7/2015 Dyke et al.
2015/0205615 A1 7/2015 Cunningham et al.
2015/0205724 A1 7/2015 Hancock et al.
2015/0221063 A1 8/2015 Kim et al.
2015/0235338 A1 8/2015 Alla et al.
2015/0254104 A1 9/2015 Kessler et al.
2015/0261683 A1 9/2015 Hong et al.
2015/0268940 A1 9/2015 Baghsorkhi et al.
2015/0268963 A1 9/2015 Etsion et al.
2015/0278984 A1 10/2015 Koker et al.
2015/0334043 A1 11/2015 Li et al.

(56) **References Cited**

U.S. PATENT DOCUMENTS

2015/0339229 A1 11/2015 Zhang et al.
2015/0349953 A1 12/2015 Kruglick
2015/0371407 A1 12/2015 Kim et al.
2015/0378741 A1 12/2015 Lukyanov et al.
2015/0378920 A1 12/2015 Gierach et al.
2015/0380088 A1 12/2015 Naeimi et al.
2016/0062947 A1 3/2016 Chetlur et al.
2016/0070536 A1 3/2016 Maeda
2016/0071242 A1* 3/2016 Uralsky ..... G06T 5/70 345/611
2016/0079997 A1 3/2016 Raichelgauz et al.
2016/0086303 A1 3/2016 Bae et al.
2016/0092118 A1 3/2016 Kumar et al.
2016/0092239 A1 3/2016 Maiyuran et al.
2016/0092240 A1 3/2016 Maiyuran et al.
2016/0124713 A1 5/2016 Bekas et al.
2016/0124861 A1 5/2016 Fujii et al.
2016/0140686 A1 5/2016 Lueh et al.
2016/0179535 A1 6/2016 Chen et al.
2016/0188295 A1 6/2016 Rarick et al.
2016/0239065 A1 8/2016 Lee et al.
2016/0246723 A1 8/2016 Doshi et al.
2016/0255169 A1 9/2016 Kovvuri et al.
2016/0283249 A1* 9/2016 Forsell ..... G06F 9/3875
2016/0307362 A1 10/2016 Farrell et al.
2016/0321187 A1 11/2016 Bernat et al.
2016/0342892 A1 11/2016 Ross
2016/0350227 A1 12/2016 Hooker et al.
2016/0357680 A1 12/2016 Hooker et al.
2016/0378366 A1 12/2016 Tomishima et al.
2016/0378465 A1 12/2016 Venkatesh et al.
2016/0378674 A1 12/2016 Cheng et al.
2017/0024327 A1 1/2017 Saidi et al.
2017/0039144 A1 2/2017 Wang et al.
2017/0039674 A1 2/2017 Klosowski et al.
2017/0061279 A1 3/2017 Yang et al.
2017/0083240 A1 3/2017 Rogers et al.
2017/0090924 A1 3/2017 Mishra et al.
2017/0109282 A1 4/2017 Frank et al.
2017/0132134 A1 5/2017 Gschwind et al.
2017/0139748 A1 5/2017 Tipparaju et al.
2017/0177336 A1 6/2017 Astafev et al.
2017/0185379 A1 6/2017 Anderson et al.
2017/0200303 A1 7/2017 Havran et al.
2017/0214930 A1 7/2017 Loughry
2017/0220592 A1 8/2017 Foltz
2017/0228252 A1 8/2017 Gadelrab et al.
2017/0228342 A1 8/2017 Narayanaswami et al.
2017/0277460 A1 9/2017 Li et al.
2017/0308800 A1 10/2017 Cichon et al.
2017/0315921 A1 11/2017 Hooker et al.
2017/0315932 A1 11/2017 Moyer
2017/0323042 A1 11/2017 Zhang
2017/0344822 A1 11/2017 Popescu et al.
2017/0357506 A1 12/2017 Wang et al.
2018/0011790 A1 1/2018 Gaur et al.
2018/0018153 A1 1/2018 Mukai
2018/0018266 A1 1/2018 Jones, III
2018/0026651 A1 1/2018 Gopal et al.
2018/0040096 A1 2/2018 Benthin
2018/0046898 A1 2/2018 Lo
2018/0046900 A1 2/2018 Dally et al.
2018/0046906 A1 2/2018 Dally et al.
2018/0052920 A1 2/2018 Klein et al.
2018/0067869 A1 3/2018 Yang et al.
2018/0107602 A1 4/2018 Dasgupta et al.
2018/0114114 A1 4/2018 Molchanov et al.
2018/0129608 A9 5/2018 Damodaran et al.
2018/0157464 A1 6/2018 Lutz et al.
2018/0165204 A1 6/2018 Venkatesh et al.
2018/0173623 A1 6/2018 Koob et al.
2018/0174353 A1 6/2018 Shin et al.
2018/0181519 A1 6/2018 Cheng
2018/0183577 A1 6/2018 Suresh et al.
2018/0189231 A1 7/2018 Fleming, Jr. et al.
2018/0189925 A1 7/2018 Lee et al.
2018/0210830 A1 7/2018 Malladi et al.
2018/0210836 A1 7/2018 Lai et al.
2018/0232846 A1 8/2018 Gruber et al.
2018/0253635 A1 9/2018 Park
2018/0276044 A1 9/2018 Fong et al.
2018/0276150 A1 9/2018 Eckert et al.
2018/0284994 A1 10/2018 Haller et al.
2018/0285261 A1 10/2018 Mandal et al.
2018/0285264 A1 10/2018 Kayiran et al.
2018/0285278 A1 10/2018 Appu et al.
2018/0286053 A1 10/2018 Labbe et al.
2018/0293173 A1 10/2018 Zhu et al.
2018/0293784 A1 10/2018 Benthin et al.
2018/0293965 A1 10/2018 Vembu et al.
2018/0295039 A1 10/2018 Yasman et al.
2018/0300105 A1 10/2018 Langhammer
2018/0300258 A1 10/2018 Wokhlu et al.
2018/0307494 A1 10/2018 Ould-Ahmed-Vall et al.
2018/0307498 A1 10/2018 Yang et al.
2018/0315159 A1 11/2018 Ould-Ahmed-Vall et al.
2018/0315398 A1 11/2018 Kaul et al.
2018/0315399 A1 11/2018 Kaul et al.
2018/0321938 A1 11/2018 Boswell et al.
2018/0322387 A1 11/2018 Sridharan et al.
2018/0336136 A1 11/2018 Hijaz et al.
2018/0373200 A1 12/2018 Shi et al.
2018/0373635 A1 12/2018 Mukherjee et al.
2018/0373809 A1 12/2018 Ylitie et al.
2019/0035140 A1 1/2019 Fricke et al.
2019/0042193 A1 2/2019 Pasca et al.
2019/0042237 A1 2/2019 Azizi et al.
2019/0042244 A1 2/2019 Henry et al.
2019/0042457 A1 2/2019 Doshi et al.
2019/0042534 A1 2/2019 Butera et al.
2019/0042542 A1 2/2019 Narayanamoorthy et al.
2019/0042923 A1 2/2019 Janedula et al.
2019/0065051 A1 2/2019 Mills et al.
2019/0065150 A1 2/2019 Heddes et al.
2019/0065195 A1 2/2019 Pool et al.
2019/0065338 A1 2/2019 Bramley et al.
2019/0066255 A1 2/2019 Nalluri et al.
2019/0073590 A1 3/2019 Wu et al.
2019/0079767 A1 3/2019 Heinecke et al.
2019/0079768 A1 3/2019 Heinecke et al.
2019/0095223 A1 3/2019 Dubel et al.
2019/0095336 A1 3/2019 Barczak
2019/0108651 A1 4/2019 Gu et al.
2019/0114534 A1 4/2019 Teng et al.
2019/0114535 A1 4/2019 Ng et al.
2019/0121638 A1 4/2019 Knowles
2019/0121679 A1 4/2019 Wilkinson et al.
2019/0121680 A1 4/2019 Wilkinson et al.
2019/0129847 A1 5/2019 Roh
2019/0130271 A1 5/2019 Narang et al.
2019/0146800 A1 5/2019 Ould-Ahmed-Vall et al.
2019/0155768 A1 5/2019 Wilkinson et al.
2019/0164050 A1 5/2019 Chen et al.
2019/0164268 A1 5/2019 Gallo et al.
2019/0179757 A1 6/2019 Walker et al.
2019/0188142 A1 6/2019 Rappoport et al.
2019/0205146 A1 7/2019 Patel et al.
2019/0205358 A1 7/2019 Diril et al.
2019/0221556 A1 7/2019 Gomes et al.
2019/0227936 A1 7/2019 Jang
2019/0251034 A1 8/2019 Bernat et al.
2019/0266217 A1 8/2019 Arakawa et al.
2019/0278593 A1 9/2019 Elango et al.
2019/0278600 A1 9/2019 Frumkin et al.
2019/0294413 A1 9/2019 Vantrease et al.
2019/0327124 A1 10/2019 Lai et al.
2019/0347043 A1 11/2019 Hasbun et al.
2019/0361954 A1 11/2019 Page et al.
2019/0369988 A1 12/2019 Kaul et al.
2019/0370173 A1 12/2019 Boyer et al.
2019/0392287 A1 12/2019 Ovsiannikov et al.
2020/0021540 A1 1/2020 Marolia et al.
2020/0042362 A1 2/2020 Cui et al.
2020/0050830 A1 2/2020 Krueger et al.

(56) **References Cited**

U.S. PATENT DOCUMENTS

2020/0058155 A1 2/2020 Bakalash et al.
2020/0061811 A1 2/2020 Iqbal et al.
2020/0065241 A1 2/2020 Cho et al.
2020/0073825 A1 3/2020 Zaydman
2020/0081714 A1 3/2020 Britto et al.
2020/0097409 A1 3/2020 Nathella et al.
2020/0097411 A1 3/2020 Pusdesris et al.
2020/0098725 A1 3/2020 Liff et al.
2020/0117463 A1 4/2020 Nassi et al.
2020/0117999 A1 4/2020 Yoon et al.
2020/0133898 A1 4/2020 Therene et al.
2020/0150926 A1 5/2020 Connor et al.
2020/0174897 A1 6/2020 McNamara et al.
2020/0175074 A1 6/2020 Li et al.
2020/0184309 A1 6/2020 Patel
2020/0201810 A1 6/2020 Felix et al.
2020/0202195 A1 6/2020 Patel
2020/0203332 A1 6/2020 Schreiber et al.
2020/0210192 A1 7/2020 Alexander et al.
2020/0211147 A1 7/2020 Doyle et al.
2020/0211253 A1 7/2020 Liktor et al.
2020/0211267 A1 7/2020 Rowley et al.
2020/0218538 A1 7/2020 Mansell
2020/0218540 A1 7/2020 Kesiraju et al.
2020/0226920 A1 7/2020 Takenaka et al.
2020/0234124 A1 7/2020 Park
2020/0242049 A1 7/2020 Loh et al.
2020/0250098 A1 8/2020 Ma et al.
2020/0285592 A1 9/2020 Ambroladze et al.
2020/0311531 A1 10/2020 Liu et al.
2020/0371752 A1 11/2020 Murray et al.
2020/0380369 A1 12/2020 Case et al.
2021/0012197 A1 1/2021 Simonyan et al.
2021/0019591 A1 1/2021 Venkatesh et al.
2021/0034979 A1 2/2021 Robinson et al.
2021/0035258 A1 2/2021 Ray et al.
2021/0103550 A1 4/2021 Appu et al.
2021/0117084 A1 4/2021 Shabi et al.
2021/0117201 A1 4/2021 Gray
2021/0124579 A1 4/2021 Kaul et al.
2021/0150663 A1 5/2021 Maiyuran et al.
2021/0150770 A1 5/2021 Appu et al.
2021/0182024 A1 6/2021 Mueller et al.
2021/0182058 A1 6/2021 Kaul et al.
2021/0182140 A1 6/2021 Gruber
2021/0200680 A1 7/2021 Vogelsang et al.
2021/0211643 A1 7/2021 Gabriel et al.
2021/0216316 A1 7/2021 Bhoria et al.
2021/0224125 A1 7/2021 Liu et al.
2021/0248085 A1 8/2021 Bian et al.
2021/0312697 A1 10/2021 Maiyuran et al.
2021/0374897 A1 12/2021 Ray et al.
2022/0019431 A1 1/2022 Kaul et al.
2022/0066931 A1 3/2022 Ray et al.
2022/0066971 A1 3/2022 Brewer et al.
2022/0070115 A1 3/2022 Brewer et al.
2022/0100518 A1 3/2022 Tomei et al.
2022/0107914 A1 4/2022 Koker et al.
2022/0114096 A1 4/2022 Striramassarma et al.
2022/0114108 A1 4/2022 Koker et al.
2022/0121421 A1 4/2022 Appu et al.
2022/0122215 A1 4/2022 Ray et al.
2022/0129265 A1 4/2022 Appu et al.
2022/0129266 A1 4/2022 Maiyuran et al.
2022/0129271 A1 4/2022 Appu et al.
2022/0129521 A1 4/2022 Surti et al.
2022/0137967 A1 5/2022 Koker et al.
2022/0138101 A1 5/2022 Appu et al.
2022/0138104 A1 5/2022 Koker et al.
2022/0138895 A1 5/2022 Raganathan et al.
2022/0156202 A1 5/2022 Koker et al.
2022/0171710 A1 6/2022 Koker et al.
2022/0179787 A1 6/2022 Koker et al.
2022/0180467 A1 6/2022 Koker et al.
2022/0197664 A1 6/2022 Patterson et al.
2022/0197800 A1 6/2022 Appu et al.
2022/0197975 A1 6/2022 Adelman et al.
2022/0237478 A1 7/2022 Lafford et al.
2022/0335563 A1 10/2022 Elzur
2022/0350751 A1 11/2022 Koker et al.
2022/0350768 A1 11/2022 Brewer et al.
2022/0357945 A1 11/2022 Kaul et al.
2022/0365901 A1 11/2022 Maiyuran et al.
2022/0382555 A1 12/2022 Ould-Ahmed-Vall et al.
2023/0014565 A1 1/2023 Ray et al.
2023/0046506 A1 2/2023 Kaul et al.
2023/0095363 A1 3/2023 Hsu et al.
2023/0333981 A1 10/2023 Lee et al.
2023/0419585 A1 12/2023 Smith et al.
2024/0111609 A1 4/2024 George et al.
2024/0138299 A1 5/2024 O'Connor et al.

FOREIGN PATENT DOCUMENTS

CN 101238454 8/2008
CN 100458738 C 2/2009
CN 102214160 10/2011
CN 104011676 8/2014
CN 104321741 A 1/2015
CN 104603748 5/2015
CN 105468542 A 4/2016
CN 106683036 A 5/2017
CN 107688745 A 2/2018
CN 108268422 A 7/2018
CN 112506567 A 3/2021
CN 111666066 B 11/2021
CN 115756384 A 3/2023
EP 0656592 A1 6/1995
EP 2937794 A1 10/2015
EP 2849410 B1 2/2018
EP 3382533 A1 10/2018
EP 3396533 A2 10/2018
EP 3407183 A2 11/2018
EP 3407183 A3 2/2019
EP 3958116 A1 2/2022
EP 4270201 A2 11/2023
EP 4290370 A1 12/2023
GB 2286909 A 8/1995
GB 2296155 A 6/1996
GB 2447428 A 9/2008
GB 2455401 A 6/2009
GB 2612167 10/2009
GB 2612167 A 4/2023
JP 2011530234 12/2011
JP 2012528377 A 11/2012
JP 2013246598 12/2013
JP 2017184250 A 10/2017
JP 2018120589 A 8/2018
JP 2019008444 A 1/2019
JP 2019036298 A 3/2019
JP 2019029023 B 3/2020
JP 2024138299 10/2024
KR 20120113777 A1 10/2012
KR 1020120113777 A 10/2012
KR 20170052432 A 5/2017
KR 1020170093697 A 8/2017
KR 20170126999 A 11/2017
KR 1020170126999 A 11/2017
KR 1020190003849 A 1/2019
KR 20190027367 A 3/2019
KR 1020170126999 B1 4/2020
KR 1020190027367 B1 8/2023
TW 200949691 A 12/2009
TW 201344564 A 11/2013
TW 201614997 A 4/2016
WO 2013095619 A1 6/2013
WO 2013101018 A1 7/2013
WO 2013119226 A1 8/2013
WO 2016097813 A1 6/2016
WO 2018125250 A1 7/2018
WO 2018213636 A1 11/2018
WO 2020190796 A1 9/2020
WO 2020190797 A1 9/2020
WO 2020190798 A1 9/2020

(56) **References Cited**

FOREIGN PATENT DOCUMENTS

| | | | |
|---|---|---|---|
| WO | 2020190799 | A2 | 9/2020 |
| WO | 2020190800 | A1 | 9/2020 |
| WO | 2020190801 | A1 | 9/2020 |
| WO | 2020190802 | A1 | 9/2020 |
| WO | 2020190803 | A1 | 9/2020 |
| WO | 2020190805 | A1 | 9/2020 |
| WO | 2020190806 | A1 | 9/2020 |
| WO | 2020190807 | A1 | 9/2020 |
| WO | 2020190808 | A1 | 9/2020 |
| WO | 2020190809 | A1 | 9/2020 |
| WO | 2020190810 | A1 | 9/2020 |
| WO | 2020190811 | A1 | 9/2020 |
| WO | 2020190812 | A1 | 9/2020 |
| WO | 2020190813 | A1 | 9/2020 |
| WO | 2020190814 | A1 | 9/2020 |
| WO | 2023069384 | A1 | 4/2023 |
| WO | 2023249742 | | 12/2023 |

OTHER PUBLICATIONS

Herrera, A., "Nvidia Grid: Graphics Accelerated VDI with the Visual Performance of a Workstation", 18 pages, May 2014.

Macri, J., "AMD's Next Generation GPU and High Bandwidth Memory Architecture: Fury", 26 pages, Aug. 2015.

Office Action for TW Application No. 112124508, mailed Aug. 22, 2023, 4 pages.

Office Action for U.S. Appl. No. 17/862,739 mailed Sep. 21, 2023, 19 pages.

Sze, V. et al., "Efficient Processing of Deep Neural Networks: A Tutorial and Survey", 31 pages. Mar. 27, 2017.

U.S. Appl. No. 17/429,873 "Notice of Allowance" mailed Sep. 7, 2023, 8 pages.

U.S. Appl. No. 17/590,362 "Non-Final Office Action" mailed Jul. 10, 2023, 29 pages.

U.S. Appl. No. 17/967,283 "Non-Final Office Action" mailed Aug. 18, 2023, 23 pages.

Yuan, et al., "Complexity Effective Memory Access Scheduling for Many-Core Accelerator Architectures", 11 pages, Dec. 2009.

Alfredo Buttari et al. "Using Mixed Precision for Sparse Matrix Computations to Enhance the Performance while Achieving 64-bit Accuracy" University of Tennessee Knoxville, Oak Ridge National Laboratory (Year: 2006).

AMD, "Graphics Core Next Architecture, Generation 3", Aug. 2016. (Year: 2016).

Anonymous: "bfloat16 floating-point format", retrieved from wikipedia on May 26, 2020, Aug. 24, 2018, 5 pages.

Cano, Alberto: A survey on graphic processing unit computing for large-scale data mining. In: WIREs Data Mining and Knowledge Discovery, 8, Dec. 14, 2017, (Online), 1, S. e1232:1-e1232:24. ISSN 1942-4795.

Cui Xuewen et al: Directive-Based Partitioning and Pipelining for Graphics Processing Units11 , 2017 IEEE International Parallel and Distributed Processing Symposium (IPDPS), IEEE, May 29, 2017 (May 29, 2017), pp. 575-584, XP033113969.

Decision on Allowance for Taiwan Application No. 110127153, 1 page, mailed Oct. 27, 2022.

Eriko Nurvitadhi et al: "Can FPGAs Beat GPUs in Accelerating Next-Generation Deep Neural Networks?" Proceedings of The 2017 ACM/SIGDA International Symposium On 1Field-Programmable Gate Arrays, FPGA 17, Feb. 22, 2017, pp. 5-14, XP055542383.

Goodfellow, et al. "Adaptive Computation and Machine Learning Series", Book, Nov. 18, 2016, pp. 98-165, Chapter 5, The MIT Press, Cambridge, MA.

International Patent Application No. PCT/US20/022840 "International Preliminary Report on Patentability" issued Sep. 16, 2021, 9 pages.

International Patent Application No. PCT/US20/22833 "International Preliminary Report on Patentability" issued Sep. 16, 2021, 6 pages.

International Patent Application No. PCT/US20/22833 "International Search Report and Written Opinion" mailed Jul. 8, 2020, 8 pages.

International Patent Application No. PCT/US20/22835 "International Preliminary Report on Patentability" issued Sep. 16, 2021, 7 pages.

International Patent Application No. PCT/US20/22835 "International Search Report and Written Opinion" mailed Jun. 10, 2020, 10 pages.

International Patent Application No. PCT/US20/22836 "International Preliminary Report on Patentability" issued Sep. 16, 2021, 8 pages.

International Patent Application No. PCT/US20/22836 "International Search Report and Written Opinion" mailed Jun. 26, 2020, 11 pages.

International Patent Application No. PCT/US20/22837 "International Preliminary Report on Patentability" issued Sep. 16, 2021, 9 pages.

International Patent Application No. PCT/US20/22837 "International Search Report and Written Opinion" mailed Sep. 15, 2020, 14 pages.

International Patent Application No. PCT/US20/22838 "International Preliminary Report on Patentability" issued Sep. 16, 2021, 8 pages.

International Patent Application No. PCT/US20/22838 "International Search Report and Written Opinion" mailed Jul. 6, 2020, 11 pages.

International Patent Application No. PCT/US20/22839 "International Preliminary Report on Patentability" issued Sep. 16, 2021, 6 pages.

International Patent Application No. PCT/US20/22839 "International Search Report and Written Opinion" mailed Jul. 8, 2020, 8 pages.

International Patent Application No. PCT/US20/22840 "International Search Report and Written Opinion" mailed Jul. 3, 2020, 11 pages.

International Patent Application No. PCT/US20/22841 "International Preliminary Report on Patentability" issued Sep. 16, 2021, 7 pages.

International Patent Application No. PCT/US20/22841 "International Search Report and Written Opinion" mailed Jun. 7, 2020, 10 pages.

International Patent Application No. PCT/US20/22843 "International Preliminary Report on Patentability" issued Sep. 16, 2021, 7 pages.

International Patent Application No. PCT/US20/22843 "International Search Report and Written Opinion" mailed Jul. 28, 2020, 13 pages.

International Patent Application No. PCT/US20/22844 "International Preliminary Report on Patentability" issued Sep. 16, 2021, 10 pages.

International Patent Application No. PCT/US20/22844 "International Search Report and Written Opinion" mailed Jul. 2, 2020, 13 pages.

International Patent Application No. PCT/US20/22845 "International Preliminary Report on Patentability" issued Sep. 16, 2021, 10 pages.

International Patent Application No. PCT/US20/22845 "International Search Report and Written Opinion" mailed Jun. 30, 2020, 13 pages.

International Patent Application No. PCT/US20/22846 "International Preliminary Report on Patentability" issued Sep. 16, 2021, 12 pages.

International Patent Application No. PCT/US20/22846 "International Search Report and Written Opinion" mailed Aug. 31, 2020, 16 pages.

International Patent Application No. PCT/US20/22847 "International Preliminary Report on Patentability" issued Sep. 16, 2021, 6 pages.

International Patent Application No. PCT/US20/22847 "International Search Report and Written Opinion" mailed Jul. 6, 2020, 8 pages.

(56) **References Cited**

OTHER PUBLICATIONS

International Patent Application No. PCT/US20/22848 "International Preliminary Report on Patentability" issued Sep. 16, 2021, 9 pages.
International Patent Application No. PCT/US20/22848 "International Search Report and Written Opinion" mailed Jul. 3, 2020, 12 pages.
International Patent Application No. PCT/US20/22849 "International Preliminary Report on Patentability" issued Sep. 16, 2021, 6 pages.
International Patent Application No. PCT/US20/22849 "International Search Report and Written Opinion" mailed Jul. 8, 2020, 9 pages.
International Patent Application No. PCT/US20/22850 "International Preliminary Report on Patentability" issued Sep. 16, 2021, 7 pages.
International Patent Application No. PCT/US20/22850 "International Search Report and Written Opinion" mailed Jul. 21, 2020, 9 pages.
International Patent Application No. PCT/US20/22851 "International Preliminary Report on Patentability" issued Sep. 16, 2021, 7 pages.
International Patent Application No. PCT/US20/22851 "International Search Report and Written Opinion" mailed Jun. 29, 2020, 10 pages.
International Patent Application No. PCT/US20/22852 "International Preliminary Report on Patentability" issued Sep. 16, 2021, 6 pages.
International Patent Application No. PCT/US20/22852 "International Search Report and Written Opinion" mailed Jul. 21, 2020, 9 pages.
Janzen Johan et al: "Partitioning GPUs for Improved Scalability", 2016 28th International Symposium On Computer Architecture and High Performance Computing (SBAC-PAD), IEEE, Oct. 26, 2016 (Oct. 26, 2016), pp. 42-49, XP033028008.
Kaseridis Dimitris, et al., "Minimalist open-page: A DRAM page-mode scheduling polidy for the many-core era", 2011 44th Annual IEEE/ACM International Symposium on Michoarechtecture ACM, Dec. 3, 2011, p. 24-35, XP033064864.
Li Bingchao et al: "Elastic-Cache: GPU Cache Architecture for Efficient Fine- and Coarse-Grained Cache-Line Management", 2017 IEEE International Parallel and Distributed Processing Symposium (IPDPS), IEEE, May 29, 2017 (May 29, 2017), pp. 82-91, XP033113921.
Luo Cheng et al., "An Efficient Task Partitioning and Scheduling Method for Symmetric Multiple GPU Architecture", 2013 12th IEEE International Conference On Trust, Security and Privacy in Computing and Communications, IEEE, Jul. 16, 2013 (Jul. 16, 2013), p. 1133-1142, XP032529503.
Mark Harris: "Mixed-Precision Programming 1-5 with CUDA 8", Oct. 19, 2016, XP055509917, 11 pages.
Meng, J. et al. Dynamic wrap subdivision for integrated branch and memory divergence tolerance. ACM SIGARCH Computer Architecture News, Jun. 2010 [online], [retrieved on Feb. 16, 2021]. Retrieved from the internet.
U.S. Appl. No. 16/432,402 "Non-Final Office Action" mailed Jul. 10, 2020, 17 pages.
U.S. Appl. No. 16/432,402 "Notice of Allowance" mailed Jun. 23, 2021, 5 pages.
U.S. Appl. No. 17/064,427 "Final Office Action" mailed Jan. 21, 2021, 13 pages.
U.S. Appl. No. 17/064,427 "Non-Final Office Action" mailed Dec. 7, 2020, 11 pages.
U.S. Appl. No. 17/064,427 "Notice of Allowability" mailed Jun. 10, 2021, 2 pages.
U.S. Appl. No. 17/064,427 "Notice of Allowance" mailed May 12, 2021, 7 pages.
U.S. Appl. No. 17/095,544 "Notice of Allowability" mailed Feb. 10, 2023, 2 pages.
U.S. Appl. No. 17/095,544 "Notice of Allowance" mailed Jan. 31, 2023, 10 pages.
U.S. Appl. No. 17/095,590 "Restriction Requirement" mailed Feb. 9, 2023, 5 pages.
U.S. Appl. No. 17/122,905 "Final Office Action" mailed Apr. 1, 2022, 29 pages.
U.S. Appl. No. 17/122,905 "Final Office Action" mailed Dec. 27, 2022, 23 pages.
U.S. Appl. No. 17/122,905 "Final Office Action" mailed May 11, 2021, 23 pages.
U.S. Appl. No. 17/122,905 "Non-Final Office Action" mailed Aug. 30, 2022, 26 pages.
U.S. Appl. No. 17/122,905 "Non-Final Office Action" mailed Feb. 17, 2021, 20 pages.
U.S. Appl. No. 17/122,905 "Non-Final Office Action" mailed Nov. 16, 2021, 23 pages.
U.S. Appl. No. 17/169,232 "Notice of Allowability" mailed Jun. 14, 2021, 3 pages.
U.S. Appl. No. 17/169,232 "Notice of Allowance" mailed Apr. 7, 2021, 3 pages.
U.S. Appl. No. 17/303,654 "Non-Final Office Action" mailed Oct. 31, 2022, 15 pages.
U.S. Appl. No. 17/303,654 "Notice of Allowability" mailed Feb. 27, 2023, 2 pages.
U.S. Appl. No. 17/303,654 "Notice of Allowance" mailed Feb. 14, 2023, 8 pages.
U.S. Appl. No. 17/304,092 "Non-Final Office Action" mailed Oct. 28, 2021, 17 pages.
U.S. Appl. No. 17/304,092 "Notice of Allowability" mailed Apr. 20, 2022, 3 pages.
U.S. Appl. No. 17/304,092 "Notice of Allowability" mailed Feb. 28, 2022, 2 pages.
U.S. Appl. No. 17/304,092 "Notice of Allowance" mailed Feb. 11, 2022, 9 pages.
U.S. Appl. No. 17/305,355 "Non-Final Office Action" mailed Nov. 5, 2021, 28 pages.
U.S. Appl. No. 17/305,355 "Notice of Allowability" mailed May 4, 2022, 2 pages.
U.S. Appl. No. 17/305,355 "Notice of Allowance" mailed Feb. 23, 2022, 7 pages.
U.S. Appl. No. 17/428,527 "Non-Final Office Action" mailed Mar. 30, 2023, 20 pages.
U.S. Appl. No. 17/428,539 "Non-Final Office Action" mailed Jan. 5, 2023, 14 pages.
U.S. Appl. No. 17/429,873 "Non-Final Office Action" mailed Jan. 9, 2023, 13 pages.
U.S. Appl. No. 17/429,873 "Final Office Action" mailed Apr. 26, 2023, 14 pages.
U.S. Appl. No. 17/430,574 "Non-Final Office Action" mailed Jan. 23, 2023, 8 pages.
U.S. Appl. No. 17/732,308 "Non-Final Office Action" mailed Jul. 27, 2022, 10 pages.
U.S. Appl. No. 17/732,308 "Notice of Allowability" mailed Mar. 3, 2023, 2 pages.
U.S. Appl. No. 17/732,308 "Notice of Allowance" mailed Dec. 6, 2022, 5 pages.
U.S. Appl. No. 17/827,067 "Non-Final Office Action" mailed Nov. 25, 2022, 25 pages.
U.S. Appl. No. 17/827,067 "Notice of Allowance" mailed Mar. 2, 2023, 8 pages.
U.S. Appl. No. 17/834,482 "Notice of Allowance" mailed Mar. 8, 2023, 7 pages.
U.S. Appl. No. 17/839,856 "Non-Final Office Action" mailed Feb. 10, 2023, 15 pages.
U.S. Appl. No. 17/430,574 Final Office Action mailed May 24, 2023, 9 pages.
Yin Jieming, et al., "Modular Routing Design for Chiplet-Based Systems", 2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA), IEEE, Jun. 1, 2018, pp. 726-738, XP033375532.
Young Vinson et al: "Combining HW/SW Mechanisms to Improve NUMA Performance of Multi-GPU Systems", 2018 51st Annual

(56) **References Cited**

OTHER PUBLICATIONS

IEEE/ACM International Symposium On Microarchitecture (MICRO), IEEE, Oct. 20, 2018 (Oct. 20, 2018), pp. 339-351, XP033473308.
Brunie, "Mixed-precision Fused Multiply and Add", https://ens-lyon.hal.science/ensl-00642157, Nov. 17, 2011, 7 pages.
Decision to Grant for CN Application No. 202110224132.2, mailed Jan. 5, 2024, 7 pages.
Decision to Grant for JP Application No. 2022-104265, Dec. 5, 2023, 2 pages.
Extended European Search Report for EP Application No. 23197619.2, Jan. 5, 2024, 12 pages.
Final OA for U.S. Appl. No. 17/590,362, Dec. 4, 2023, 30 pages.
Final Office Action for U.S. Appl. No. 17/674,703, mailed Dec. 13, 2023, 26 pages.
Notice of Allowance issued in U.S. Appl. No. 17/429,873, mailed Jan. 11, 2024, 16 pages.
Office Action for CN 202110214543.3, received Jan. 18, 2024, 12 pages, no translation available.
Office Action for U.S. Appl. No. 17/428,216, mailed Jan. 1, 2024, 29 pages.
Communication pursuant to Article 94(3) for EP22210195, mailed Oct. 4, 2023, 7 pages.
Final Office Action for U.S. Appl. No. 17/428,529, mailed Oct. 20, 2023, 20 pages.
Final Office Action for U.S. Appl. No. 17/430,611 mailed Oct. 26, 2023, 11 pages.
Hong, et al., "Attache: Towards Ideal Memory Compression by Mitigating Metadata Bandwidth Overheads.", 2018 (Year: 2018).
Notice of Allowance for U.S. Appl. No. 17/310,540, mailed Nov. 21, 2023, 8 pages.
Notice of Allowance for U.S. Appl. No. 17/849,201), mailed Oct. 2, 2023, 9 pages.
Notice of Reasons for Refusal in JP Application No. 2021-544544, Mailed Oct. 31, 2023, 4 pages.
Office Action for CN202110906984, 6 pages, Nov. 21, 2023.
Office Action for U.S. Appl. No. 17/428,530, mailed Nov. 14, 2023, 7 pages.
U.S. Appl. No. 17/429,277 "Notice of Allowance" mailed Oct. 31, 2023, 11 pages.
U.S. Appl. No. 17/429,291, Final Office Action, mailed Sep. 28, 2023.
U.S. Appl. No. 17/430,963 Non-Final OA mailed Sep. 28, 2023, 18 pages.
Communication pursuant to Article 94(3) for EP 22 198 615.1, received on Apr. 14, 2023, 5 pages.
Non-Final OA Issued in U.S. Appl. No. 17/429,277 mailed Jul. 7, 2023, 9 pages.
Office Action for JP 2021-547450, mailed Jun. 6, 2023, 15 pages.
Office Action for U.S. Appl. No. 17/095,590, mailed Jul. 28, 2023, 23 pages.
Office Action for U.S. Appl. No. 17/428,216, mailed Jul. 7, 2023, 21 pages.
Office Action for U.S. Appl. No. 17/428,523, mailed Aug. 3, 2023, 12 pages.
Office Action for U.S. Appl. No. 17/428,534, mailed Jul. 17, 2023, 20 pages.
Office Action for U.S. Appl. No. 17/428,529, mailed Jun. 7, 2023, 18 pages.
U.S. Appl. No. 17/674,703 Non-Final Office Action mailed Jul. 14, 2023, 23 pages.
U.S. Appl. No. 17/430,611 "Non-Final Office Action" mailed Jun. 2, 2023, 11 pages.
Nicholas Wilt, "The CUDA Handbook; A Comprehensive Guide to GPU Programming", Book, Jun. 22, 2013, pp. 41-57, Addison-Wesley Professional, Boston, MA.
Non-Final Office Action in U.S. Appl. No. 17/429,291, mailed May 22, 2023, 20 pages.
Notification of Grant of Taiwan Patent Application No. 107105949 mailed Apr. 14, 2022, 43 pages.
Nvidia: "Nvidia Tesla P100 Nvidia Tesla P100 Whitepaper", Jun. 21, 16, pp. 1-45, downloaded fromn https://images.nvidia.com/content/pdf/tesla/whitepaper/pascal-architecture-white paper.pdf.
Nvidia's Next Generation CUDA Compute Architecture: Fermi. Datasheet [online]. NVIDIA, 2009 [retrieved on Oct. 3, 2019]. Retrieved from the internet.
Office Action and Search Report for Taiwan Patent Application No. 108141986 mailed Mar. 4, 2022, 24 pages (including translation).
Office Action for U.S. Appl. No. 17/430,611 mailed Jun. 2, 2023, 10 pages.
Office Action for U.S. Appl. No. 17/122,905, mailed Apr. 25, 2023, 24 pages.
Office Action for U.S. Appl. No. 17/849,201, mailed Jun. 1, 2023, 6 pages.
Office Action from DE Application No. 112020003700.2, mailed Mar. 16, 2023, 18 pages (no translation included).
Office Action, "Translation of Notice" for TW Application No. 111130374, mailed Mar. 9, 2023, 5 pages.
Ogg, et al., "Improved Data Compression for Serial Interconnected Network on Chip through Unused Significant Bit Removal", VLSO Design, 2006. Held jointly with 5th International Conference on Embedded Systems and Design. 19th International Conference on, Piscataway, NJ, USA, IEEE Jan. 3, 2006, pp. 525-529, XP010883136. ISBN: 978-0-7695-2502-0.
Olivares-Amaya et al. "Accelerating Correlated Quantum Chemistry Calculations Using Graphical Processing Units and a Mixed Precision Matrix Multipication Library" Journal of Chemical Theory and Computation, vol. 6, No. 1, Oct. 13, 2009, pp. 135-144, American Chemical Society.
Pearson Carl et al: "NUMA-Aware Data-Transfer Measurements for Power/NVLink Multi-GPU Systems", Jan. 25, 2019 (Jan. 25, 2019), Robocup 2008: Robot Soccer World Cup XII; pp. 448-454, XP047501461, ISBN: 978-3-319-10403-4 [retrieved on Jan. 25, 2019] page 452, line 14-p. 453, line 6.
Ross, et al. "Intel Processor Graphics: Architecture & Programming", Power Point Presentation, Aug. 2015, 78 pages, Intel Corporation, Santa Clara, CA.
Shane Cook, "CUDA Programming", Book, 2013, pp. 37-52, Chapter 3, Elsevier Inc., Amsterdam Netherlands.
Stephen Junkins, "The Compute Architecture of Intel Processor Graphics Gen9", paper, Aug. 14, 2015, 22 pages, Version 1.0, Intel Corporation, Santa Clara, CA.
Tawain IPO Search Report for TW Application No. 111130374 mailed Mar. 9, 2023 1 page.
Temam O et al: "Software assistance for data caches", High-Performance Computer Architecture, 1995. Proceedings., First IEEE Symposium On Raleigh, NC, USA Jan. 22-25, 1995, Los Alamitos, CA, USA.IEEE COMPUT. SOC, Jan. 22, 1995 (Jan. 22, 1995), pp. 154-163, XP010130123.
Tong Chen et al: "Prefetching irregular references for software cache on cell", Sixth Annual IEEE/ACM International Symposium On Code Generation and 1OPTIMIZATION (CGO 08). Boston, MA, USA, Apr. 5-9, 2008, New York, NY: ACM, 2 Penn Plaza, Suite 701 New York NY 10121-0701 USA, Apr. 6, 2008 (Apr. 6, 2008), pp. 155-164, XP058192632.
U.S. Appl. No. 15/494,773 "Final Office Action" mailed Oct. 12, 2018, 18 pages.
U.S. Appl. No. 15/494,773 "Non-Final Office Action" mailed Apr. 18, 2018, 14 pages.
U.S. Appl. No. 15/494,773 "Notice of Allowability" mailed Jul. 31, 2019, 2 pages.
U.S. Appl. No. 15/494,773 "Notice of Allowance" mailed Jun. 19, 2019, 8 pages.
U.S. Appl. No. 15/787,129 "Non-Final Office Action" mailed Aug. 27, 2018, 9 pages.
U.S. Appl. No. 15/787,129 "Non-Final Office Action" mailed Mar. 7, 2019, 10 pages.
U.S. Appl. No. 15/787,129 "Notice of Allowance" mailed Jul. 10, 2019, 8 pages.
U.S. Appl. No. 15/819,152 "Final Office Action" mailed Nov. 19, 2018, 9 pages.
U.S. PaU.S. Appl. No. 15/819,152 "Non-Final Office Action" mailed Feb. 28, 2018, 7 pages.

(56) **References Cited**

OTHER PUBLICATIONS

U.U.S. Appl. No. 15/819,152 "Notice of Allowability" mailed Apr. 11, 2019, 2 pages.
U.S. Appl. No. 15/819,152 "Notice of Allowance" mailed Mar. 19, 2019, 5 pages.
U.S. Appl. No. 15/819,167 "Final Office Action" mailed Aug. 19, 2020, 22 pages.
U.S. Appl. No. 15/819,167 "Final Office Action" mailed Jun. 2, 2021, 20 pages.
U.S. Appl. No. 15/819,167 "Final Office Action" mailed Oct. 1, 2018, 17 pages.
U.S. Appl. No. 15/819,167 "Final Office Action" mailed Oct. 18, 2019, 17 pages.
U.S. Appl. No. 15/819,167 "Non-Final Office Action" mailed Dec. 14, 2021, 20 pages.
U.S. Appl. No. 15/819,167 "Non-Final Office Action" mailed Jan. 26, 2021, 19 pages.
U.S. Appl. No. 15/819,167 "Non-Final Office Action" mailed Mar. 1, 2018, 12 pages.
U.S. Appl. No. 15/819,167 "Non-Final Office Action" mailed Mar. 31, 2020, 19 pages.
U.S. Appl. No. 15/819,167 "Non-Final Office Action" mailed May 2, 2019, 15 pages.
U.S. Appl. No. 15/819,167 "Notice of Allowability" mailed Jul. 7, 2022, 4 pages.
U.S. Appl. No. 15/819,167 "Notice of Allowance" mailed Apr. 12, 2022, 7 pages.
U.S. Appl. No. 16/227,645 "Final Office Action" mailed Jul. 9, 2021, 23 pages.
U.S. Appl. No. 16/227,645 "Final Office Action" mailed Sep. 8, 2020, 20 pages.
U.S. Appl. No. 16/227,645 "Non-Final Office Action" mailed Apr. 8, 2020, 19 pages.
U.S. Appl. No. 16/227,645 "Non-Final Office Action" mailed Dec. 17, 2021, 23 pages.
U.S. Appl. No. 16/227,645 "Non-Final Office Action" mailed Feb. 22, 2021, 23 pages.
U.S. Appl. No. 16/227,645 "Notice of Allowability" mailed Aug. 4, 2022, 2 pages.
U.S. Appl. No. 16/227,645 "Notice of Allowance" mailed May 25, 2022, 7 pages.
U.S. Appl. No. 16/432,402 "Final Office Action" mailed Feb. 19, 2021, 9 pages.
Communication pursuant to Article 94(3) EPC for EP Application 20718900.2, mailed Apr. 10, 2024, 6 pages.
Final OA for U.S. Appl. No. 17/115,989, mailed Mar. 25, 2024, 20 pages.
Final Office Action U.S. Appl. No. 17/430,963 mailed Apr. 19, 2024, 12 pages.
Intent to Grant for EP Application No. 22210195.8, mailed Mar. 28, 2024, 5 pages.
Intention to Grant EP 21192702.5, mailed Apr. 2, 2024, 5 pages.
Notice for Eligibility of Grant for SG Application No. 11202107290Q, 4 pages, Mar. 5, 2024.
Notice of Allowance for JP Application No. 2021-547288, 4 pages, Apr. 25, 2024.
Notice of Allowance for TW 109139554, Mar. 11, 2024, 3 pages.
Notice of Allowance for U.S. Appl. No. 17/590,362 mailed Apr. 3, 2024, 12 pages.
Notification of Issued Patent for TW Application No. 112124508 mailed Mar. 5, 2024, 54 pages.
Notification of Publication of CN202311777921.4, 3 pages, Mar. 12, 2024.
Office Action for CN202110250102.9, mailed Apr. 16, 2024, 13 pages.
Office Action for U.S. Appl. No. 17/428,233 mailed Apr. 11, 2024, 16 pages.
Office Action for U.S. Appl. No. 17/428,530, mailed Mar. 19, 2024, 8 pages.
Office Action for U.S. Appl. No. 17/961,833, mailed Apr. 18, 2024, 10 pages.
Publication Notification for CN202311810112.9, Mar. 26, 2024, 3 pages.
Zhang et al., "Fabrication cost analysis for 2D, 2.5D, and 3D IC designs," 2011 IEEE International 3D Systems Integration Conference (3DIC), 2011 IEEE International, Osaka, Japan, 2012, pp. 1-4, doi: 10.1109/3DIC.2012.6263032.
Communication pursuant to Article 94(3) for EP20718907.7, mailed Feb. 5, 2024, 6 pages.
Notice of Allowance for U.S. Appl. No. 17/428,527, mailed Jan. 26, 2024, 10 pages.
Notice of Reasons for Refusal issued in JP Application No. 2021-544279, mailed Feb. 27, 2024, 7 pages including translation.
Office Action for U.S. Appl. No. 17/862,739, mailed Feb. 28, 2024, 21 pages.
Notice of Publication for CN202310748237.7, mailed Oct. 7, 2023, 3 pages.
Notice of Allowance for CN20190973729, Notice on Grant of Patent Right, mailed Oct. 28, 2023, 4 pages.
Korean Office Action for KR2021-7025888, mailed Jul. 29, 2024, 7 pages, No translation available at this time.
Notice of Allowance for CN202011533036.8, 8 pages, Jun. 24, 2024.
Notice of Allowance for CN202110250102.9, Jun. 20, 2024, 7 pages.
Notice of Publication for CN202410567634.9, mailed Aug. 1, 2024, 3 pages.
AC8218-PCT-US Non-Final Office Action mailed Jun. 7, 2024, 14 pages.
Balkesen et al., "Rapid: In-Memory Analytical Query Processing Engine with Extreme Performance per Watt," SIGMOD'18, Jun. 10-15, 2018, Industry 4: Graph databases & Query Processing on Modern Hardware, pp. 1407-1419. (Year: 2018).
Grant Notification for CN Application No. 202110256528.5, May 6, 2024, 9 pages.
Grant Notification for CN Application No. 2021109069843X, mailed Apr. 10, 2024, 9 pages.
Heo et al., "BOSS: Bandwidth-Optimized Search Accelerator for Storage-Class Memory," 2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA), pp. 279-291, (Year: 2021).
Intention to Grant EP Application No. 20718903.6, May 7, 2024, 5 pages.
JP Application No. 2021-544279 Decision to Grant and Documents Citing Families of 2021-544279 mailed May 29, 2024.
Notice of Allowance in U.S. Appl. No. 17/431,034 mailed May 16, 2024, 10 pages.
Notification of Publication for TW113103959, mailed May 20, 2024, 3 pages.
Publication of TW Application No. 113103959 (Pub No. 202420066), May 16, 2024, 3 pages.
Article 94(3) Communication for EP23181292.6 mailed Jun. 25, 2024, 5 pages.
Final Office Action in U.S. Appl. No. 17/430,041 mailed Jan. 6, 2025, 21 pages.
Japanese Application No. 2023-0223711 Decision to Grant mailed Jan. 7, 2025.
Notice of Allowance for CN Application No. 201810367462, Dec. 1, 2024, 5 pages.
Non-Final Office Action issued in U.S. Appl. No. 17/430,611, mailed Sep. 11, 2024, 11 pages.
Notice of Allowance and Notice of References Cited issued in U.S. Appl. No. 18/415,052, mailed Sep. 6, 2024, 9 pages.
Notice of Allowance for U.S. Appl. No. 17/428,529, mailed Aug. 28, 24, 9 pages.
Notice of Allowance for U.S. Appl. No. 18/532,245, mailed Sep. 17, 2024, 8 pages.
Office Action for U.S. Appl. No. 18/491,474, mailed Aug. 21, 2024, 7 pages.
Decision to Grant EP Application No. 20718903.6, mailed Sep. 12, 2024, 3 pages.

(56) **References Cited**

OTHER PUBLICATIONS

Office Action from KR Application No. 2021-07025864, mailed Aug. 20, 2024, 20 pages.
Notice of Allowance for JP Application No. 2020-189645, dispatched Sep. 3, 2024, 4 pages.
Certificate of Grant for AU Application No. 2020241262, 2 pages, granted on Apr. 24, 2025.
EP Intention to Grant for EP 20718909.3, Feb. 18, 2025, 5 pages.
Final Office Action in U.S. Appl. No. 18/647,549 mailed Apr. 30, 2025, 7 pages.
Non-Final Office Action issued in U.S. Appl. No. 17/430,611, mailed May 2, 2025, 17 pages.
Notice of Allowance for AC7977-PCT-US-C1, mailed Mar. 5, 2025, 11 pages.
Notice of Allowance for CN Application No. 202110214543.3, mailed Feb. 21, 2025, 5 pages.
Notice of Allowance for Korean Application No. 9-5-2025-038947914 mailed Apr. 22, 2025, 4 pages.
Office Action for KR Application No. 10-2021-7025888, Apr. 22, 2025, 8 pages.
Office Action for U.S. Appl. No. 18/305,904 Mar. 28, 25, 2025 pages.
Office Action for U.S. Appl. No. 18/490,593, Mar. 24, 2025, 25 pages.
Park, et al., "CNN Deep Learning Acceleration Algorithm for Mobile," Journal of KIIT. Vol. 16, No. 10, pp. 1-9, pISSN 1598-8619 (released on Oct. 31, 2019).
Search Report for Malaysian Patent No. PI 2021003764, 6 pages, May 28, 2025.
Notice of Publication for EP Application No. 25174786.1 mailed May 21, 25, Publication No. 4571498 (Jun. 18, 2025), 2 pages.
Notification of Publication for CN Application No. 202510085541.7 mailed May 21, 2025, 3 pages.
Non-Final Office Action in U.S. Appl. No. 18/626,775, mailed May 20, 2025, 26 pages.
Decision to Grant Application No. JP-2024-006026, Jun. 26, 2025, 4 pages.
EP Communication pursuant to Article 94(3) issued in EP 20 718 910.1 (Docket No. AC8072-PCTEP) mailed May 28, 2025.
Notice of Allowance for Application No. 113103959 (P116313-US-TW-D2-D1-D1-D1) Jun. 23, 2025, 3 pages.
TW Search Report for Application No. 113103959 Jun. 23, 2025, 2 pages.
Notice of Reasons of Refusal issued in JP Application No. 2024-102405, mailed Jun. 13, 2025, 5 pages.
Notice of Allowance for Indonesian Patent Application No. P-00202105790, mailed Jun. 12, 2025, 5 pages.
Notice of Allowance for CN Application No. 201810394160.7, mailed Jun. 18, 2025, 6 pages.
U.S. Appl. No. 17/430,041 Non-Final Office Action mailed Jun. 13, 2025, 27 pages.
Office Action for U.S. Appl. No. 18/915,492 mailed Jun. 4, 2025, 14 pages.
Gabriel Mounce et al., "Chiplet Based Approach for Heterogeneous Processing and Packaging Architectures," 2019, IEEE, pp. 1-12. (Year: 2019).
IPSJ SIG Technical Report, High-Performance Computing (HPC), 2018-HPC-164, 4, p. 1-6, Apr. 30, 2018, ISSN,2188-8841.
Non-Final Office Action in U.S. Appl. No. 18/779,461, mailed Jun. 12, 2025, 9 pages.
Non-Final Office Action issued in U.S. Appl. No. 18/738,785 mailed Jul. 23, 2025.

* cited by examiner

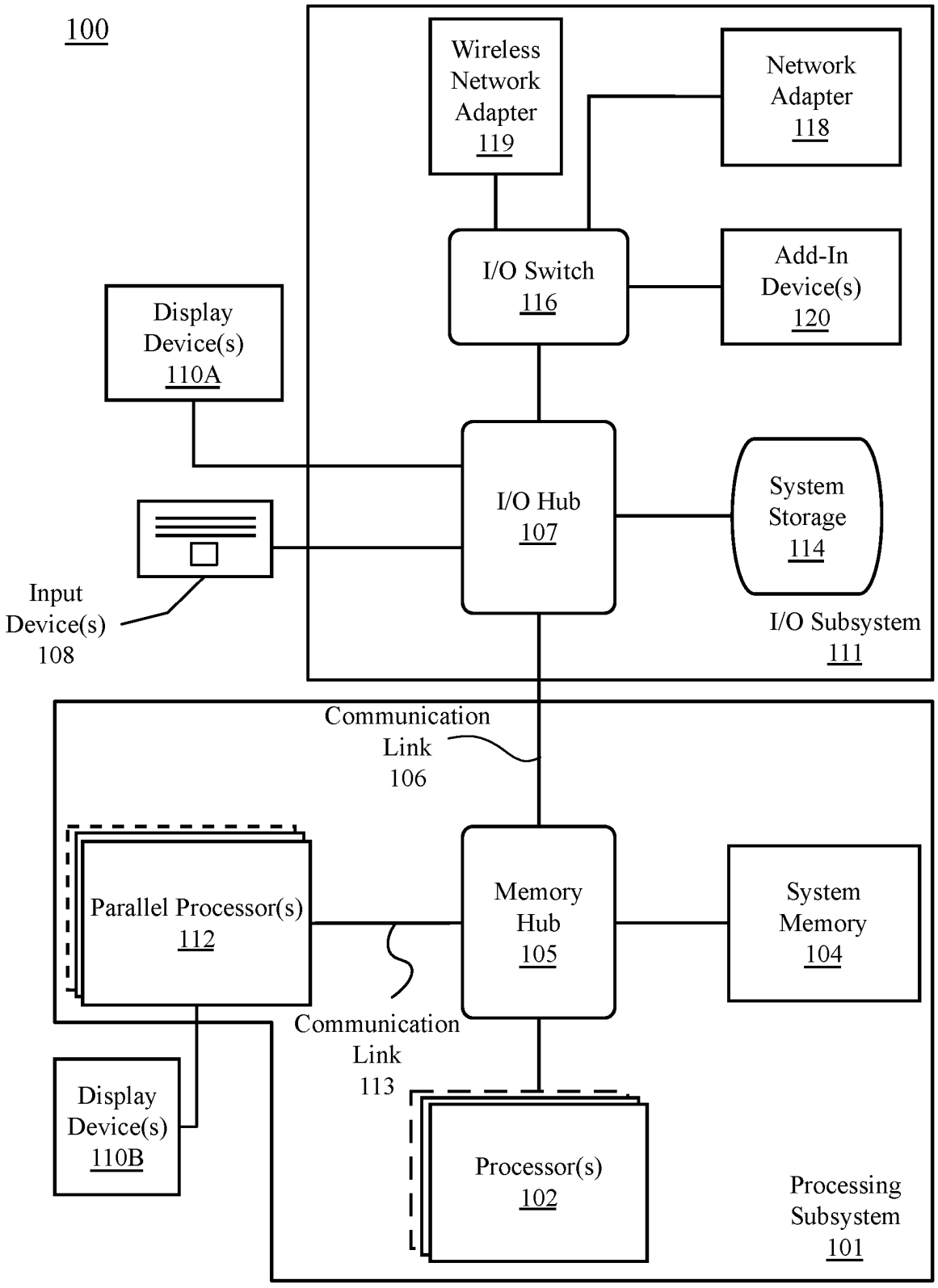

100
Wireless Network Adapter 119
Network Adapter 118
I/O Switch 116
Add-In Device(s) 120
Display Device(s) 110A
I/O Hub 107
System Storage 114
Input Device(s) 108
I/O Subsystem 111
Communication Link 106
Parallel Processor(s) 112
Memory Hub 105
System Memory 104
Communication Link 113
Display Device(s) 110B
Processor(s) 102
Processing Subsystem 101

FIG. 1

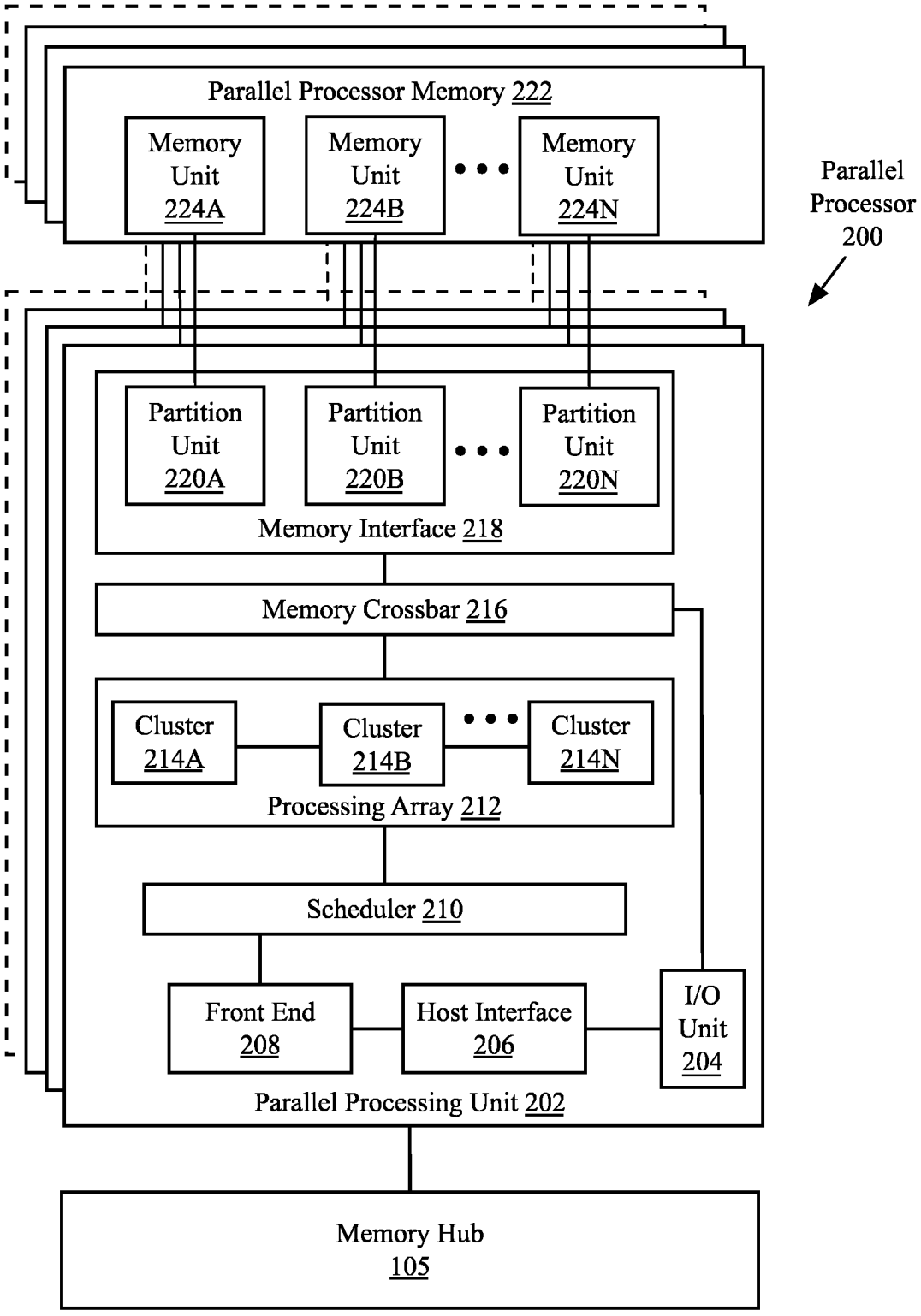

Parallel Processor Memory 222
Memory Unit 224A
Memory Unit 224B
Memory Unit 224N
Parallel Processor 200
Partition Unit 220A
Partition Unit 220B
Partition Unit 220N
Memory Interface 218
Memory Crossbar 216
Cluster 214A
Cluster 214B
Cluster 214N
Processing Array 212
Scheduler 210
Front End 208
Host Interface 206
I/O Unit 204
Parallel Processing Unit 202
Memory Hub 105

FIG. 2A

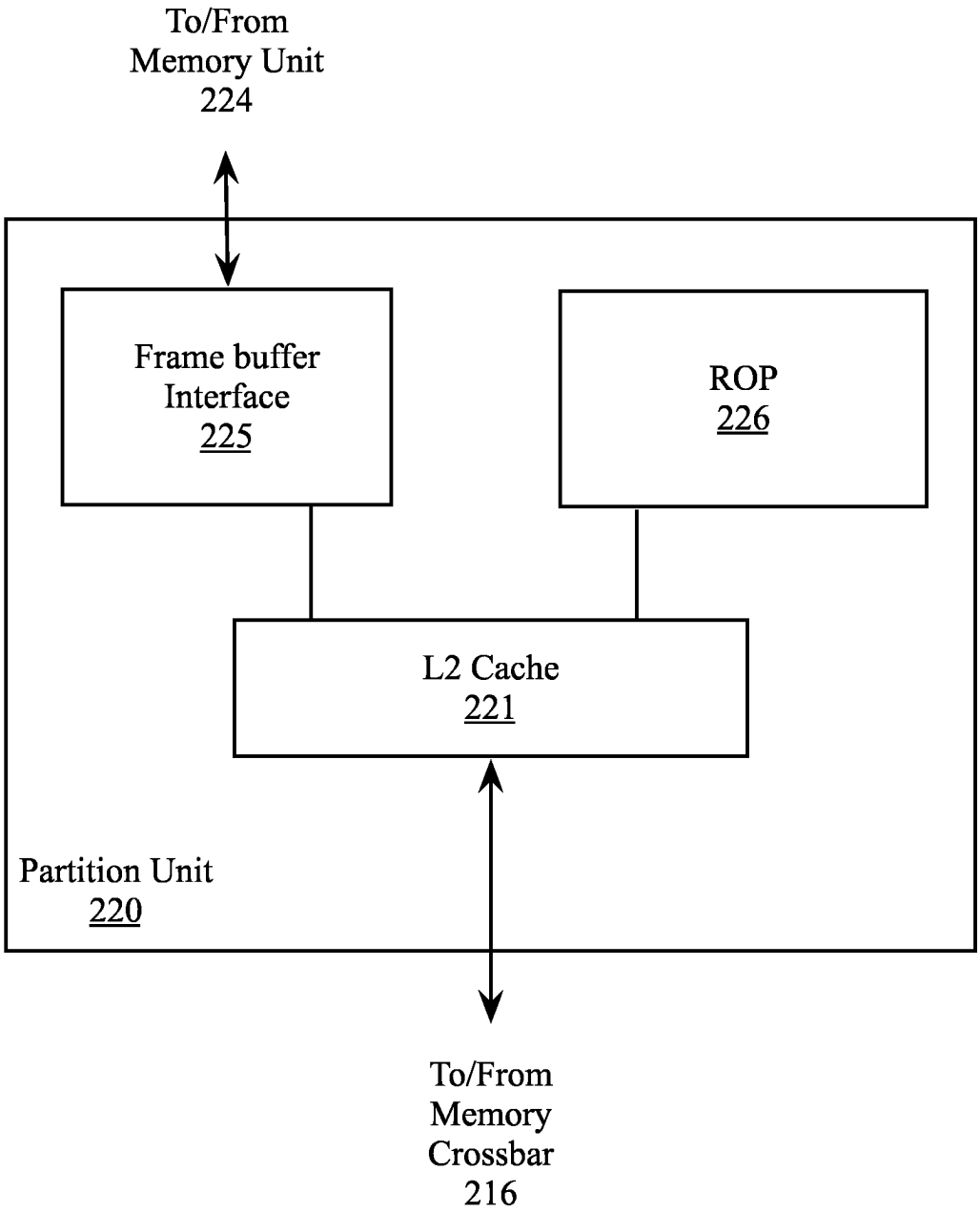

To/From
Memory Unit
224
Frame buffer
Interface
225
ROP
226
L2 Cache
221
Partition Unit
220
To/From
Memory
Crossbar
216

FIG. 2B

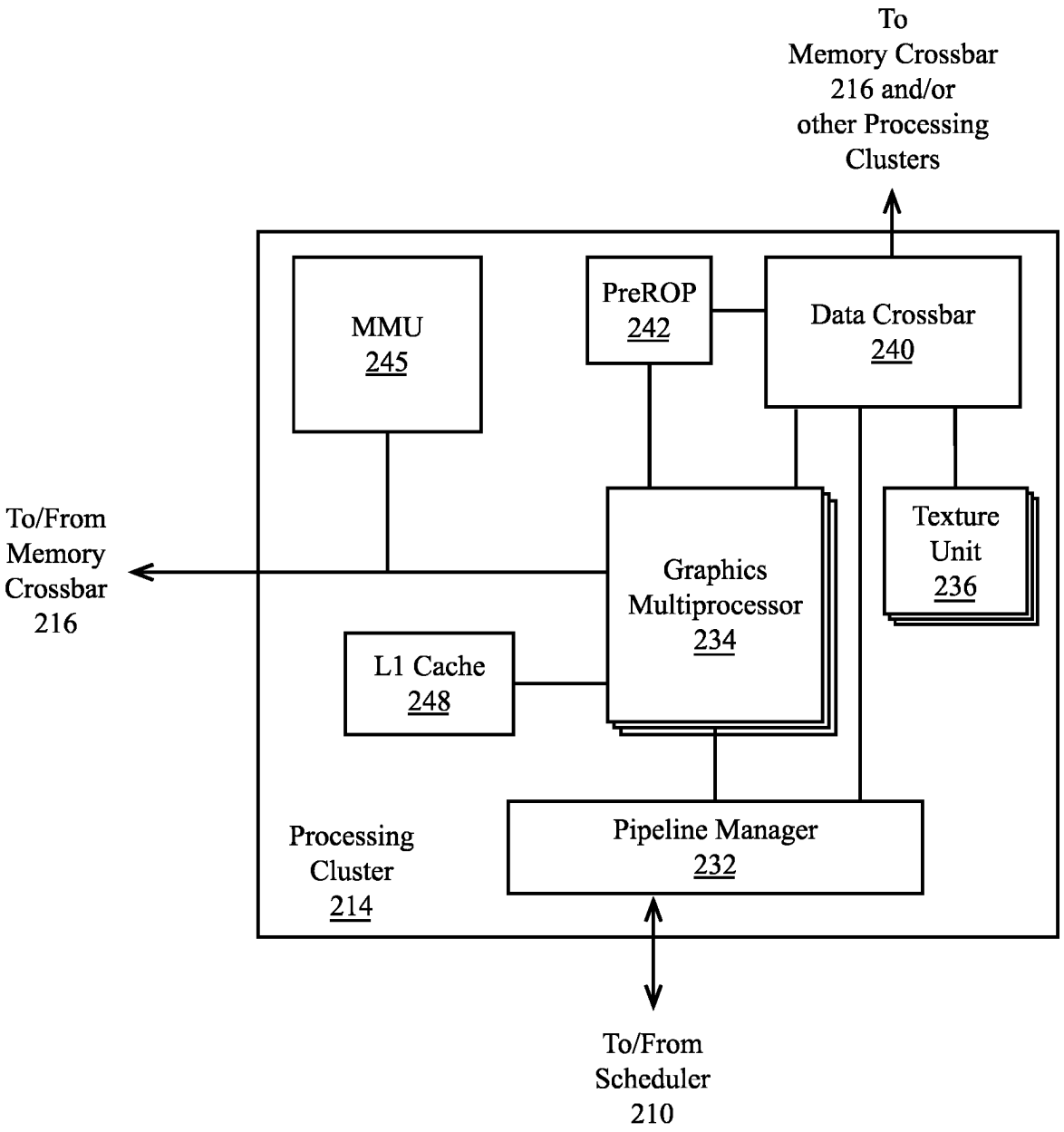

To
Memory Crossbar
216 and/or
other Processing
Clusters
MMU
245
PreROP
242
Data Crossbar
240
To/From
Memory
Crossbar
216
Graphics
Multiprocessor
234
Texture
Unit
236
L1 Cache
248
Pipeline Manager
232
Processing
Cluster
214
To/From
Scheduler
210

FIG. 2C

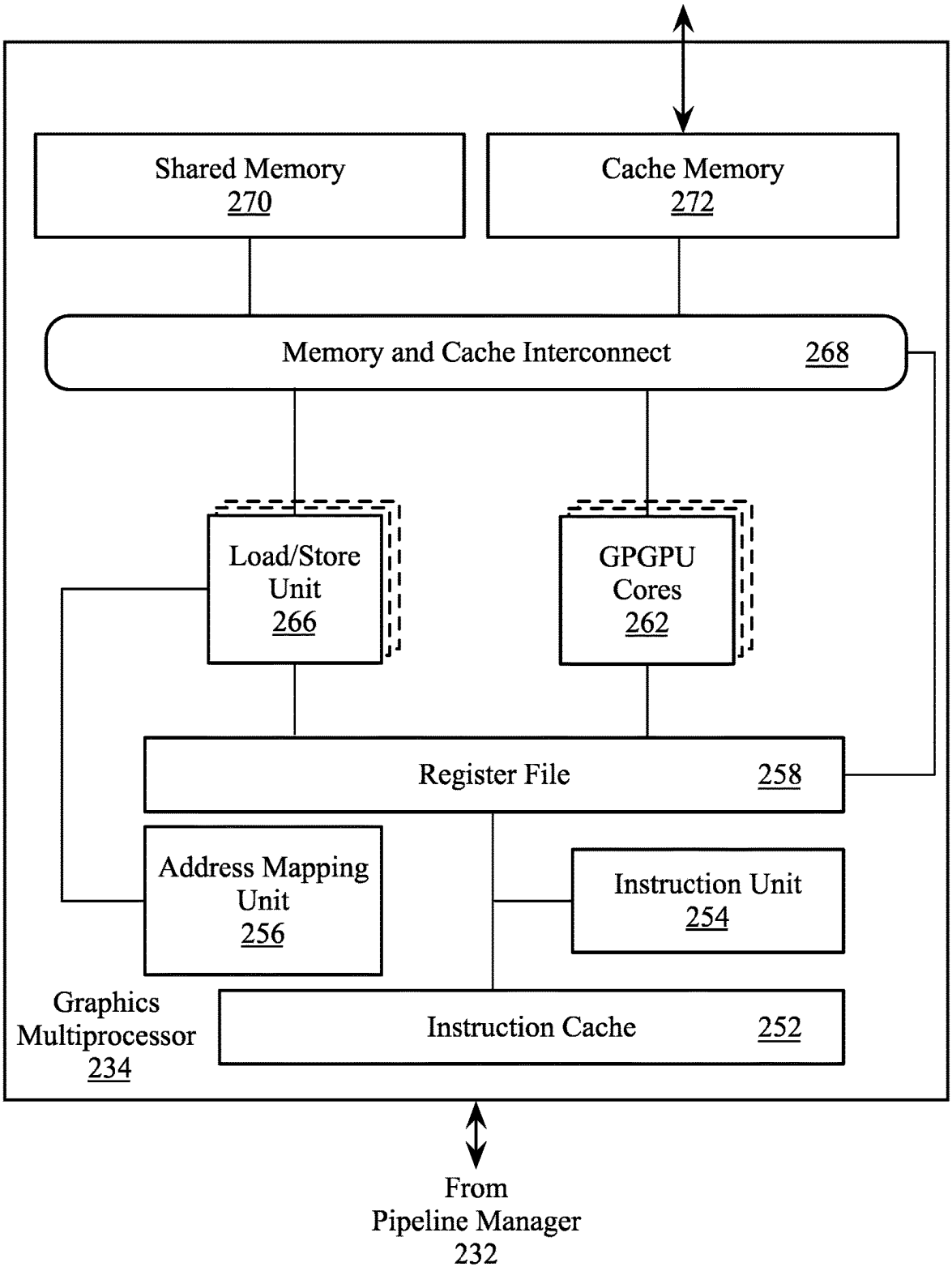

Shared Memory
270
Cache Memory
272
Memory and Cache Interconnect
268
Load/Store
Unit
266
GPGPU
Cores
262
Register File
258
Address Mapping
Unit
256
Instruction Unit
254
Graphics
Multiprocessor
234
Instruction Cache
252
From
Pipeline Manager
232

FIG. 2D

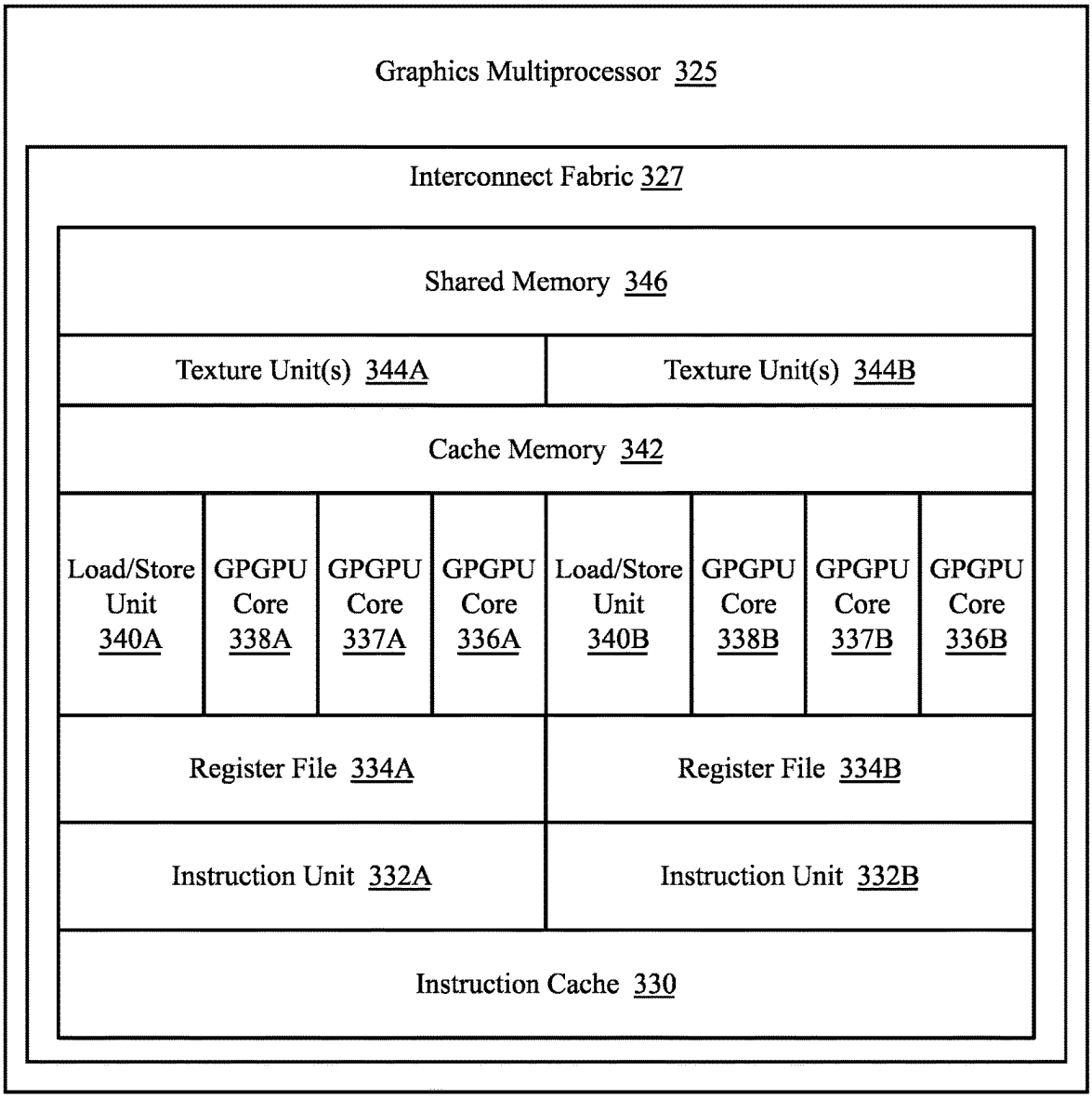

Graphics Multiprocessor 325
Interconnect Fabric 327
Shared Memory 346
Texture Unit(s) 344A
Texture Unit(s) 344B
Cache Memory 342
Load/Store Unit 340A
GPGPU Core 338A
GPGPU Core 337A
GPGPU Core 336A
Load/Store Unit 340B
GPGPU Core 338B
GPGPU Core 337B
GPGPU Core 336B
Register File 334A
Register File 334B
Instruction Unit 332A
Instruction Unit 332B
Instruction Cache 330

FIG. 3A

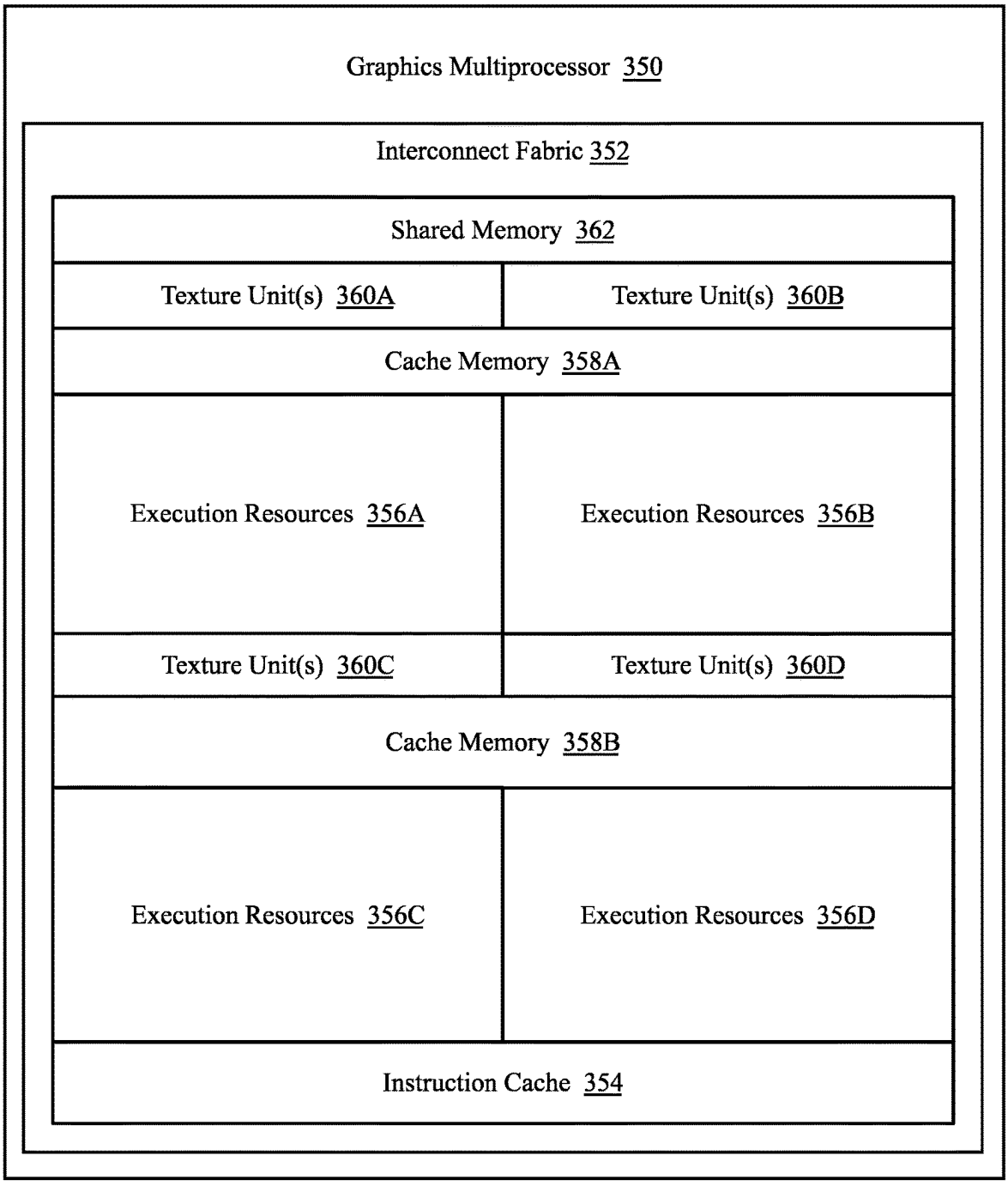

Graphics Multiprocessor 350
Interconnect Fabric 352
Shared Memory 362
Texture Unit(s) 360A
Texture Unit(s) 360B
Cache Memory 358A
Execution Resources 356A
Execution Resources 356B
Texture Unit(s) 360C
Texture Unit(s) 360D
Cache Memory 358B
Execution Resources 356C
Execution Resources 356D
Instruction Cache 354

FIG. 3B

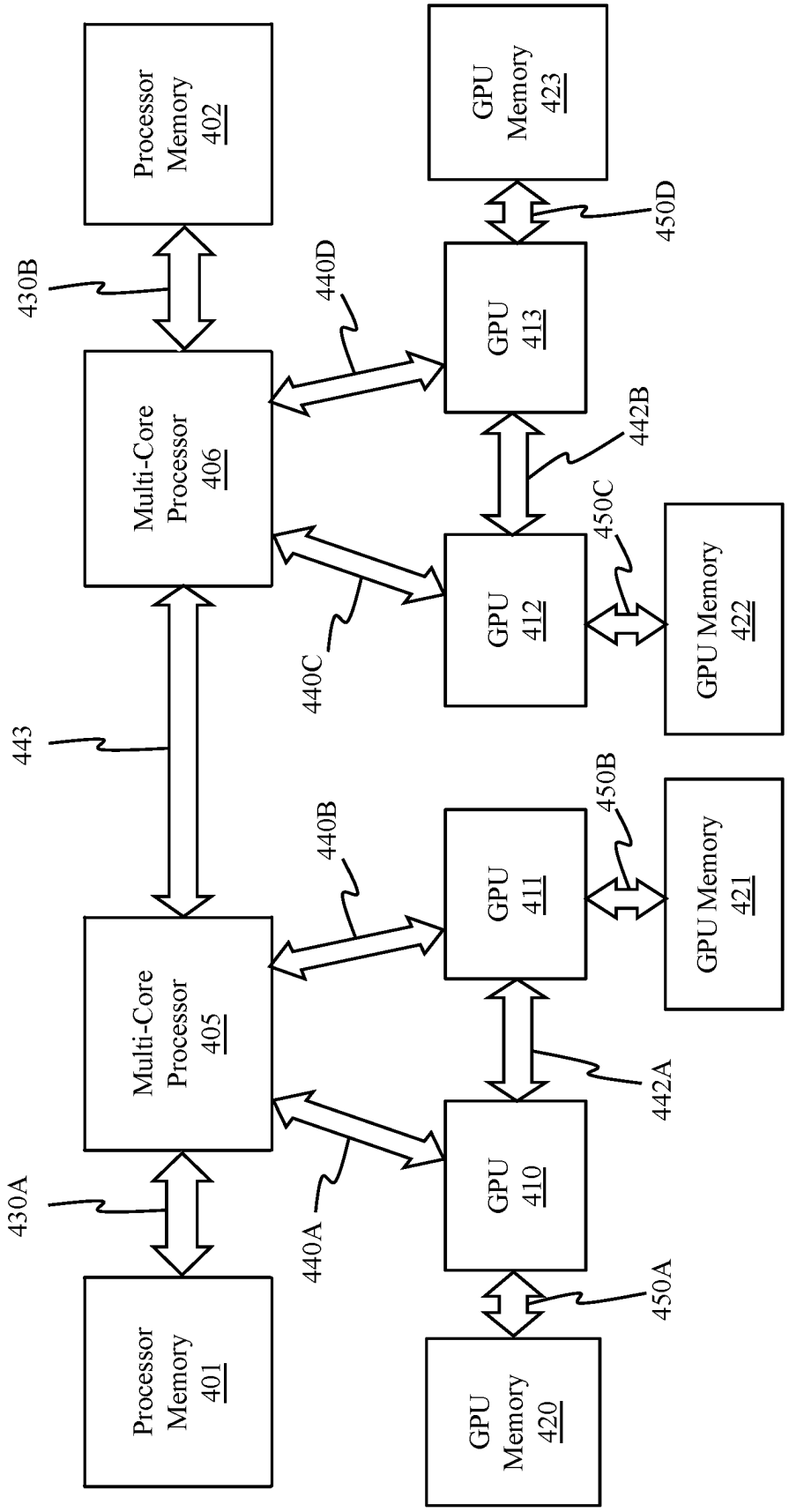

Processor Memory 401
Processor Memory 402
Multi-Core Processor 405
Multi-Core Processor 406
GPU 410
GPU 411
GPU 412
GPU 413
GPU Memory 420
GPU Memory 421
GPU Memory 422
GPU Memory 423
430A
430B
440A
440B
440C
440D
442A
442B
443
450A
450B
450C
450D

FIG. 4A

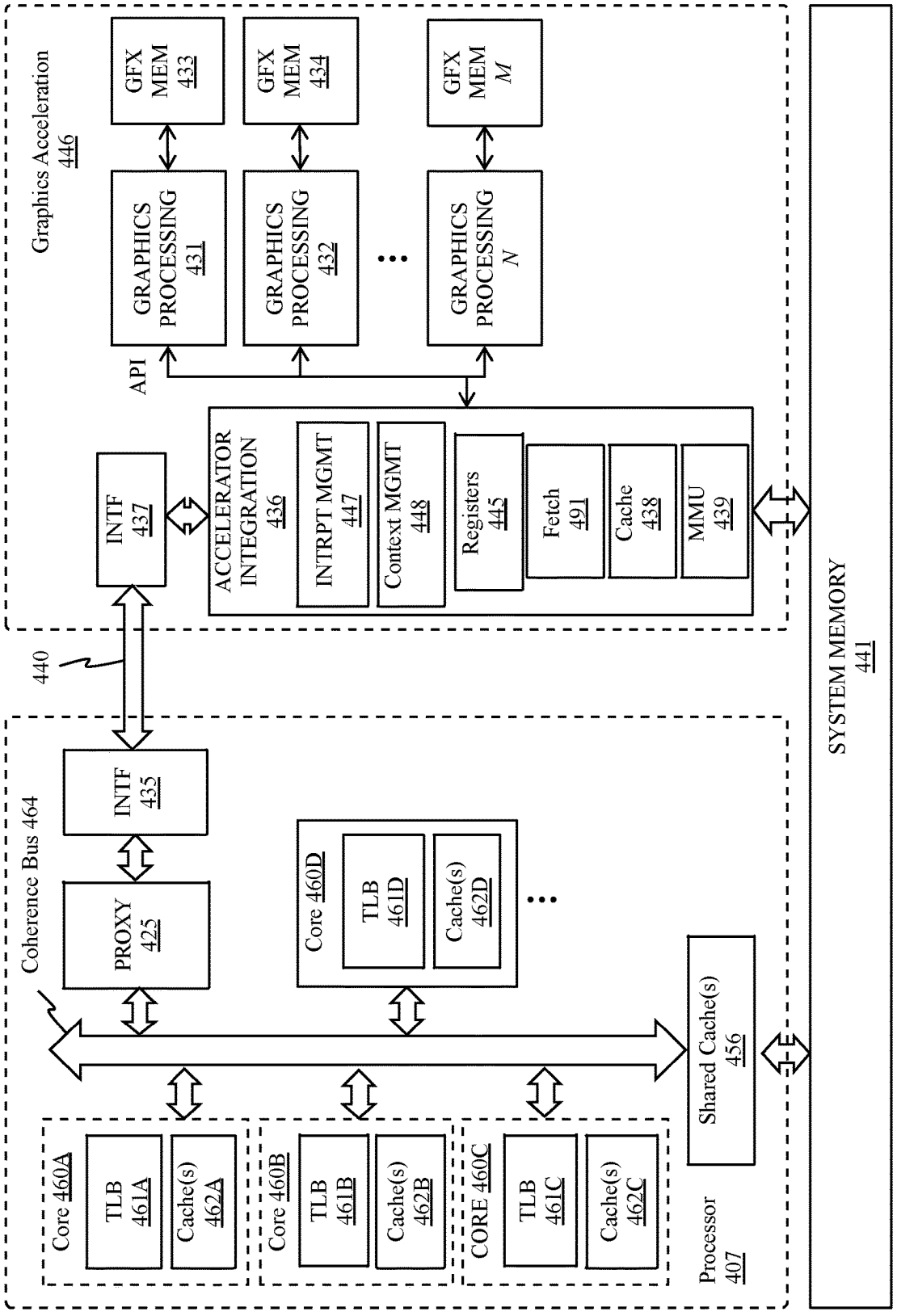

Graphics Acceleration 446
GFX MEM 433
GFX MEM 434
GFX MEM M
GRAPHICS PROCESSING 431
GRAPHICS PROCESSING 432
GRAPHICS PROCESSING N
API
INTF 437
ACCELERATOR INTEGRATION 436
INTRPT MGMT 447
Context MGMT 448
Registers 445
Fetch 491
Cache 438
MMU 439
440
SYSTEM MEMORY 441
Coherence Bus 464
INTF 435
PROXY 425
Core 460D
TLB 461D
Cache(s) 462D
Shared Cache(s) 456
Core 460A
TLB 461A
Cache(s) 462A
Core 460B
TLB 461B
Cache(s) 462B
CORE 460C
TLB 461C
Cache(s) 462C
Processor 407

FIG. 4B

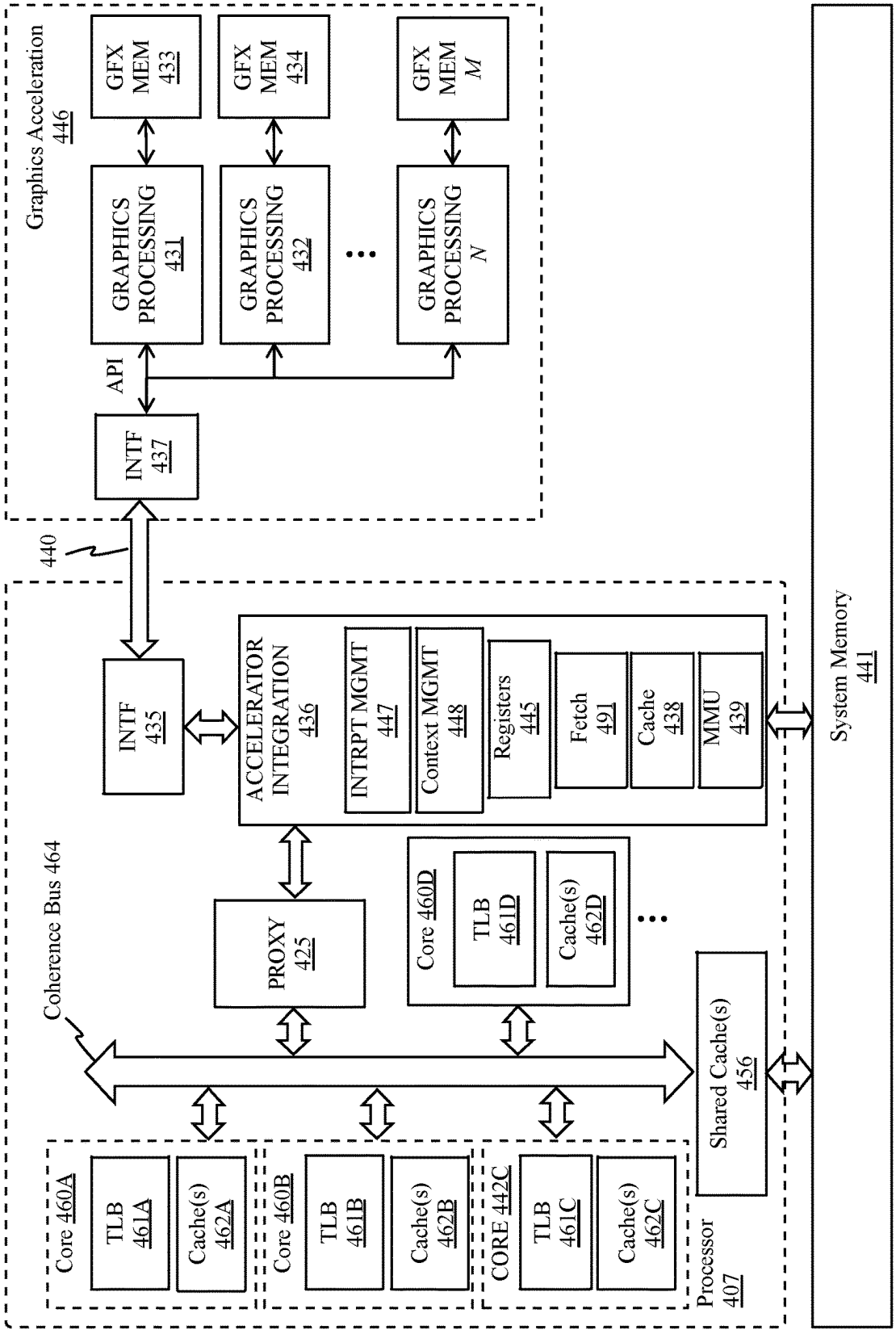

Graphics Acceleration 446
GFX MEM 433
GFX MEM 434
GFX MEM M
GRAPHICS PROCESSING 431
GRAPHICS PROCESSING 432
GRAPHICS PROCESSING N
API
INTF 437
440
INTF 435
ACCELERATOR INTEGRATION 436
INTRPT MGMT 447
Context MGMT 448
Registers 445
Fetch 491
Cache 438
MMU 439
System Memory 441
Coherence Bus 464
PROXY 425
Core 460D
TLB 461D
Cache(s) 462D
Shared Cache(s) 456
Core 460A
TLB 461A
Cache(s) 462A
Core 460B
TLB 461B
Cache(s) 462B
CORE 442C
TLB 461C
Cache(s) 462C
Processor 407

FIG. 4C

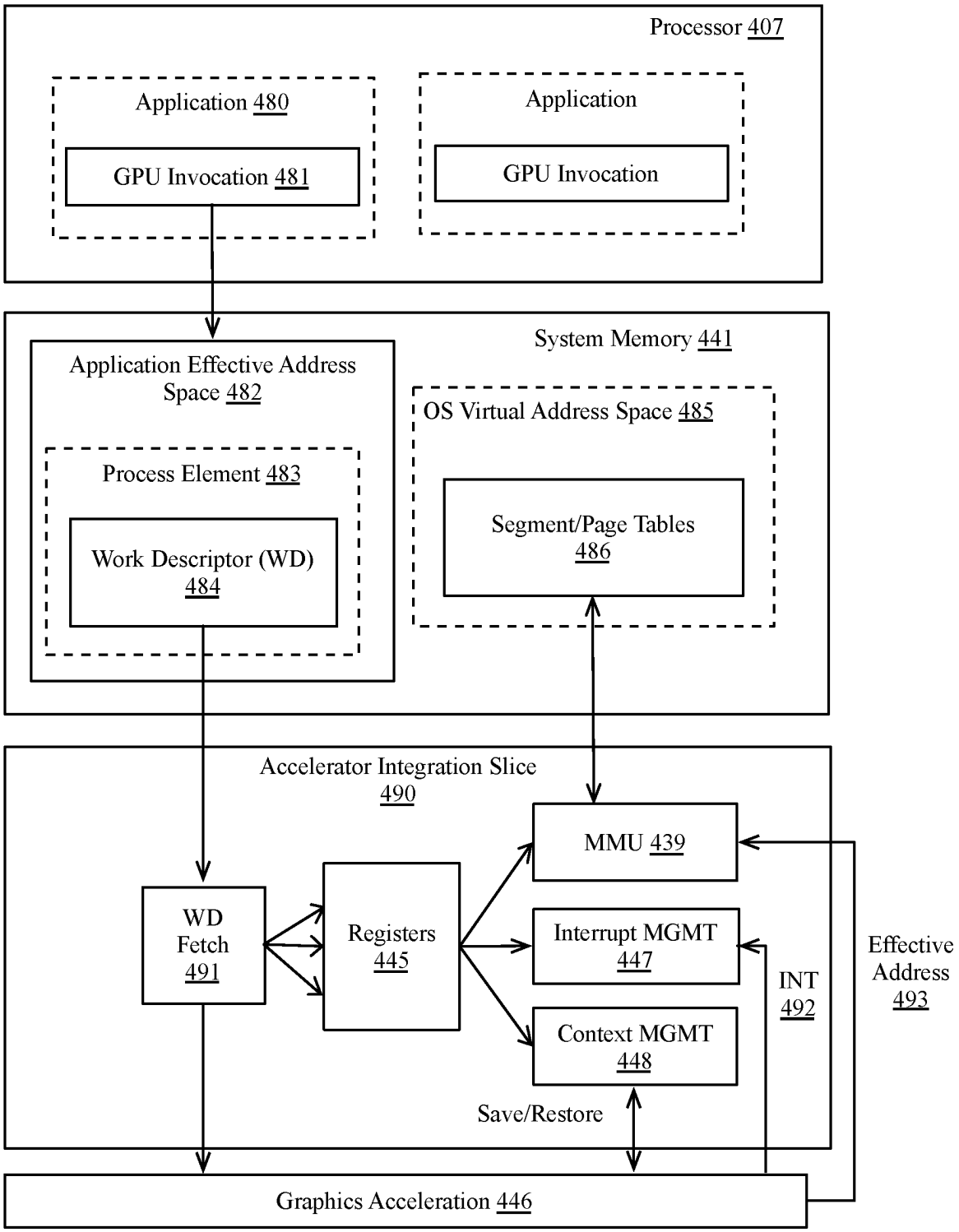

Processor 407
Application 480
GPU Invocation 481
Application
GPU Invocation
System Memory 441
Application Effective Address Space 482
Process Element 483
Work Descriptor (WD) 484
OS Virtual Address Space 485
Segment/Page Tables 486
Accelerator Integration Slice 490
MMU 439
WD Fetch 491
Registers 445
Interrupt MGMT 447
Context MGMT 448
INT 492
Effective Address 493
Save/Restore
Graphics Acceleration 446

FIG. 4D

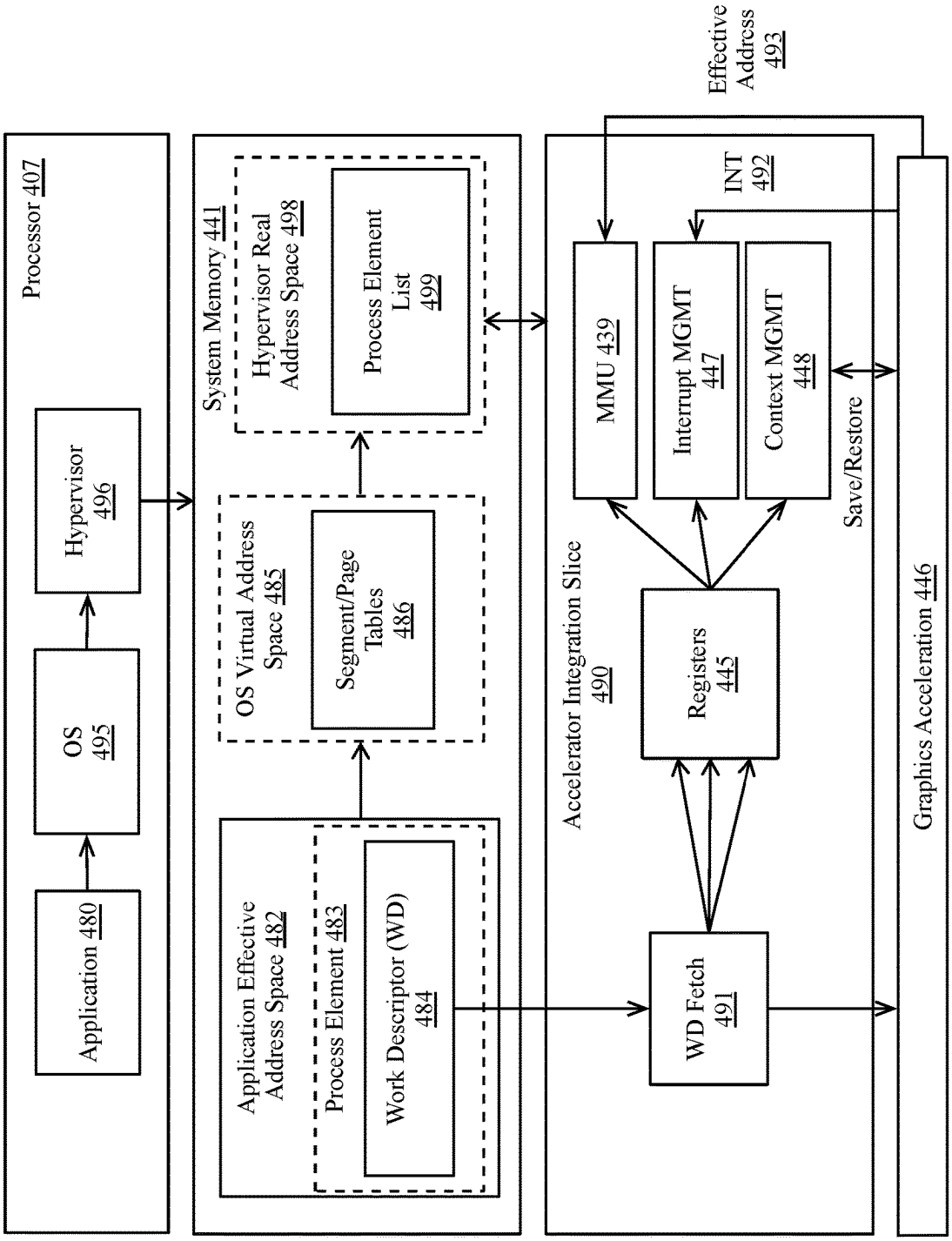

Processor 407
Application 480
OS 495
Hypervisor 496
System Memory 441
Application Effective Address Space 482
Process Element 483
Work Descriptor (WD) 484
OS Virtual Address Space 485
Segment/Page Tables 486
Hypervisor Real Address Space 498
Process Element List 499
Accelerator Integration Slice 490
WD Fetch 491
Registers 445
MMU 439
Interrupt MGMT 447
Context MGMT 448
Save/Restore
INT 492
Effective Address 493
Graphics Acceleration 446

FIG. 4E

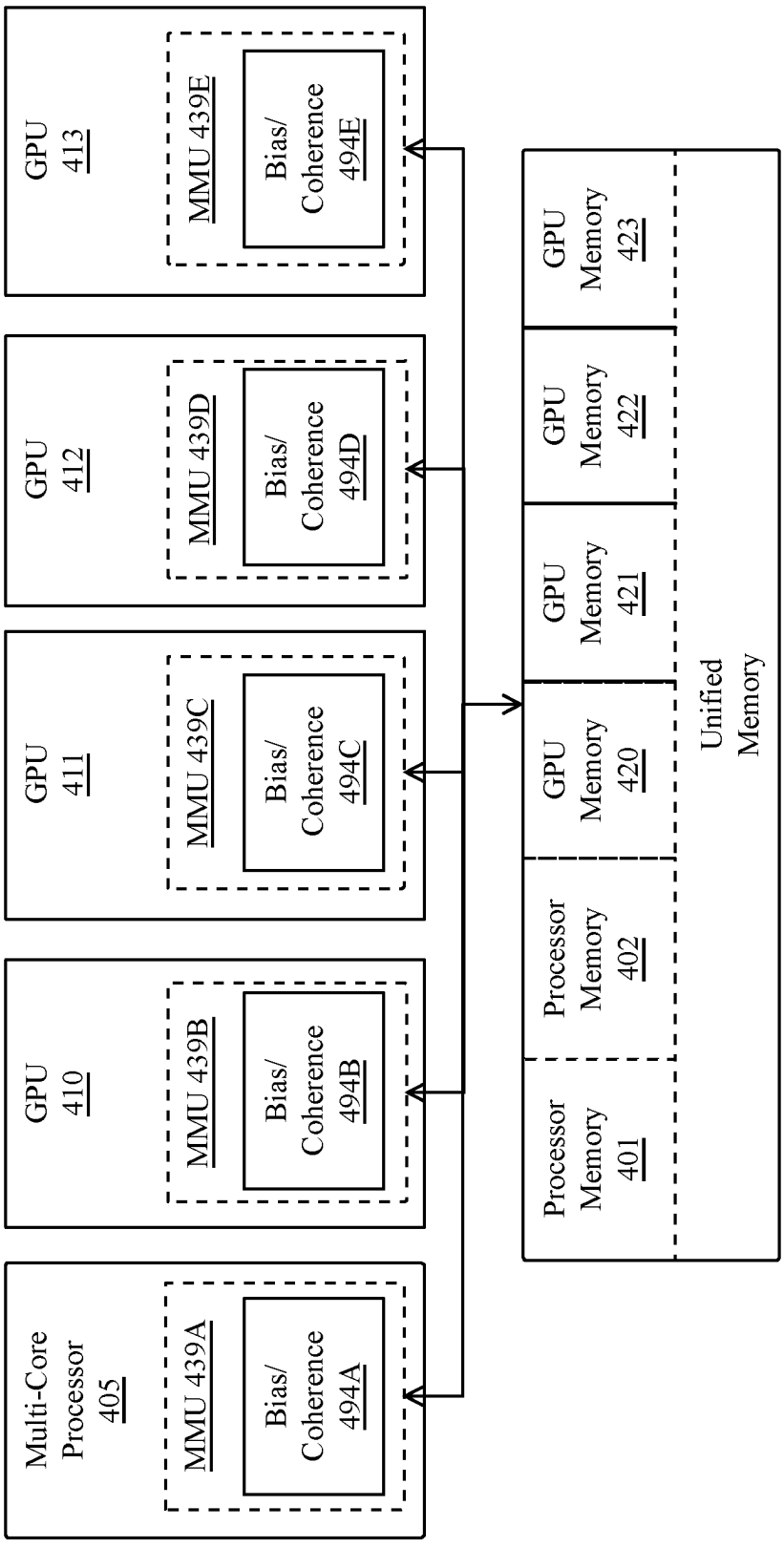

Multi-Core Processor 405
MMU 439A
Bias/ Coherence 494A
GPU 410
MMU 439B
Bias/ Coherence 494B
GPU 411
MMU 439C
Bias/ Coherence 494C
GPU 412
MMU 439D
Bias/ Coherence 494D
GPU 413
MMU 439E
Bias/ Coherence 494E
Processor Memory 401
Processor Memory 402
GPU Memory 420
GPU Memory 421
GPU Memory 422
GPU Memory 423
Unified Memory

FIG. 4F

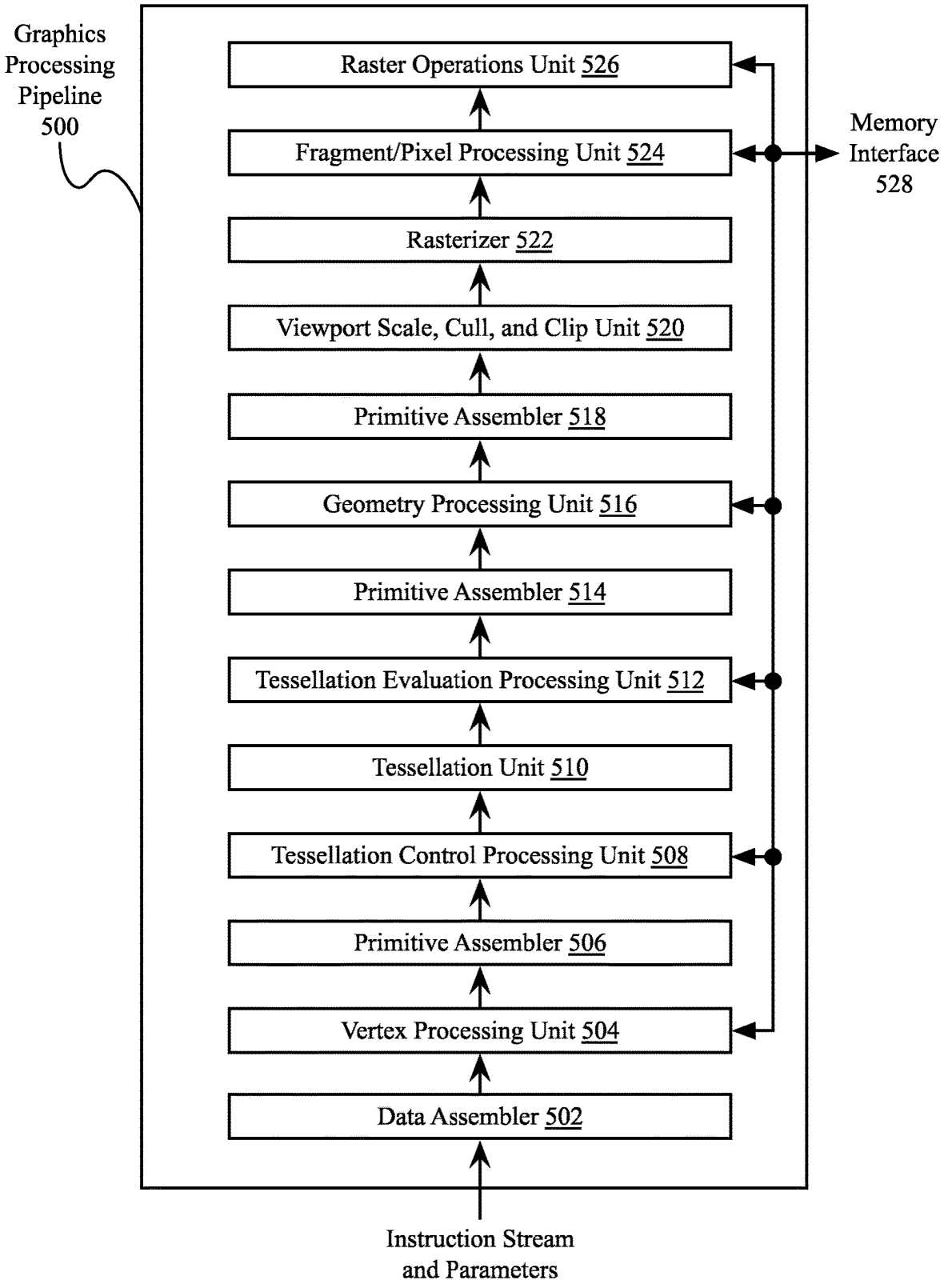

Graphics Processing Pipeline 500
Raster Operations Unit 526
Fragment/Pixel Processing Unit 524
Rasterizer 522
Viewport Scale, Cull, and Clip Unit 520
Primitive Assembler 518
Geometry Processing Unit 516
Primitive Assembler 514
Tessellation Evaluation Processing Unit 512
Tessellation Unit 510
Tessellation Control Processing Unit 508
Primitive Assembler 506
Vertex Processing Unit 504
Data Assembler 502
Memory Interface 528
Instruction Stream and Parameters

FIG. 5

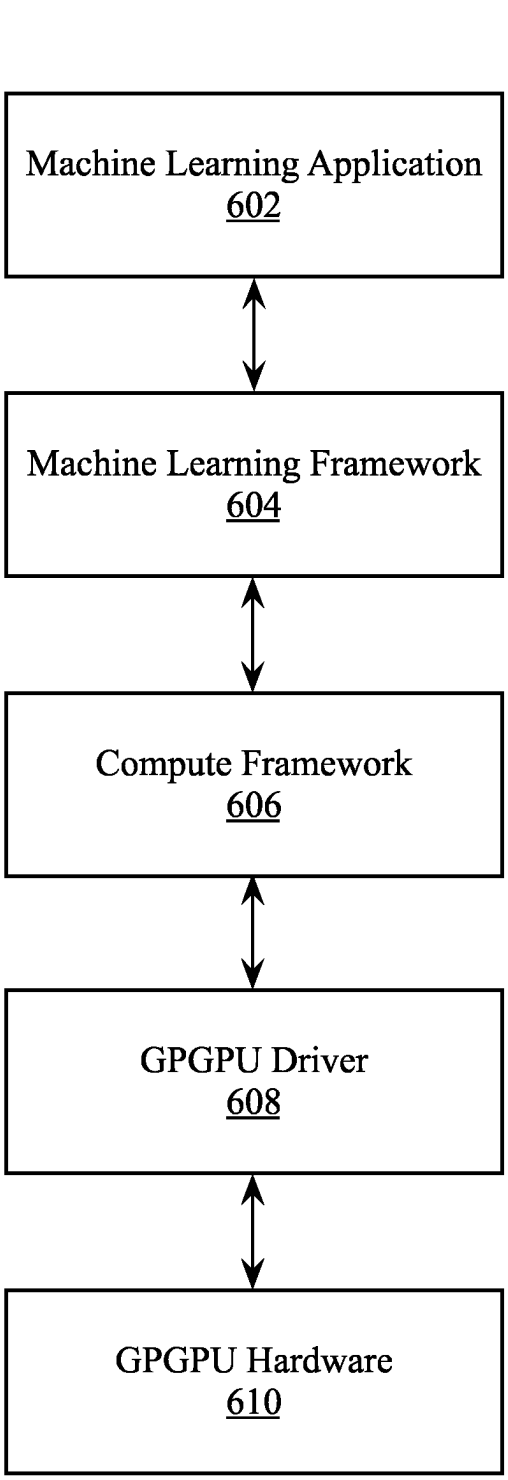

600
Machine Learning Application
602
Machine Learning Framework
604
Compute Framework
606
GPGPU Driver
608
GPGPU Hardware
610

FIG. 6

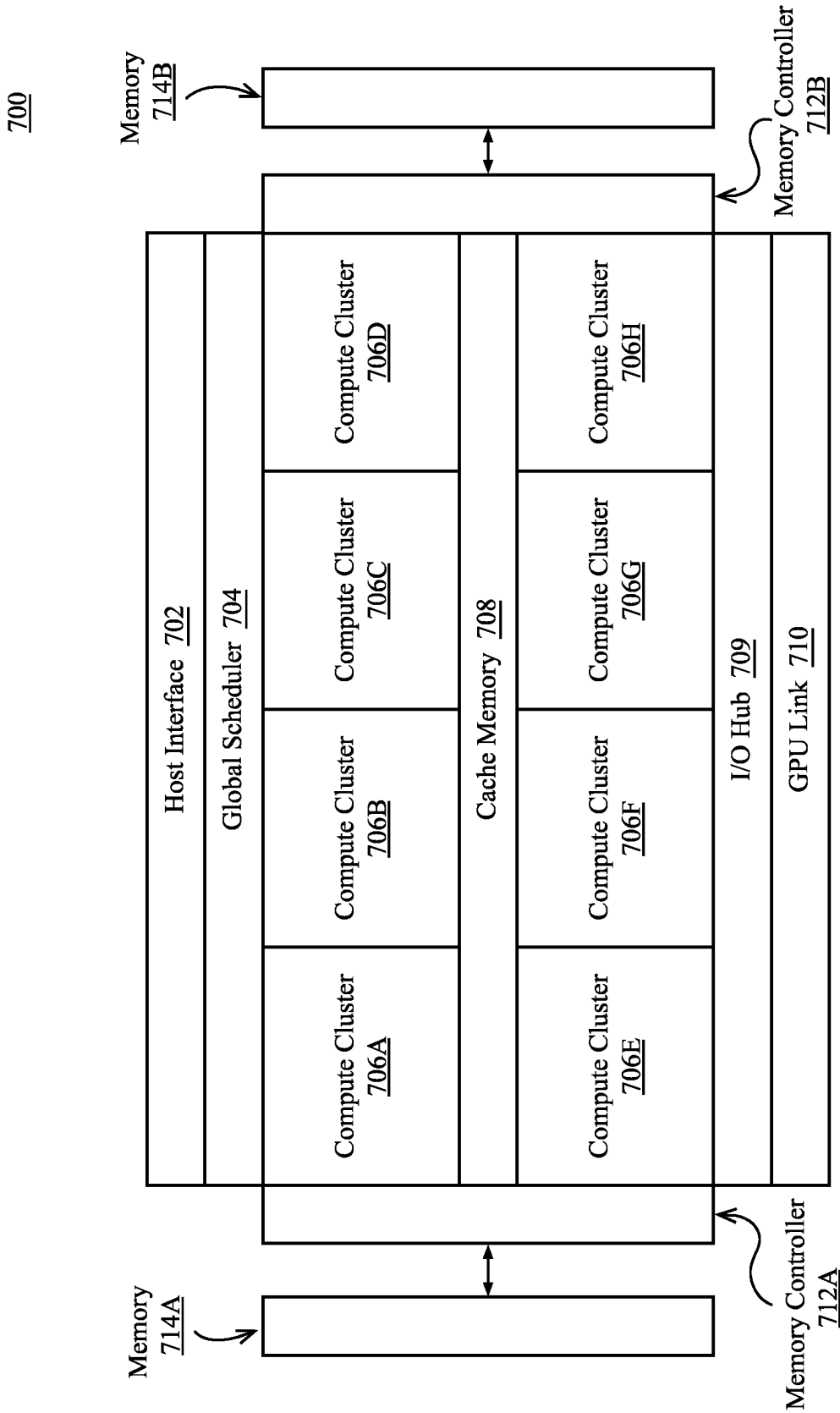

700
Memory 714B
Memory Controller 712B
Host Interface 702
Global Scheduler 704
Compute Cluster 706A
Compute Cluster 706B
Compute Cluster 706C
Compute Cluster 706D
Cache Memory 708
Compute Cluster 706E
Compute Cluster 706F
Compute Cluster 706G
Compute Cluster 706H
I/O Hub 709
GPU Link 710
Memory 714A
Memory Controller 712A

FIG. 7

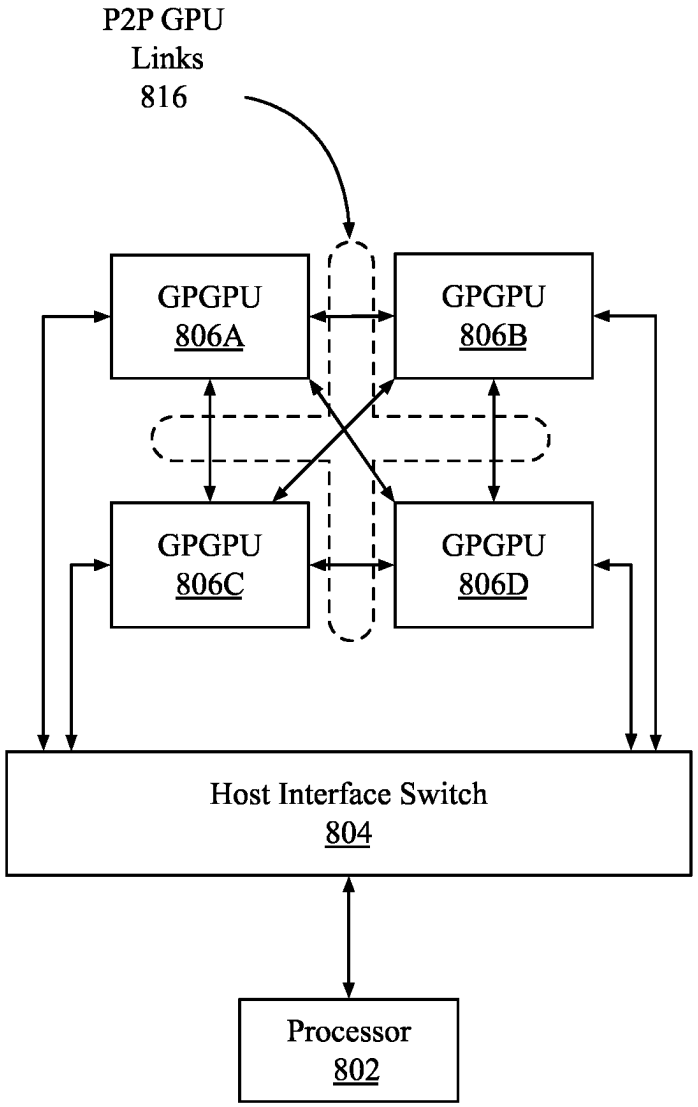

P2P GPU
Links
816
GPGPU
806A
GPGPU
806B
GPGPU
806C
GPGPU
806D
Host Interface Switch
804
Processor
802

FIG. 8

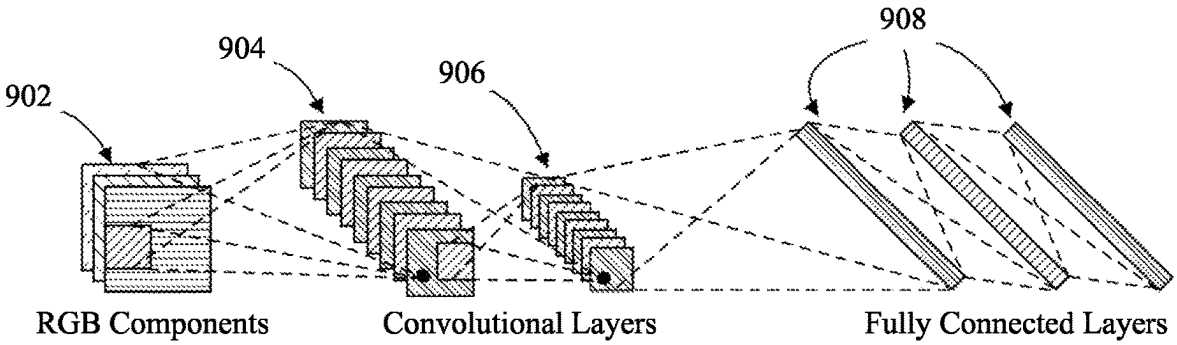

902
904
906
908
RGB Components
Convolutional Layers
Fully Connected Layers

FIG. 9A

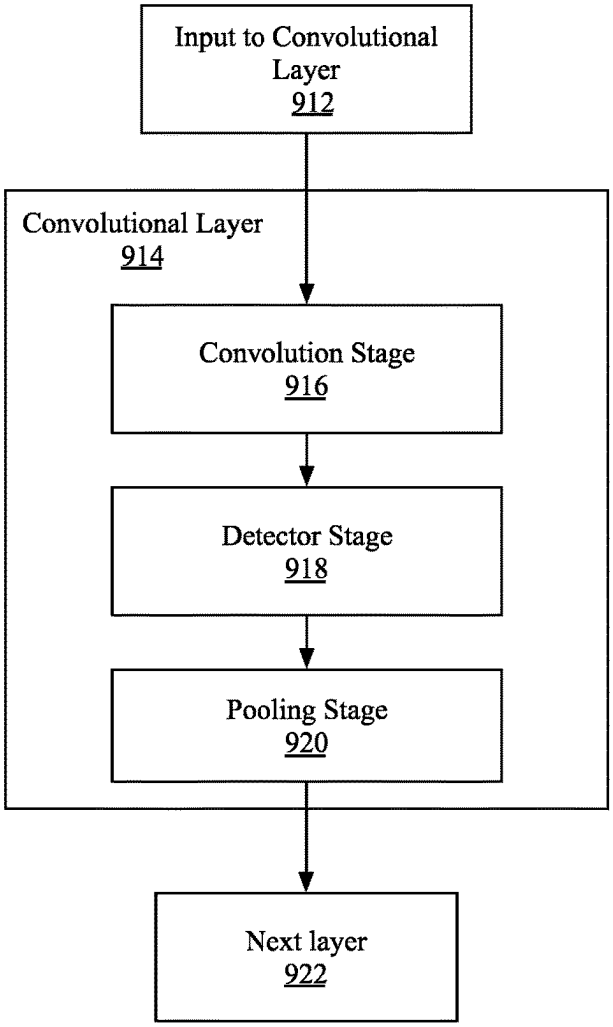

Input to Convolutional Layer
912
Convolutional Layer
914
Convolution Stage
916
Detector Stage
918
Pooling Stage
920
Next layer
922

FIG. 9B

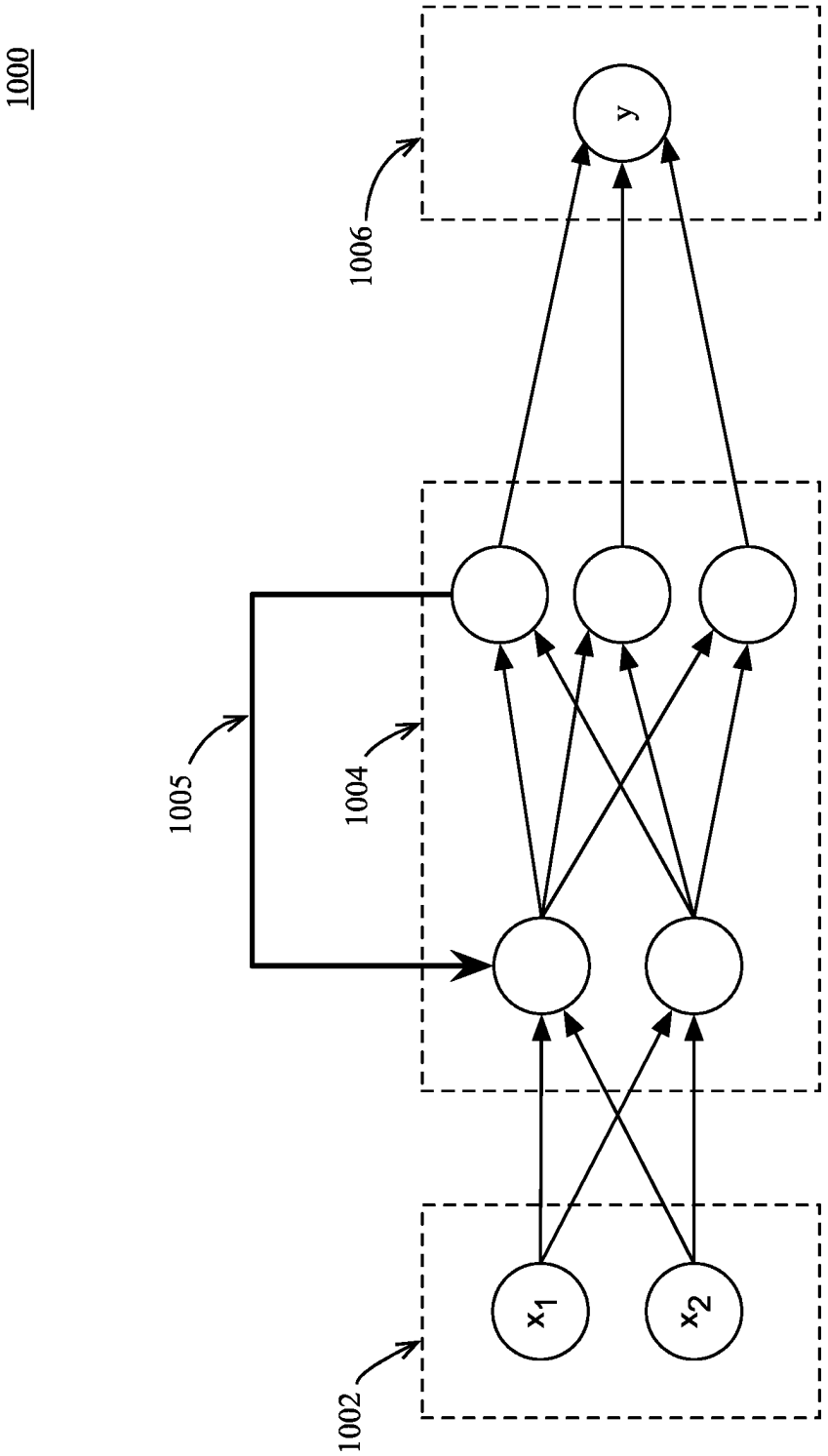



FIG. 10

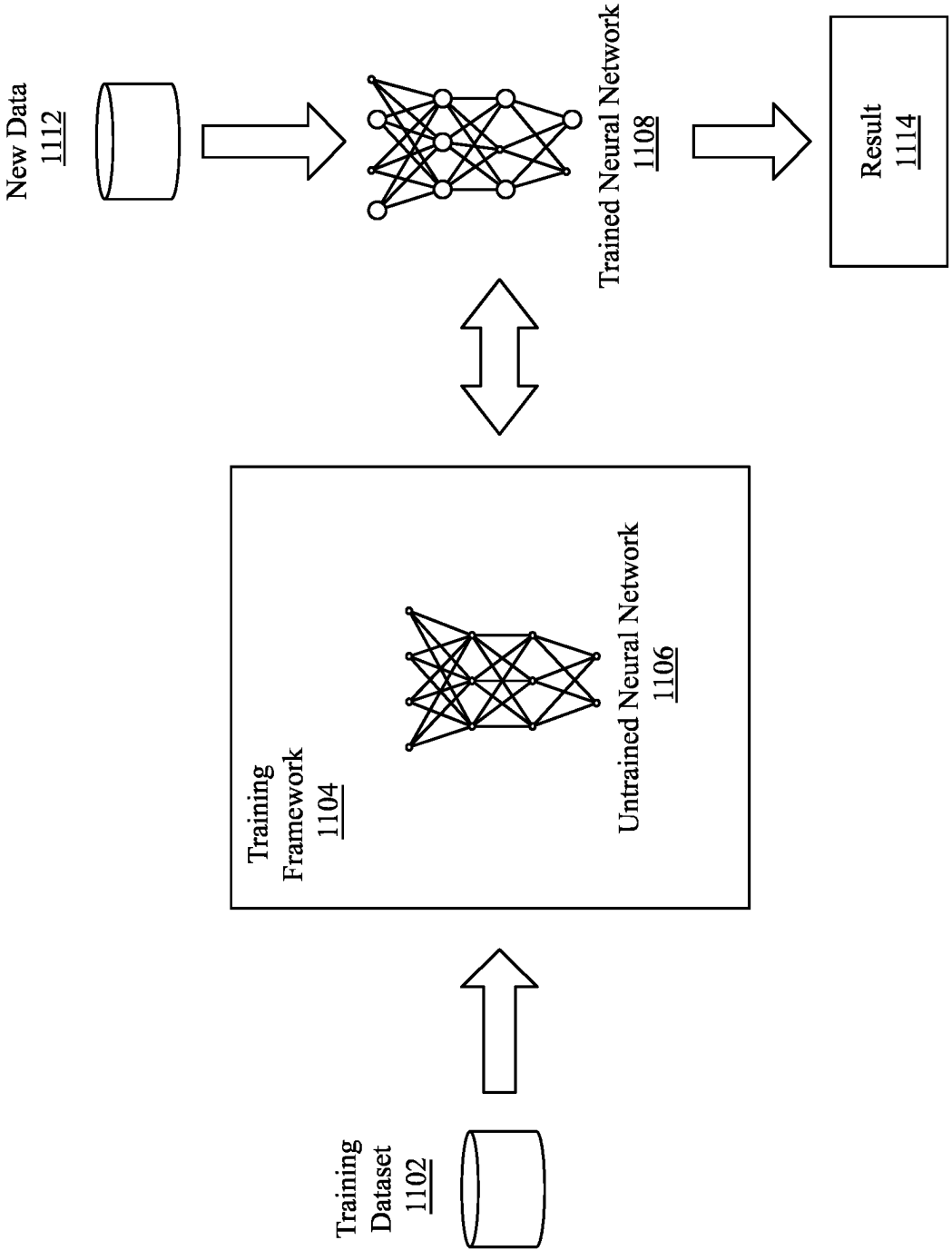

New Data
1112
Trained Neural Network
1108
Result
1114
Training
Framework
1104
Untrained Neural Network
1106
Training
Dataset
1102

FIG. 11

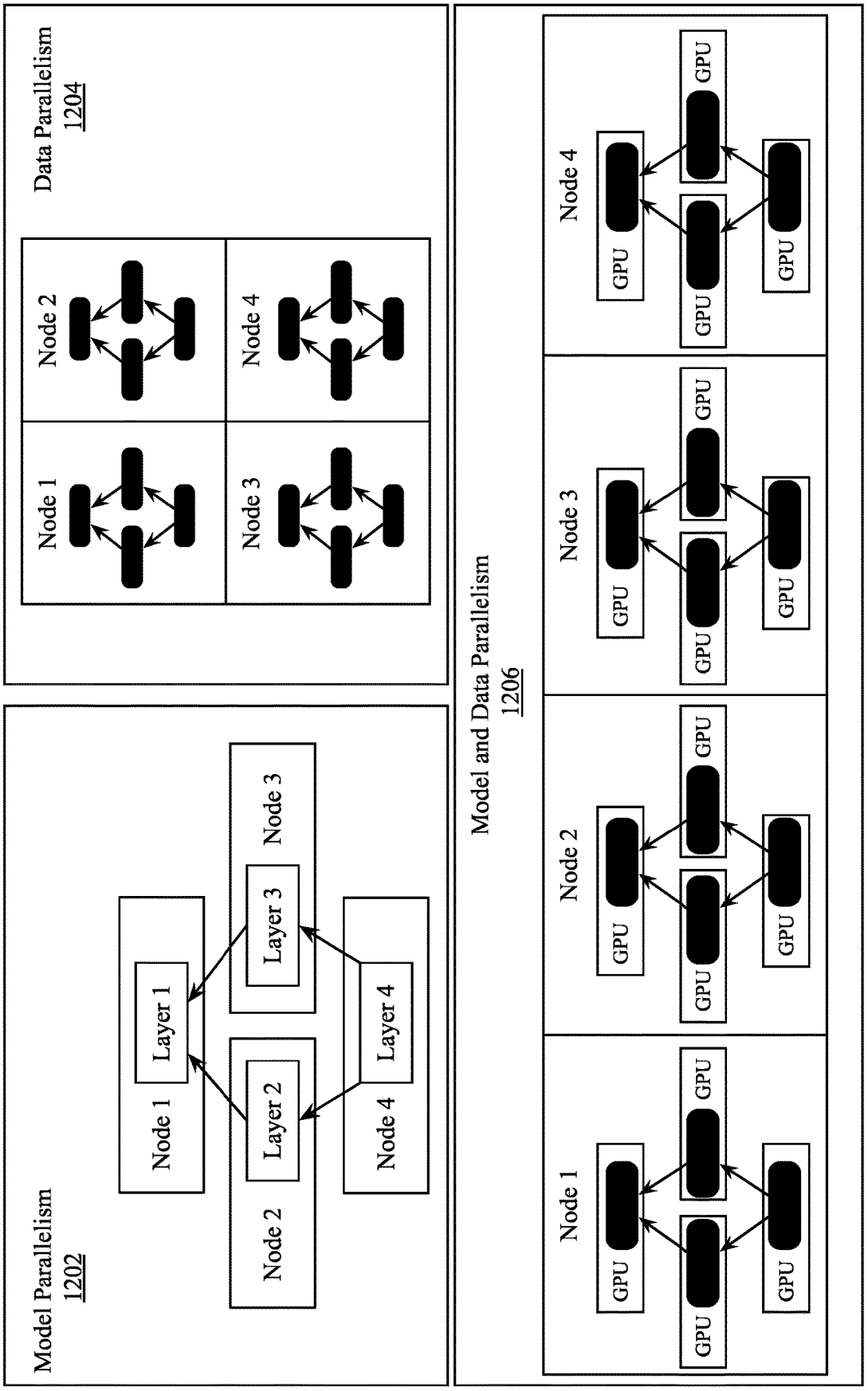

Model Parallelism
1202
Node 1
Layer 1
Node 2
Layer 2
Node 3
Layer 3
Node 4
Layer 4
Data Parallelism
1204
Node 1
Node 2
Node 3
Node 4
Model and Data Parallelism
1206
Node 1
Node 2
Node 3
Node 4
GPU

FIG. 12

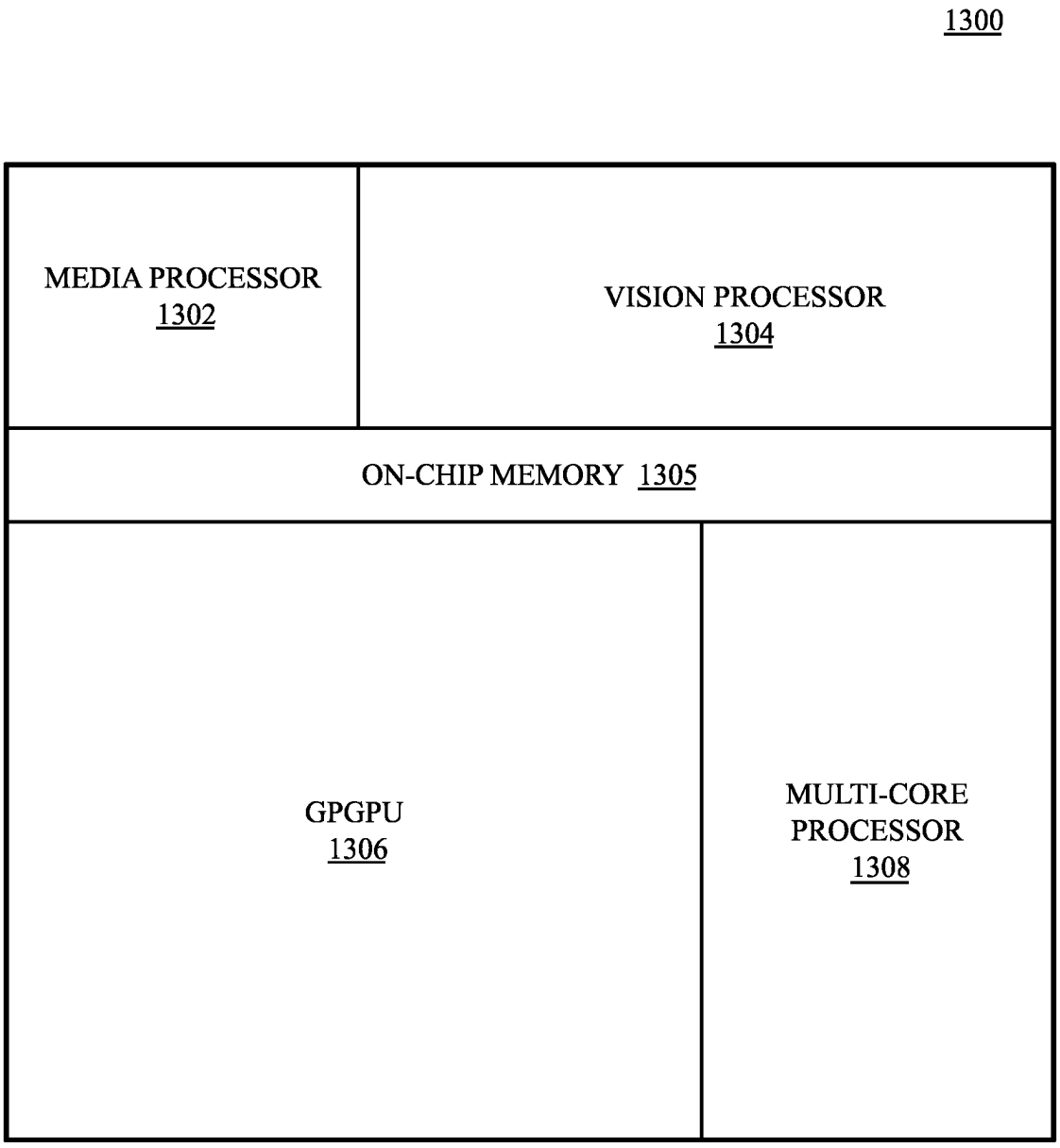

1300
MEDIA PROCESSOR
1302
VISION PROCESSOR
1304
ON-CHIP MEMORY 1305
GPGPU
1306
MULTI-CORE
PROCESSOR
1308

FIG. 13

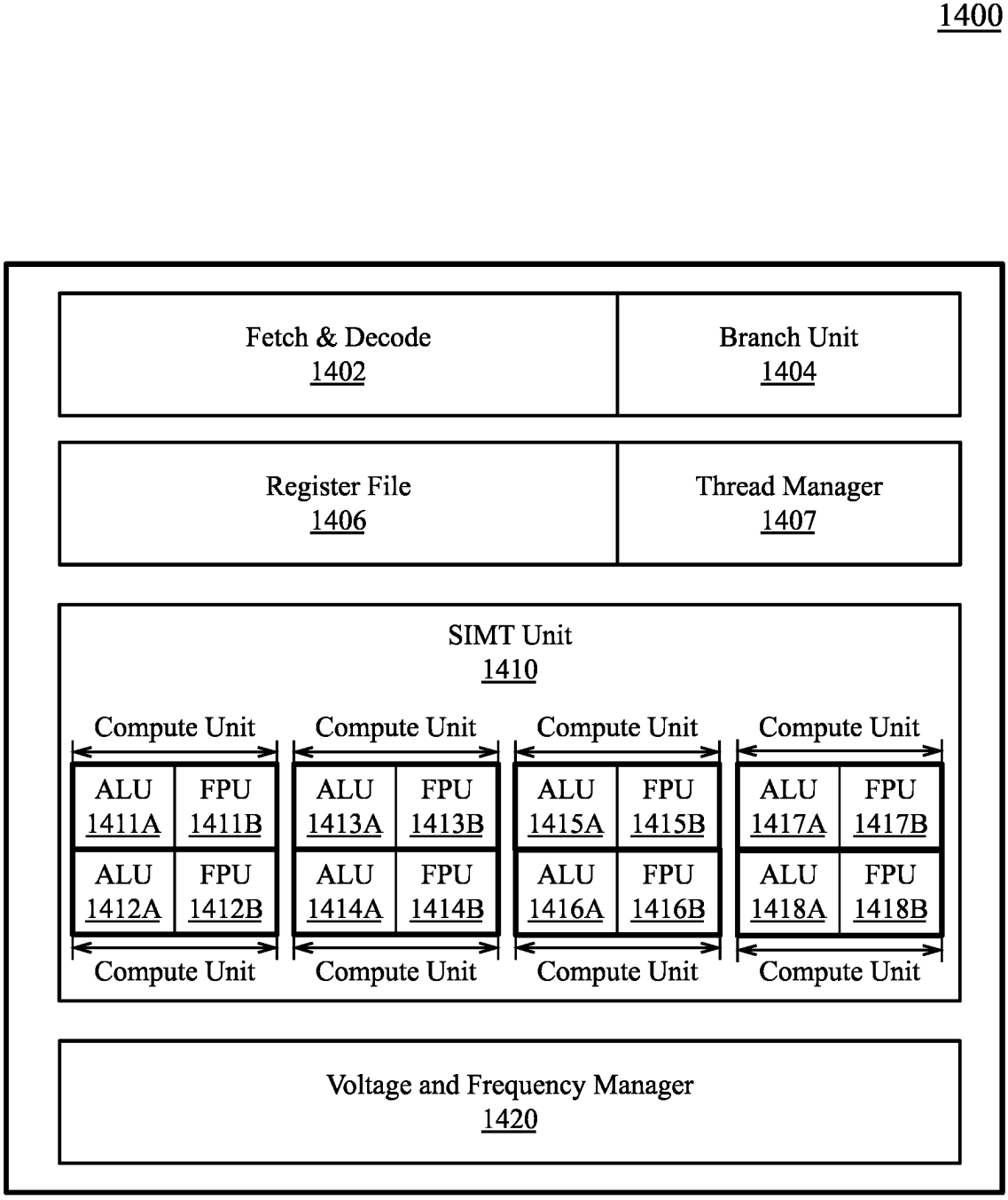

1400
Fetch & Decode
1402
Branch Unit
1404
Register File
1406
Thread Manager
1407
SIMT Unit
1410
Compute Unit
Compute Unit
Compute Unit
Compute Unit
ALU
1411A
FPU
1411B
ALU
1412A
FPU
1412B
ALU
1413A
FPU
1413B
ALU
1414A
FPU
1414B
ALU
1415A
FPU
1415B
ALU
1416A
FPU
1416B
ALU
1417A
FPU
1417B
ALU
1418A
FPU
1418B
Compute Unit
Compute Unit
Compute Unit
Compute Unit
Voltage and Frequency Manager
1420

FIG. 14

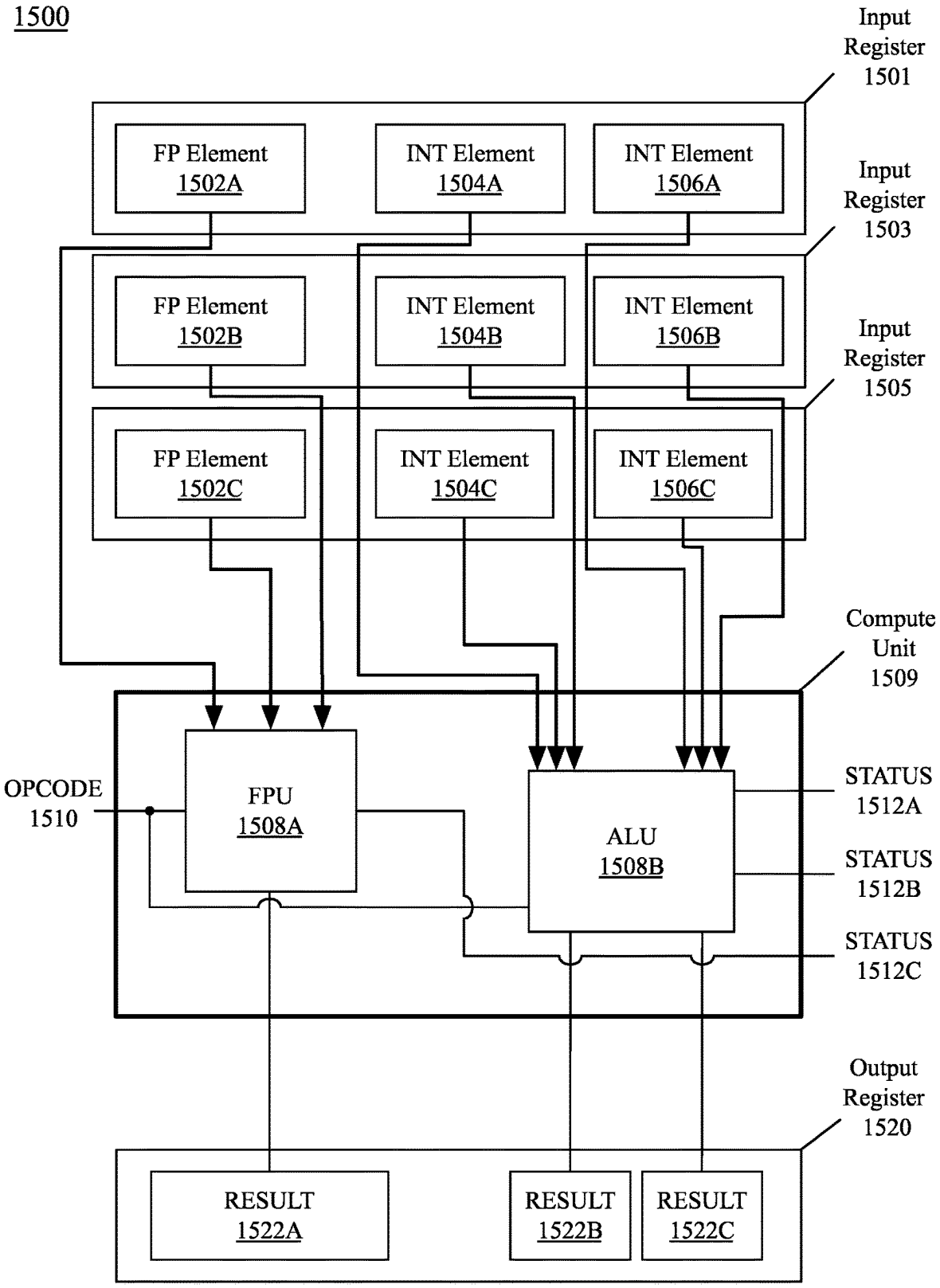

1500
Input Register 1501
FP Element 1502A
INT Element 1504A
INT Element 1506A
Input Register 1503
FP Element 1502B
INT Element 1504B
INT Element 1506B
Input Register 1505
FP Element 1502C
INT Element 1504C
INT Element 1506C
Compute Unit 1509
OPCODE 1510
FPU 1508A
ALU 1508B
STATUS 1512A
STATUS 1512B
STATUS 1512C
Output Register 1520
RESULT 1522A
RESULT 1522B
RESULT 1522C

FIG. 15

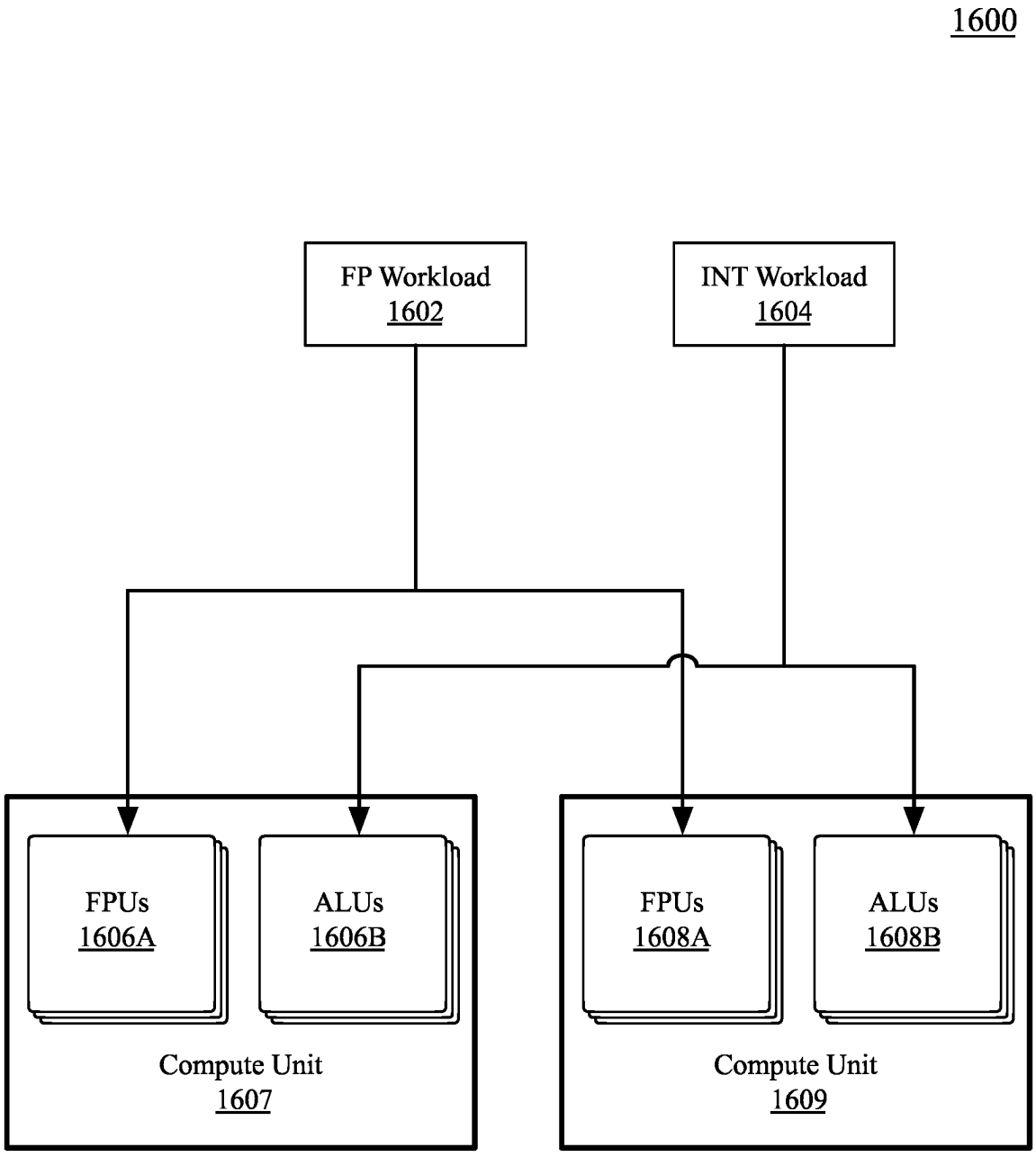

1600
FP Workload
1602
INT Workload
1604
FPUs
1606A
ALUs
1606B
Compute Unit
1607
FPUs
1608A
ALUs
1608B
Compute Unit
1609

FIG. 16

1700

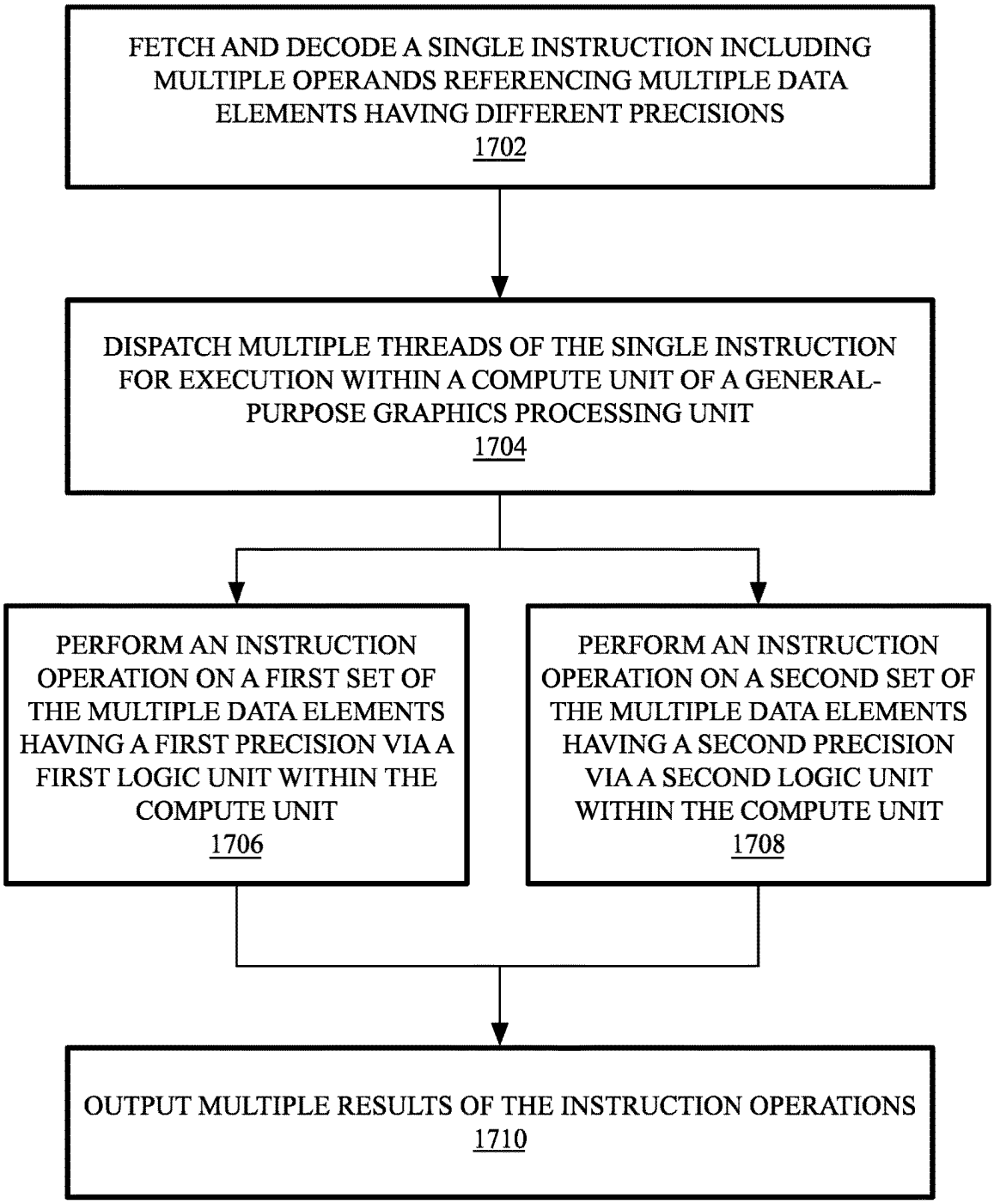

FETCH AND DECODE A SINGLE INSTRUCTION INCLUDING MULTIPLE OPERANDS REFERENCING MULTIPLE DATA ELEMENTS HAVING DIFFERENT PRECISIONS
1702
DISPATCH MULTIPLE THREADS OF THE SINGLE INSTRUCTION FOR EXECUTION WITHIN A COMPUTE UNIT OF A GENERAL-PURPOSE GRAPHICS PROCESSING UNIT
1704
PERFORM AN INSTRUCTION OPERATION ON A FIRST SET OF THE MULTIPLE DATA ELEMENTS HAVING A FIRST PRECISION VIA A FIRST LOGIC UNIT WITHIN THE COMPUTE UNIT
1706
PERFORM AN INSTRUCTION OPERATION ON A SECOND SET OF THE MULTIPLE DATA ELEMENTS HAVING A SECOND PRECISION VIA A SECOND LOGIC UNIT WITHIN THE COMPUTE UNIT
1708
OUTPUT MULTIPLE RESULTS OF THE INSTRUCTION OPERATIONS
1710

FIG. 17

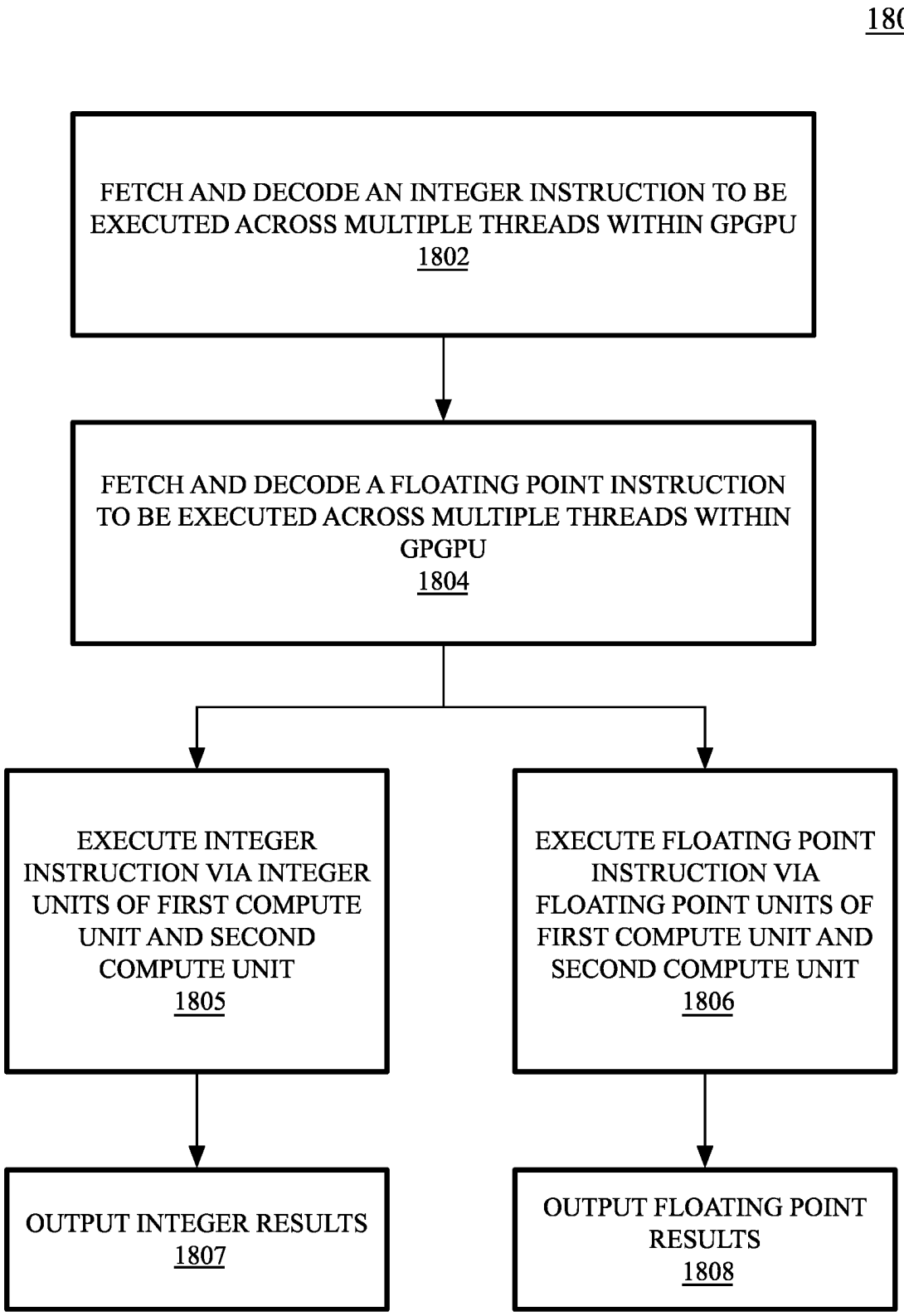

1800
FETCH AND DECODE AN INTEGER INSTRUCTION TO BE EXECUTED ACROSS MULTIPLE THREADS WITHIN GPGPU
1802
FETCH AND DECODE A FLOATING POINT INSTRUCTION TO BE EXECUTED ACROSS MULTIPLE THREADS WITHIN GPGPU
1804
EXECUTE INTEGER INSTRUCTION VIA INTEGER UNITS OF FIRST COMPUTE UNIT AND SECOND COMPUTE UNIT
1805
EXECUTE FLOATING POINT INSTRUCTION VIA FLOATING POINT UNITS OF FIRST COMPUTE UNIT AND SECOND COMPUTE UNIT
1806
OUTPUT INTEGER RESULTS
1807
OUTPUT FLOATING POINT RESULTS
1808

FIG. 18

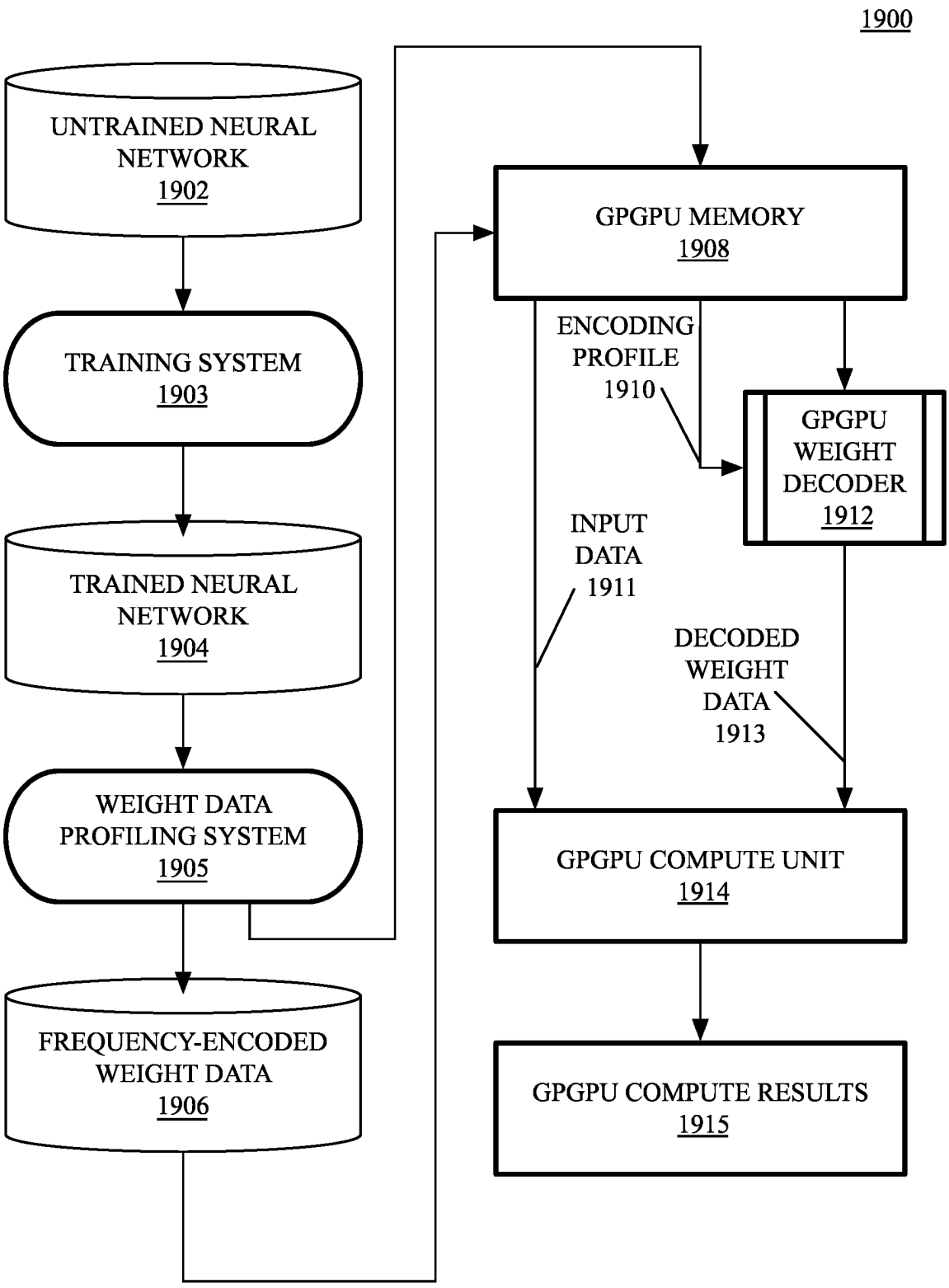

1900
UNTRAINED NEURAL NETWORK 1902
TRAINING SYSTEM 1903
TRAINED NEURAL NETWORK 1904
WEIGHT DATA PROFILING SYSTEM 1905
FREQUENCY-ENCODED WEIGHT DATA 1906
GPGPU MEMORY 1908
ENCODING PROFILE 1910
GPGPU WEIGHT DECODER 1912
INPUT DATA 1911
DECODED WEIGHT DATA 1913
GPGPU COMPUTE UNIT 1914
GPGPU COMPUTE RESULTS 1915

FIG. 19

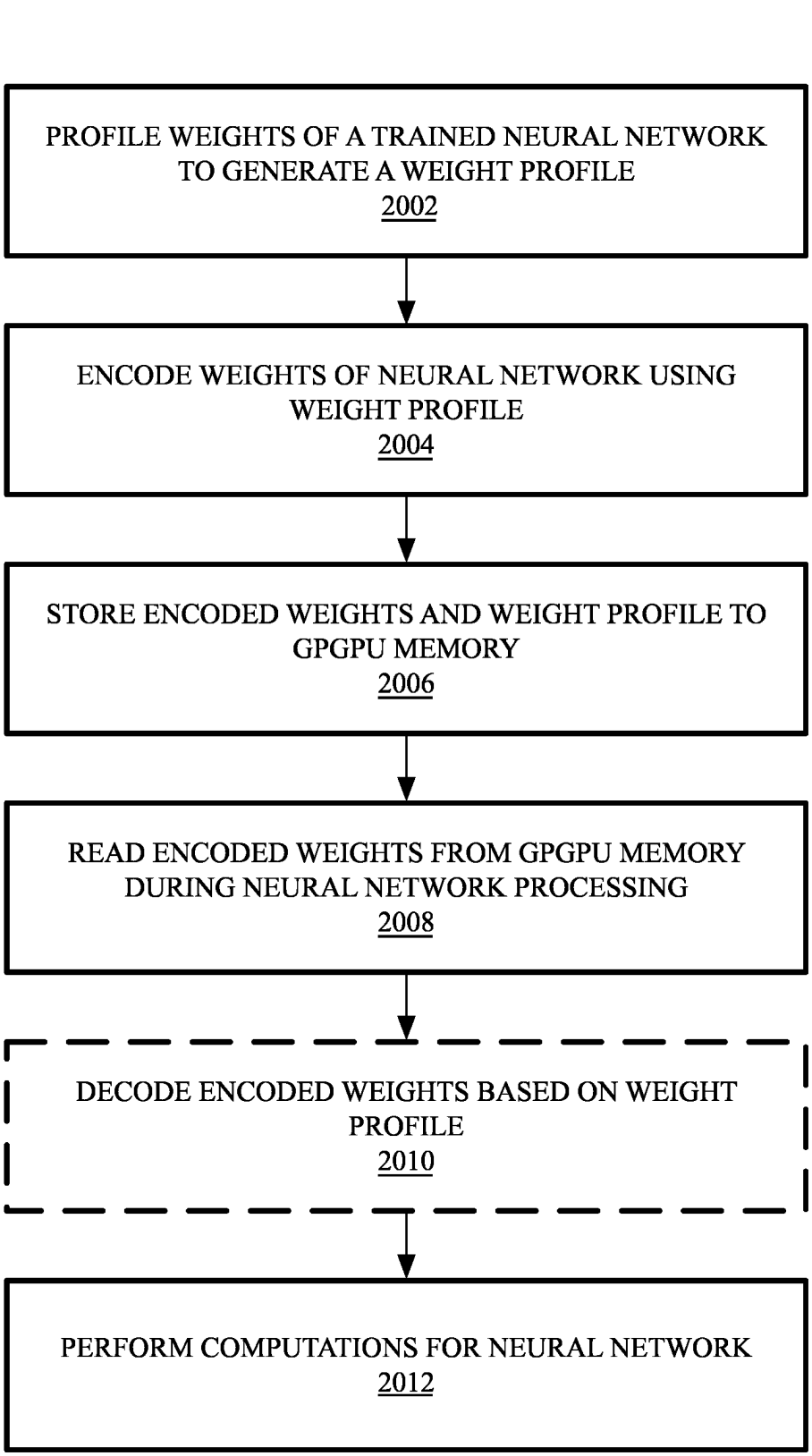

2000
PROFILE WEIGHTS OF A TRAINED NEURAL NETWORK TO GENERATE A WEIGHT PROFILE
2002
ENCODE WEIGHTS OF NEURAL NETWORK USING WEIGHT PROFILE
2004
STORE ENCODED WEIGHTS AND WEIGHT PROFILE TO GPGPU MEMORY
2006
READ ENCODED WEIGHTS FROM GPGPU MEMORY DURING NEURAL NETWORK PROCESSING
2008
DECODE ENCODED WEIGHTS BASED ON WEIGHT PROFILE
2010
PERFORM COMPUTATIONS FOR NEURAL NETWORK
2012

FIG. 20

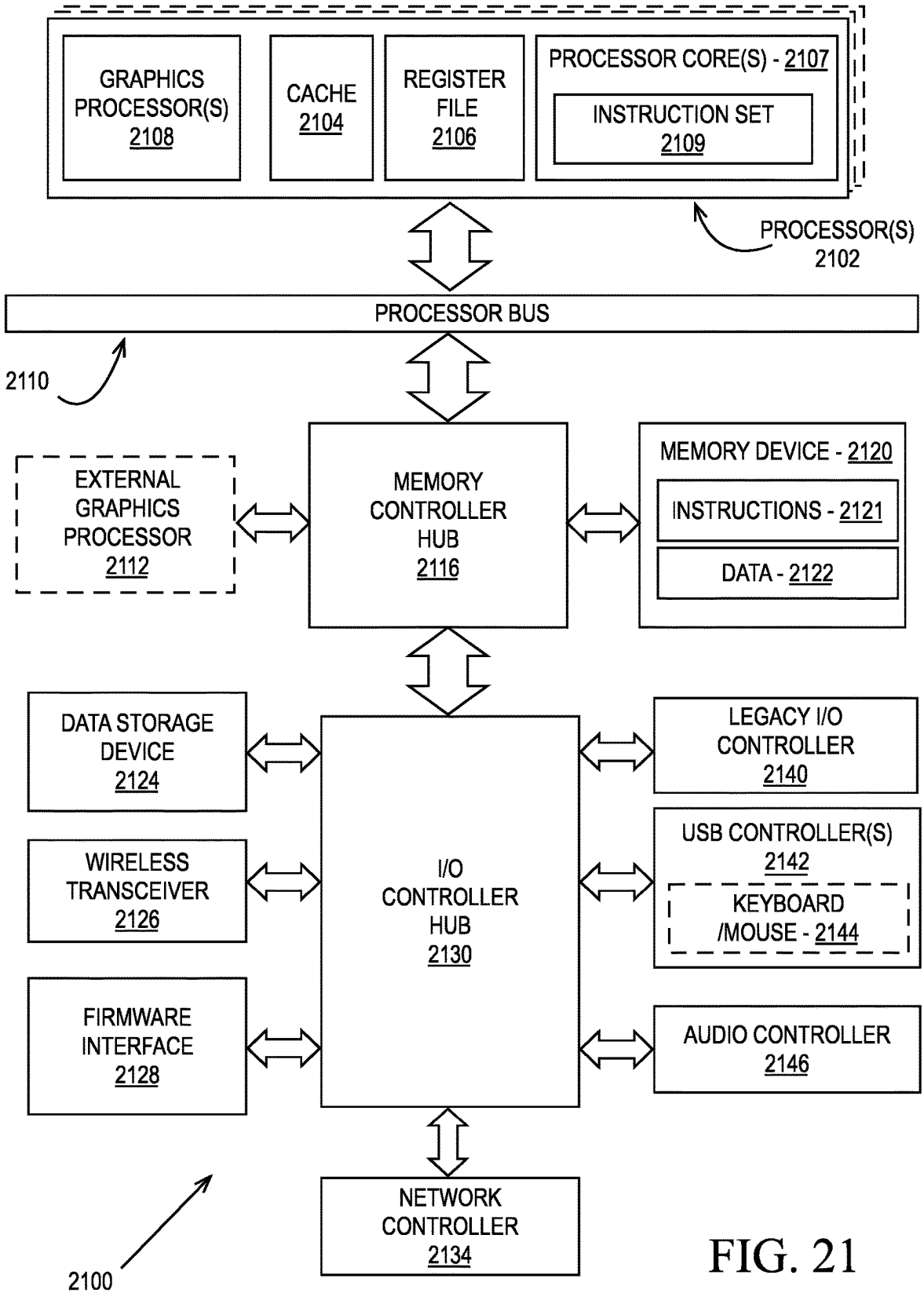

GRAPHICS PROCESSOR(S) 2108
CACHE 2104
REGISTER FILE 2106
PROCESSOR CORE(S) - 2107
INSTRUCTION SET 2109
PROCESSOR(S) 2102
PROCESSOR BUS
2110
EXTERNAL GRAPHICS PROCESSOR 2112
MEMORY CONTROLLER HUB 2116
MEMORY DEVICE - 2120
INSTRUCTIONS - 2121
DATA - 2122
DATA STORAGE DEVICE 2124
WIRELESS TRANSCEIVER 2126
FIRMWARE INTERFACE 2128
I/O CONTROLLER HUB 2130
LEGACY I/O CONTROLLER 2140
USB CONTROLLER(S) 2142
KEYBOARD /MOUSE - 2144
AUDIO CONTROLLER 2146
NETWORK CONTROLLER 2134
2100

FIG. 21

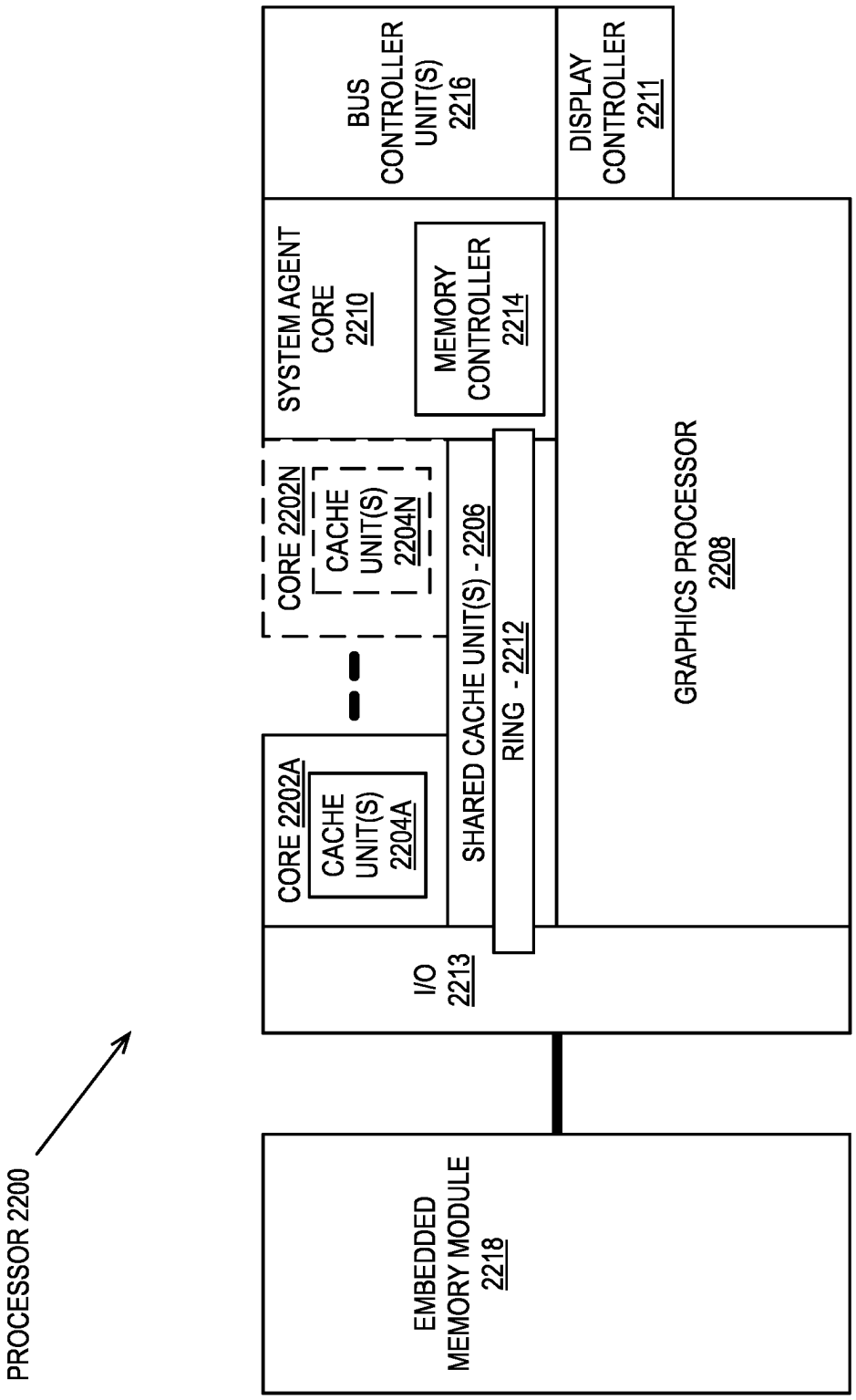

PROCESSOR 2200
EMBEDDED MEMORY MODULE 2218
I/O 2213
CORE 2202A
CACHE UNIT(S) 2204A
CORE 2202N
CACHE UNIT(S) 2204N
SHARED CACHE UNIT(S) - 2206
RING - 2212
SYSTEM AGENT CORE 2210
MEMORY CONTROLLER 2214
GRAPHICS PROCESSOR 2208
BUS CONTROLLER UNIT(S) 2216
DISPLAY CONTROLLER 2211

FIG. 22

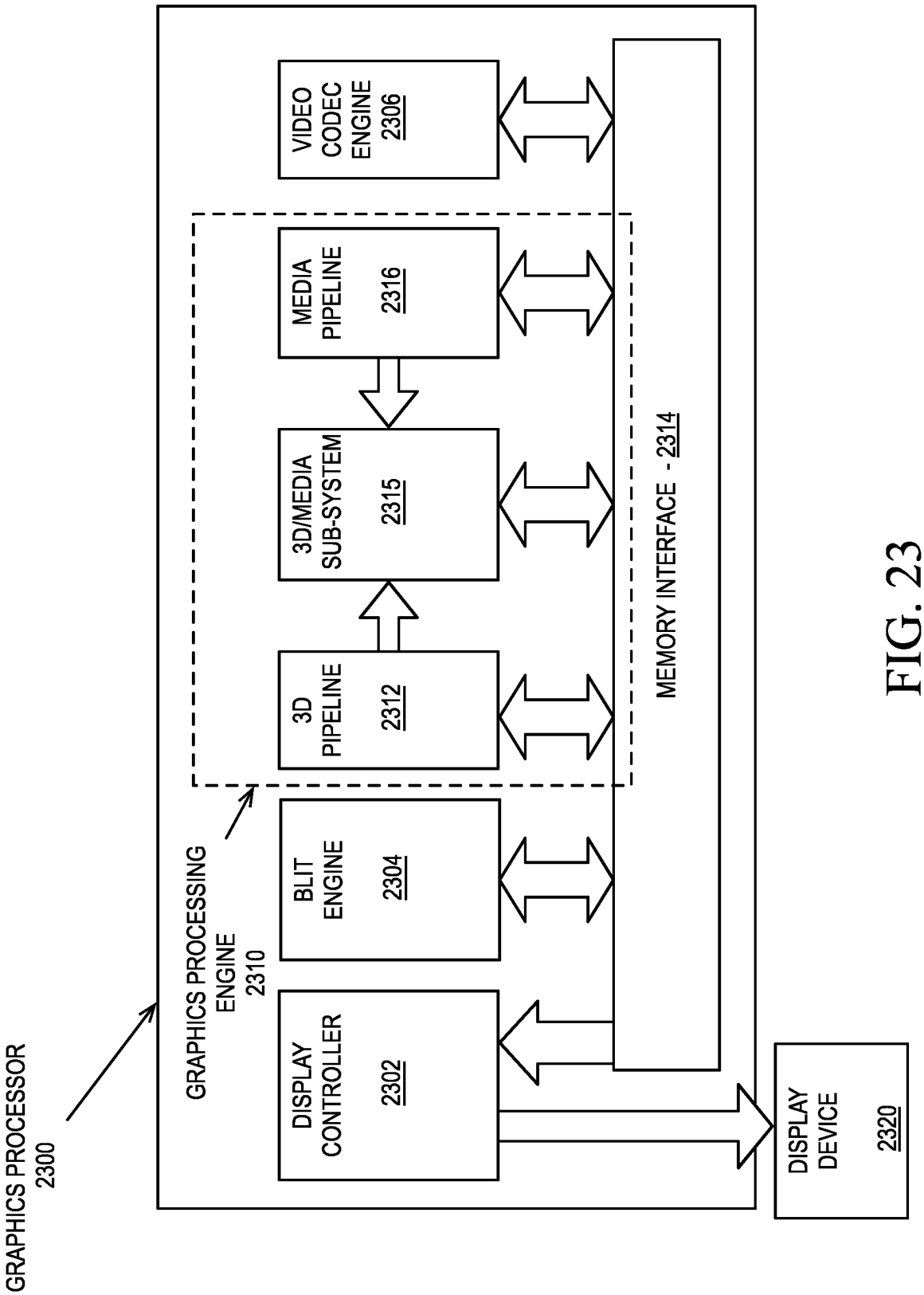

GRAPHICS PROCESSOR
2300
GRAPHICS PROCESSING
ENGINE
2310
DISPLAY
CONTROLLER
2302
BLIT
ENGINE
2304
3D
PIPELINE
2312
3D/MEDIA
SUB-SYSTEM
2315
MEDIA
PIPELINE
2316
VIDEO
CODEC
ENGINE
2306
MEMORY INTERFACE - 2314
DISPLAY
DEVICE
2320

FIG. 23

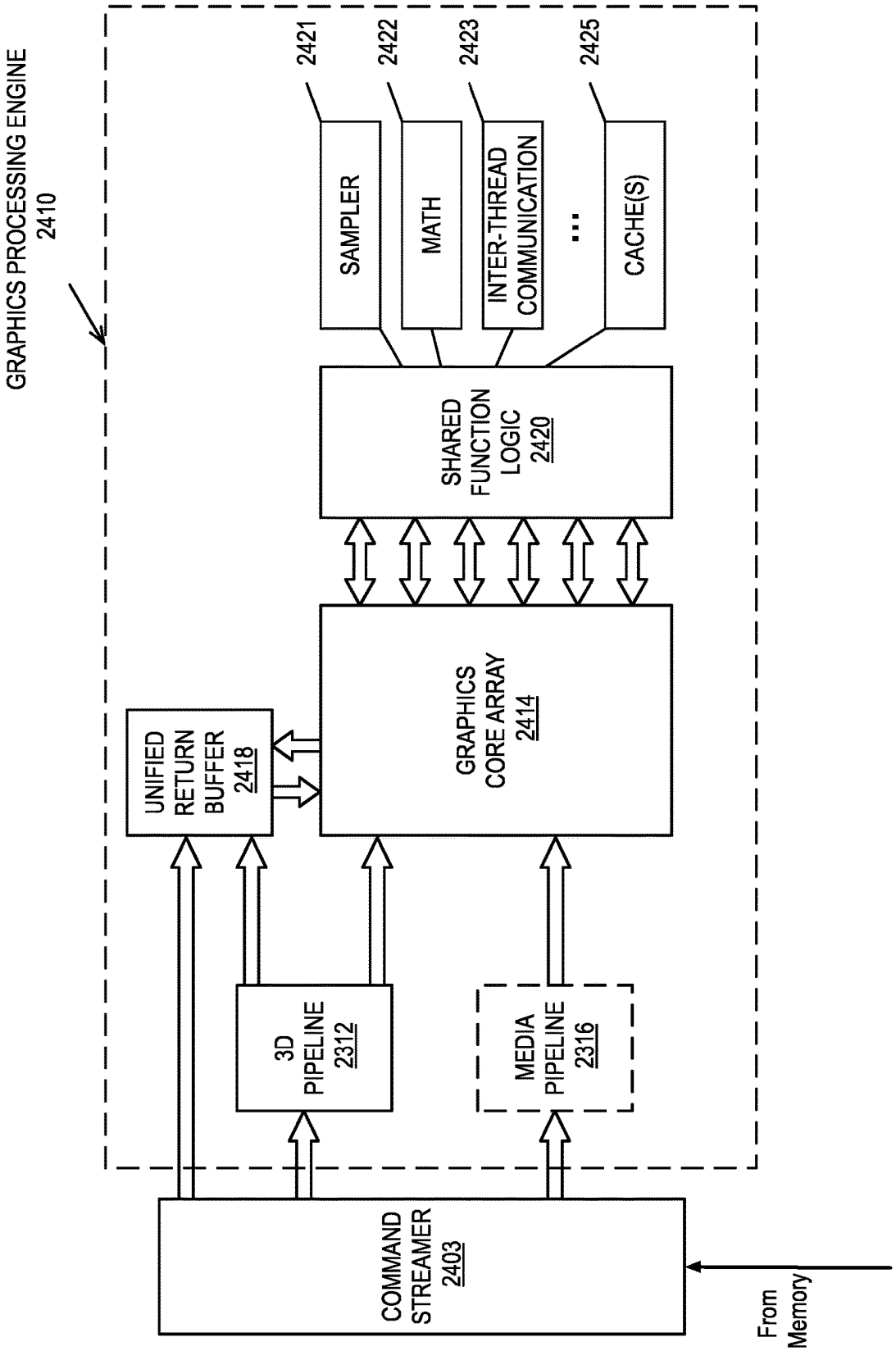

GRAPHICS PROCESSING ENGINE
2410
2421
2422
2423
2425
SAMPLER
MATH
INTER-THREAD COMMUNICATION
...
CACHE(S)
SHARED FUNCTION LOGIC 2420
GRAPHICS CORE ARRAY 2414
UNIFIED RETURN BUFFER 2418
3D PIPELINE 2312
MEDIA PIPELINE 2316
COMMAND STREAMER 2403
From Memory

FIG. 24

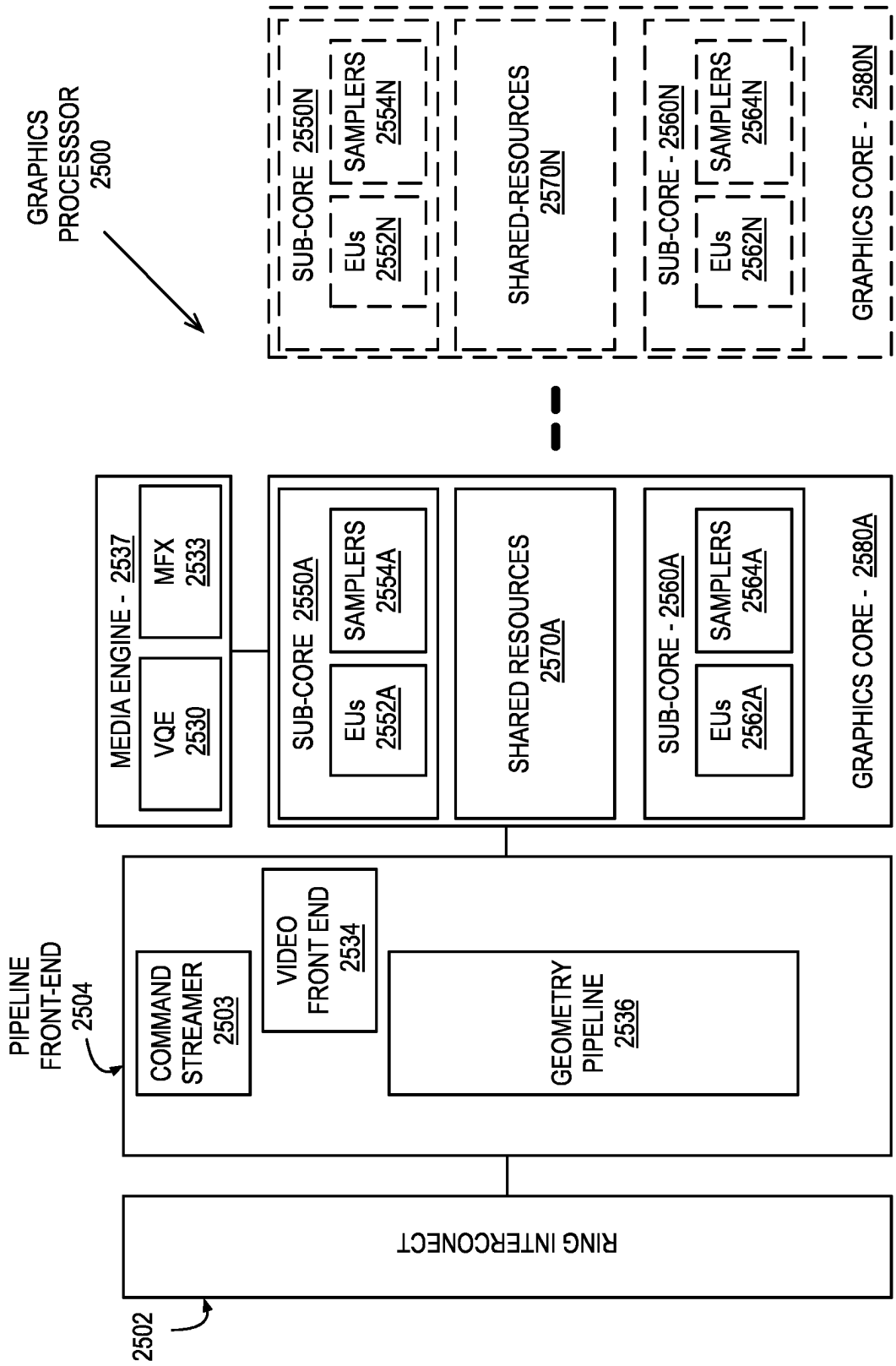

GRAPHICS PROCESSSOR 2500
SUB-CORE 2550N
EUs 2552N
SAMPLERS 2554N
SHARED-RESOURCES 2570N
SUB-CORE - 2560N
EUs 2562N
SAMPLERS 2564N
GRAPHICS CORE - 2580N
MEDIA ENGINE - 2537
VQE 2530
MFX 2533
SUB-CORE 2550A
EUs 2552A
SAMPLERS 2554A
SHARED RESOURCES 2570A
SUB-CORE - 2560A
EUs 2562A
SAMPLERS 2564A
GRAPHICS CORE - 2580A
PIPELINE FRONT-END 2504
COMMAND STREAMER 2503
VIDEO FRONT END 2534
GEOMETRY PIPELINE 2536
RING INTERCONECT
2502

FIG. 25

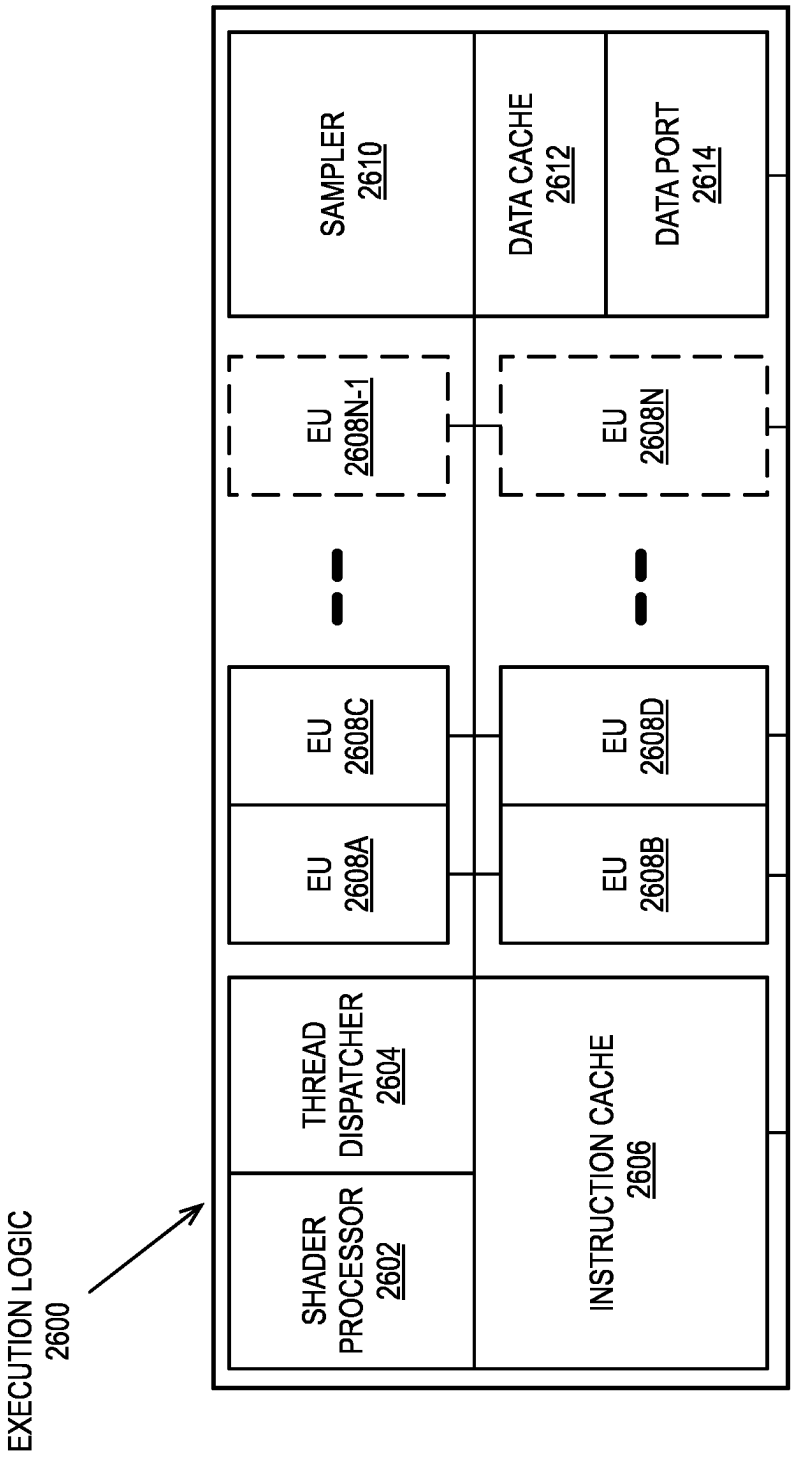

EXECUTION LOGIC
2600
SHADER PROCESSOR
2602
THREAD DISPATCHER
2604
INSTRUCTION CACHE
2606
EU
2608A
EU
2608C
EU
2608B
EU
2608D
EU
2608N-1
EU
2608N
SAMPLER
2610
DATA CACHE
2612
DATA PORT
2614

FIG. 26

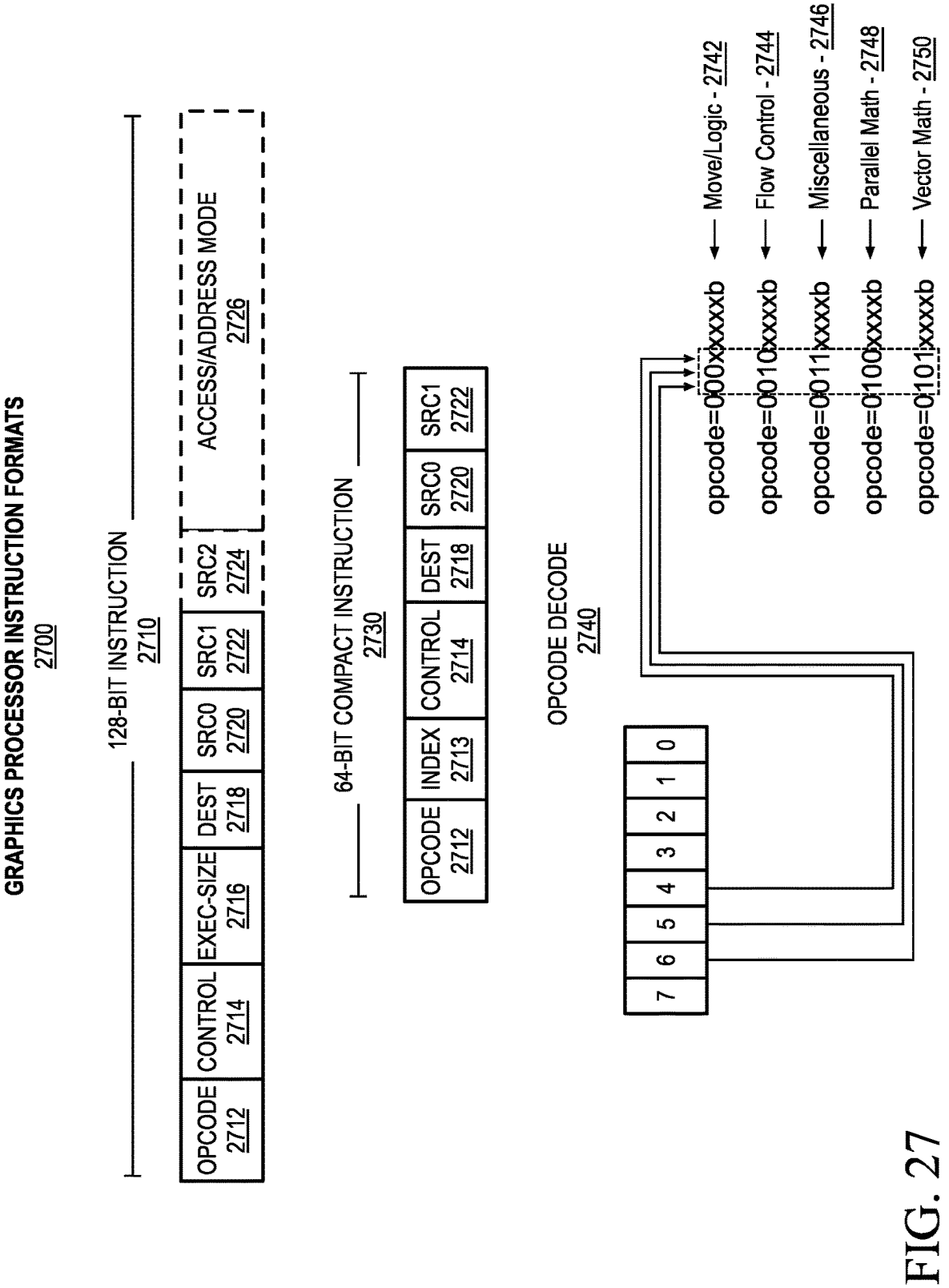

GRAPHICS PROCESSOR INSTRUCTION FORMATS
2700
128-BIT INSTRUCTION
2710
OPCODE 2712
CONTROL 2714
EXEC-SIZE 2716
DEST 2718
SRC0 2720
SRC1 2722
SRC2 2724
ACCESS/ADDRESS MODE 2726
64-BIT COMPACT INSTRUCTION
2730
OPCODE 2712
INDEX 2713
CONTROL 2714
DEST 2718
SRC0 2720
SRC1 2722
OPCODE DECODE
2740
7 6 5 4 3 2 1 0
opcode=000xxxxb
opcode=001xxxxb
opcode=011xxxxb
opcode=0100xxxxb
opcode=0101xxxxb
Move/Logic - 2742
Flow Control - 2744
Miscellaneous - 2746
Parallel Math - 2748
Vector Math - 2750

FIG. 27

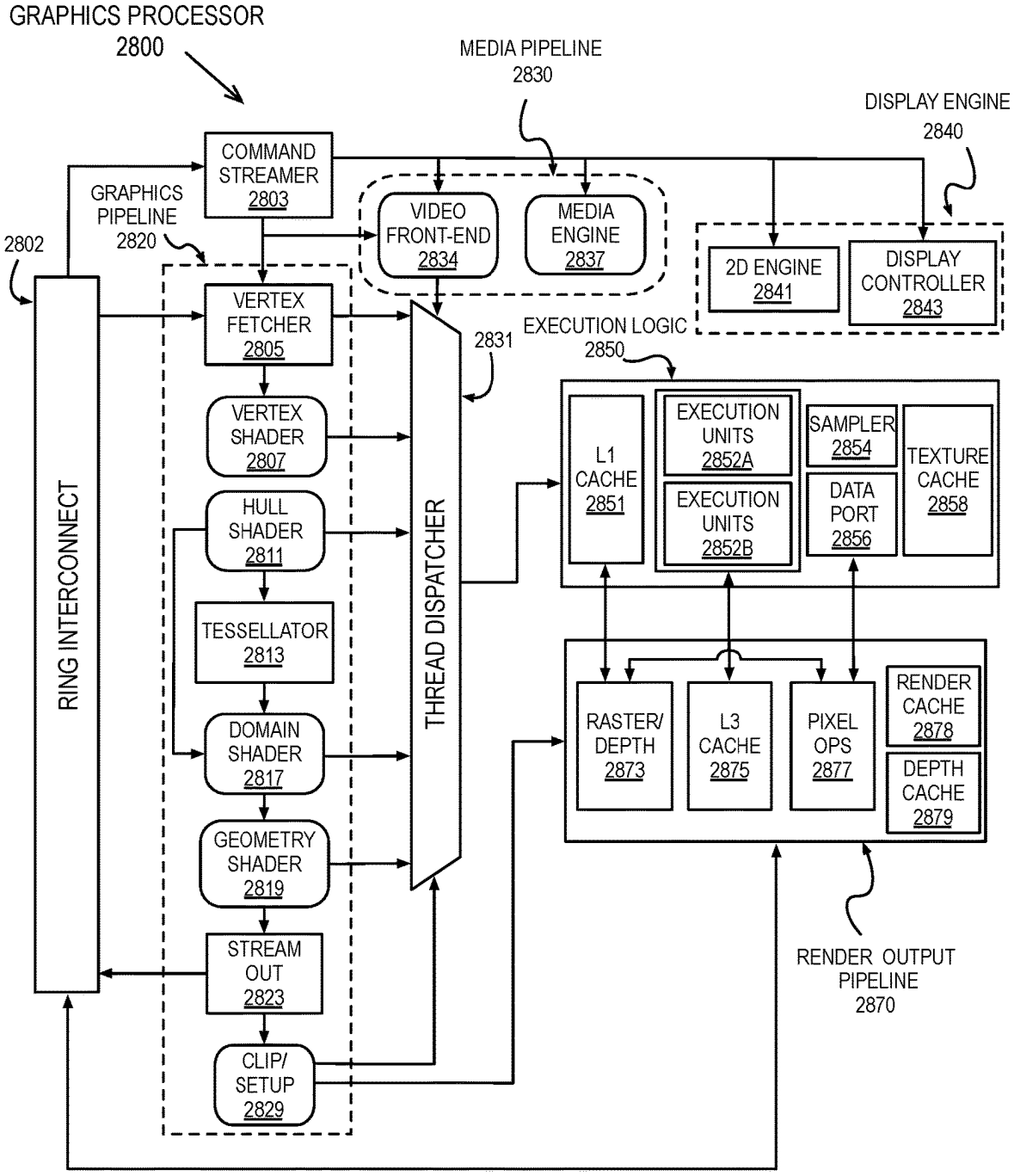

GRAPHICS PROCESSOR
2800
MEDIA PIPELINE
2830
DISPLAY ENGINE
2840
COMMAND STREAMER
2803
GRAPHICS PIPELINE
2820
2802
VIDEO FRONT-END
2834
MEDIA ENGINE
2837
2D ENGINE
2841
DISPLAY CONTROLLER
2843
VERTEX FETCHER
2805
VERTEX SHADER
2807
HULL SHADER
2811
TESSELLATOR
2813
DOMAIN SHADER
2817
GEOMETRY SHADER
2819
STREAM OUT
2823
CLIP/ SETUP
2829
RING INTERCONNECT
THREAD DISPATCHER
2831
EXECUTION LOGIC
2850
L1 CACHE
2851
EXECUTION UNITS
2852A
EXECUTION UNITS
2852B
SAMPLER
2854
DATA PORT
2856
TEXTURE CACHE
2858
RASTER/ DEPTH
2873
L3 CACHE
2875
PIXEL OPS
2877
RENDER CACHE
2878
DEPTH CACHE
2879
RENDER OUTPUT PIPELINE
2870

FIG. 28

FIG. 29A GRAPHICS PROCESSOR COMMAND FORMAT
2900

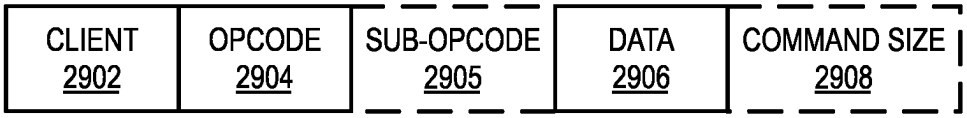
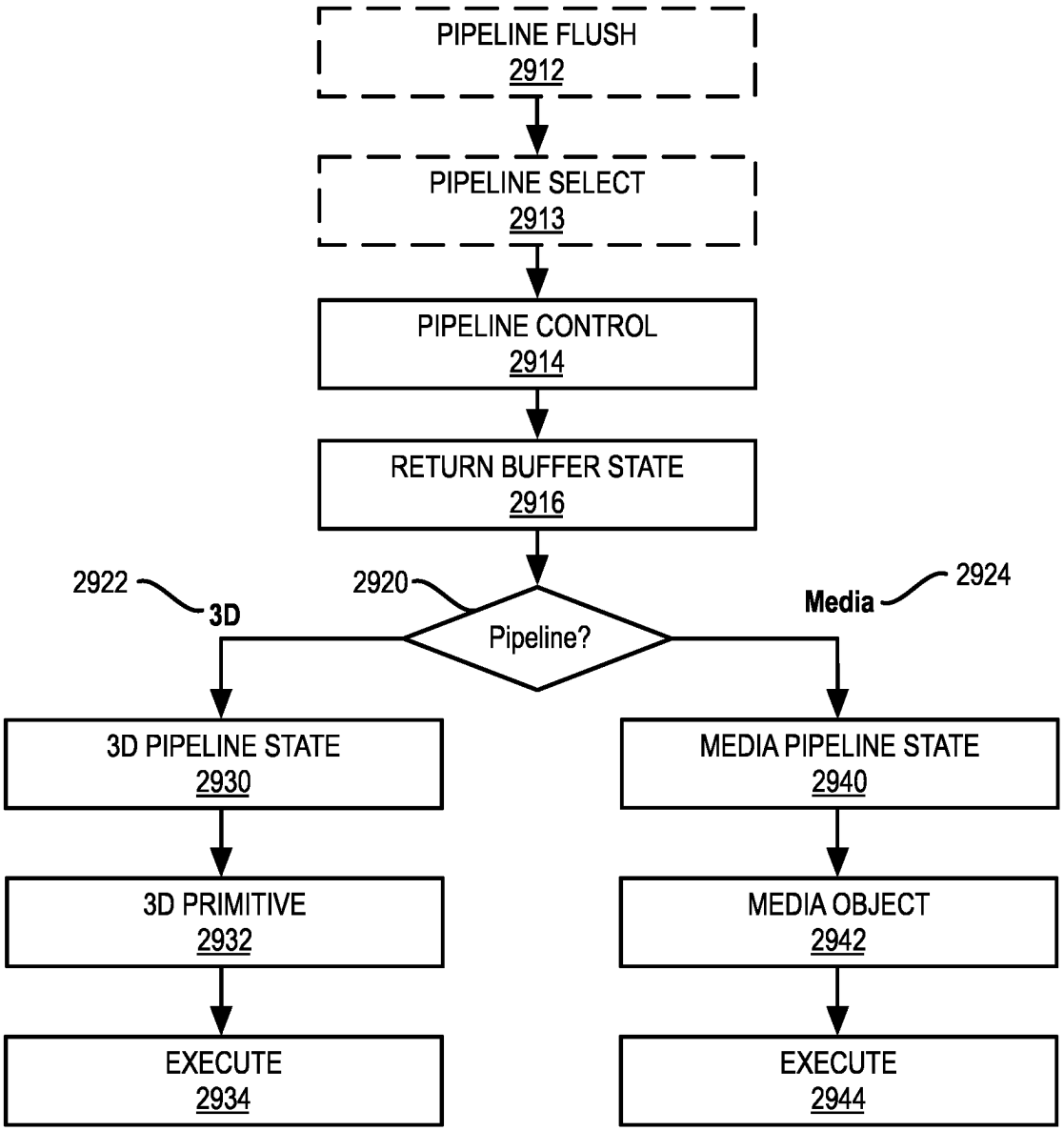



FIG. 29B GRAPHICS PROCESSOR COMMAND SEQUENCE
2910



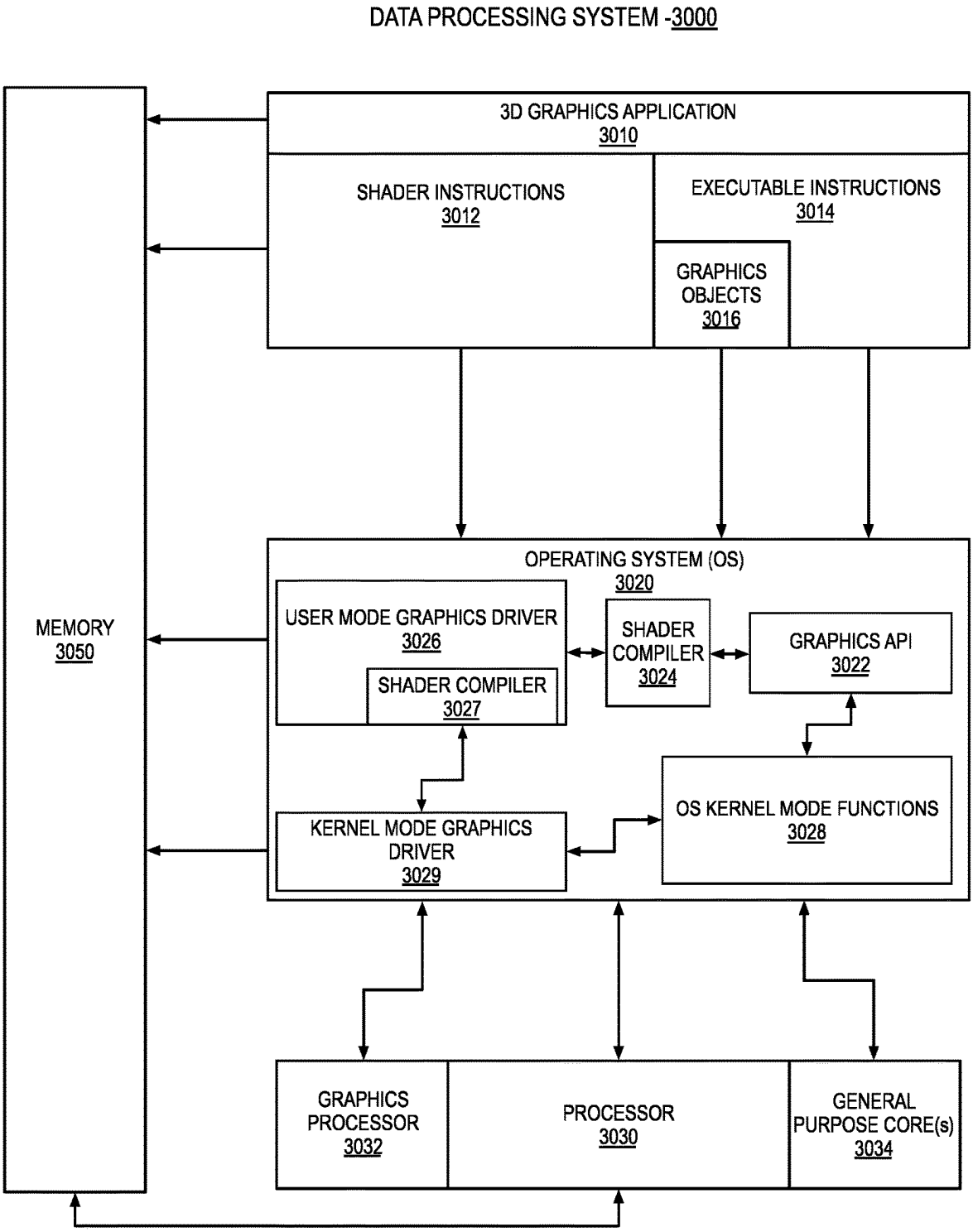

DATA PROCESSING SYSTEM -3000
3D GRAPHICS APPLICATION
3010
SHADER INSTRUCTIONS
3012
EXECUTABLE INSTRUCTIONS
3014
GRAPHICS OBJECTS
3016
MEMORY
3050
OPERATING SYSTEM (OS)
3020
USER MODE GRAPHICS DRIVER
3026
SHADER COMPILER
3027
SHADER COMPILER
3024
GRAPHICS API
3022
KERNEL MODE GRAPHICS DRIVER
3029
OS KERNEL MODE FUNCTIONS
3028
GRAPHICS PROCESSOR
3032
PROCESSOR
3030
GENERAL PURPOSE CORE(s)
3034

FIG. 30

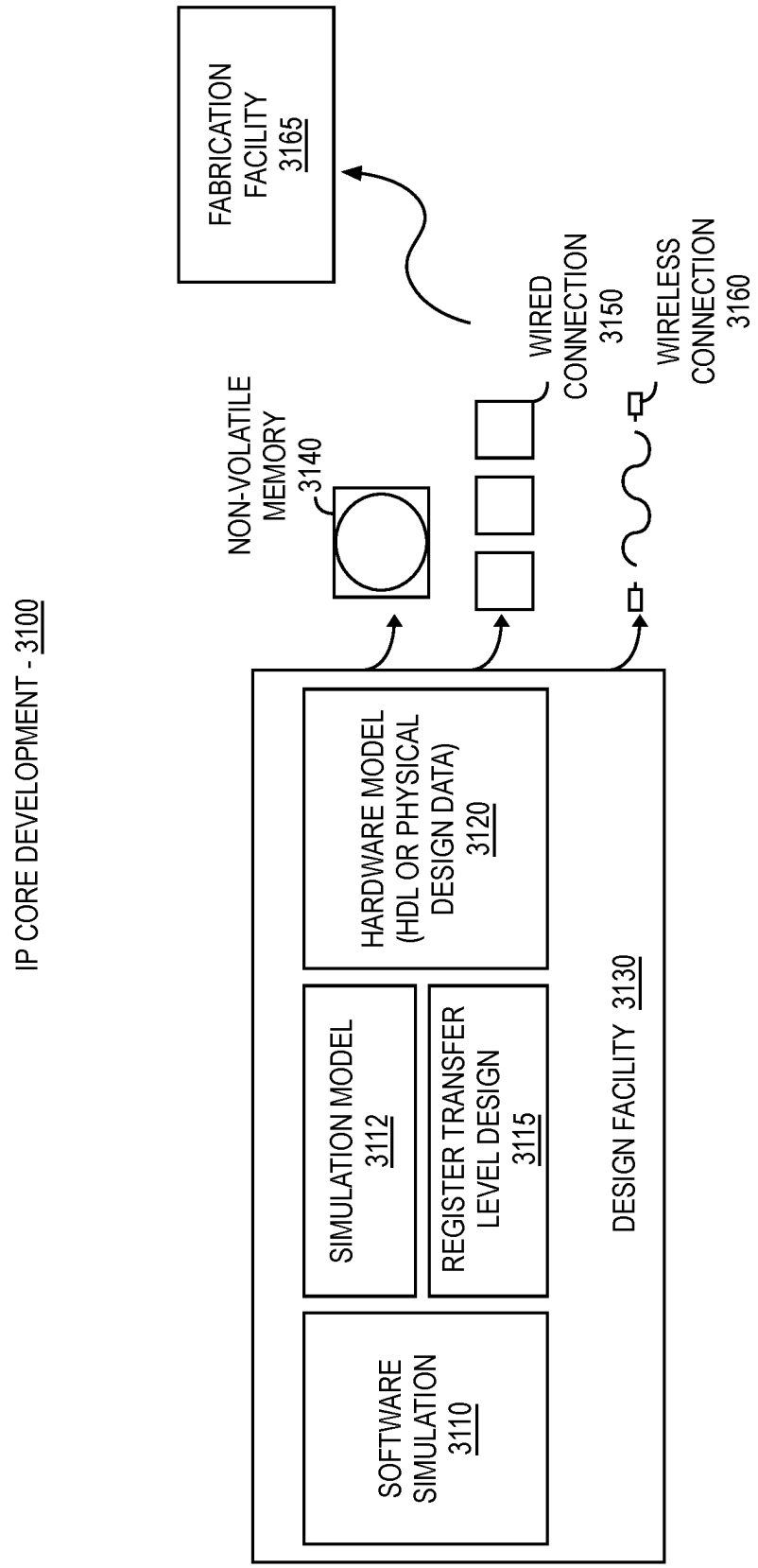

IP CORE DEVELOPMENT - 3100
DESIGN FACILITY 3130
SOFTWARE SIMULATION 3110
SIMULATION MODEL 3112
REGISTER TRANSFER LEVEL DESIGN 3115
HARDWARE MODEL (HDL OR PHYSICAL DESIGN DATA) 3120
NON-VOLATILE MEMORY 3140
WIRED CONNECTION 3150
WIRELESS CONNECTION 3160
FABRICATION FACILITY 3165

FIG. 31

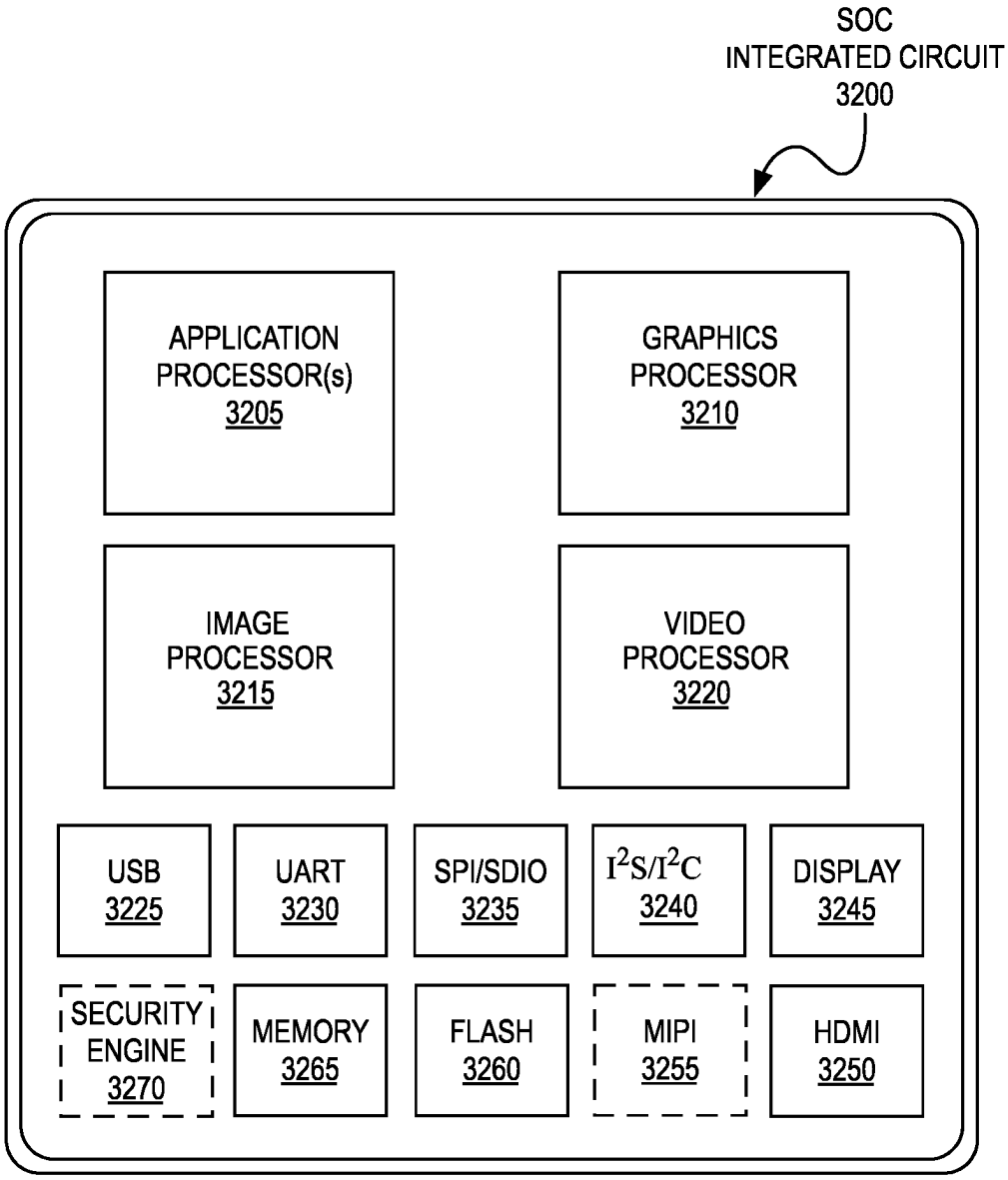

SOC
INTEGRATED CIRCUIT
3200
APPLICATION PROCESSOR(s) 3205
GRAPHICS PROCESSOR 3210
IMAGE PROCESSOR 3215
VIDEO PROCESSOR 3220
USB 3225
UART 3230
SPI/SDIO 3235
I2S/I2C 3240
DISPLAY 3245
SECURITY ENGINE 3270
MEMORY 3265
FLASH 3260
MIPI 3255
HDMI 3250

FIG. 32

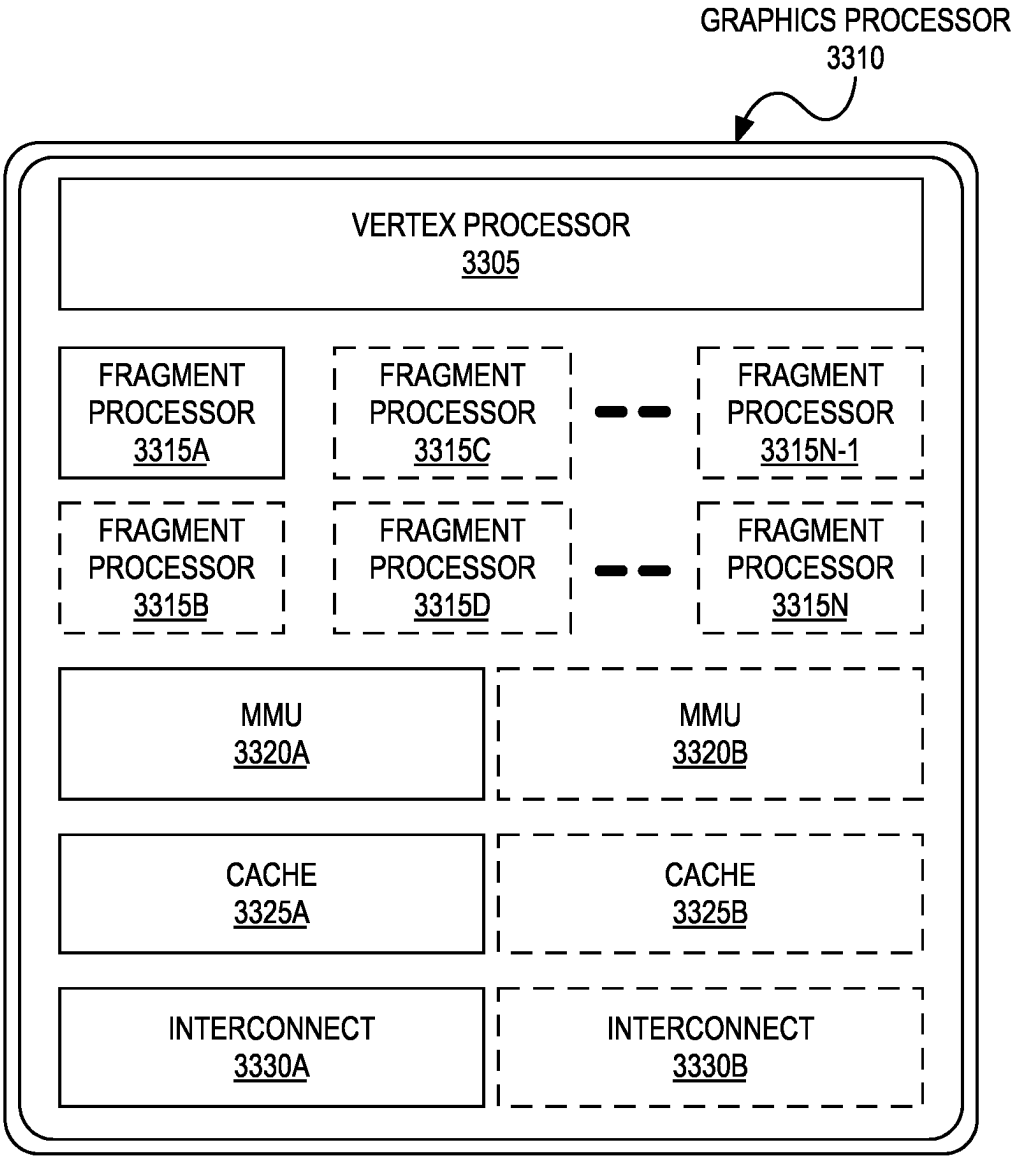

GRAPHICS PROCESSOR
3310
VERTEX PROCESSOR
3305
FRAGMENT PROCESSOR
3315A
FRAGMENT PROCESSOR
3315C
FRAGMENT PROCESSOR
3315N-1
FRAGMENT PROCESSOR
3315B
FRAGMENT PROCESSOR
3315D
FRAGMENT PROCESSOR
3315N
MMU
3320A
MMU
3320B
CACHE
3325A
CACHE
3325B
INTERCONNECT
3330A
INTERCONNECT
3330B

FIG. 33

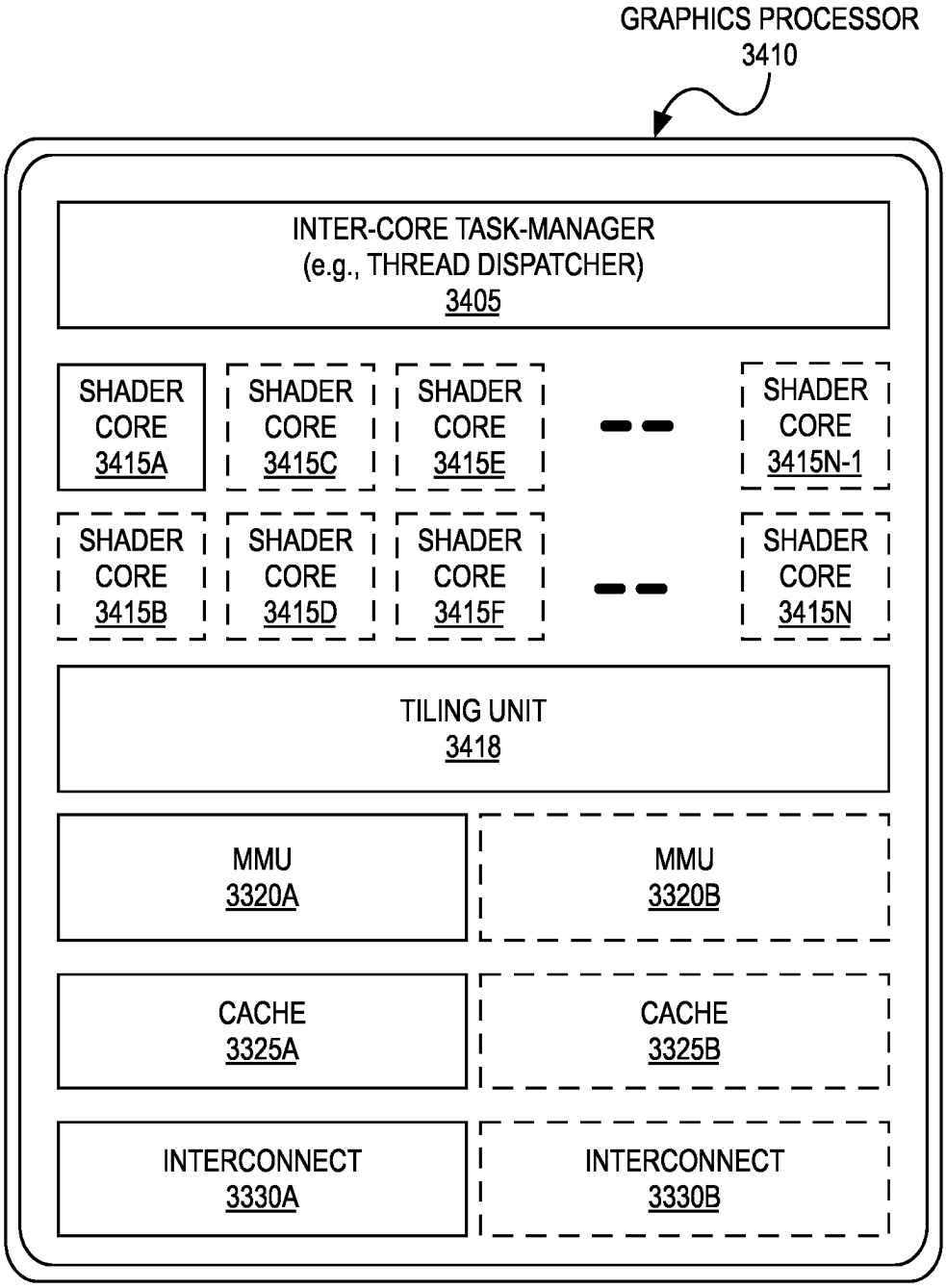

GRAPHICS PROCESSOR
3410
INTER-CORE TASK-MANAGER
(e.g., THREAD DISPATCHER)
3405
SHADER CORE 3415A
SHADER CORE 3415C
SHADER CORE 3415E
SHADER CORE 3415N-1
SHADER CORE 3415B
SHADER CORE 3415D
SHADER CORE 3415F
SHADER CORE 3415N
TILING UNIT
3418
MMU
3320A
MMU
3320B
CACHE
3325A
CACHE
3325B
INTERCONNECT
3330A
INTERCONNECT
3330B

FIG. 34

# MULTICORE PROCESSOR WITH EACH CORE HAVING INDEPENDENT FLOATING POINT DATAPATH AND INTEGER DATAPATH

## CROSS-REFERENCE

This application is a continuation of U.S. application Ser. No. 17/839,856, filed Jun. 14, 2022, which is a continuation of U.S. application Ser. No. 15/819,167, filed Nov. 21, 2017, now issued as U.S. Pat. No. 11,409,537, which is a continuation of U.S. Pat. No. 10,409,614, issued on Sep. 10, 2019, the entire contents of which are hereby incorporated herein by reference.

## FIELD

Embodiments relate generally to data processing and more particularly to data processing via a general-purpose graphics processing unit.

## BACKGROUND OF THE DESCRIPTION

Current parallel graphics data processing includes systems and methods developed to perform specific operations on graphics data such as, for example, linear interpolation, tessellation, rasterization, texture mapping, depth testing, etc. Traditionally, graphics processors used fixed function computational units to process graphics data; however, more recently, portions of graphics processors have been made programmable, enabling such processors to support a wider variety of operations for processing vertex and fragment data.

To further increase performance, graphics processors typically implement processing techniques such as pipelining that attempt to process, in parallel, as much graphics data as possible throughout the different parts of the graphics pipeline. Parallel graphics processors with single instruction, multiple thread (SIMT) architectures are designed to maximize the amount of parallel processing in the graphics pipeline. In an SIMT architecture, groups of parallel threads attempt to execute program instructions synchronously together as often as possible to increase processing efficiency. A general overview of software and hardware for SIMT architectures can be found in Shane Cook, *CUDA Programming*, Chapter 3, pages 37-51 (2013) and/or Nicholas Wilt, CUDA Handbook, *A Comprehensive Guide to GPU Programming*, Sections 2.6.2 to 3.1.2 (June 2013).

## BRIEF DESCRIPTION OF THE DRAWINGS

The present invention is illustrated by way of example and not limitation in the figures of the accompanying drawings in which like references indicate similar elements, and in which:

FIG. **1** is a block diagram illustrating a computer system configured to implement one or more aspects of the embodiments described herein;

FIG. **2**A-**2**D illustrate parallel processor components, according to an embodiment;

FIG. **3**A-**3**B are block diagrams of graphics multiprocessors, according to embodiments;

FIG. **4**A-**4**F illustrate an exemplary architecture in which a plurality of GPUs is communicatively coupled to a plurality of multi-core processors;

FIG. **5** illustrates a graphics processing pipeline, according to an embodiment;



FIG. **6** illustrates a machine learning software stack, according to an embodiment;

FIG. **7** illustrates a highly-parallel general-purpose graphics processing unit, according to an embodiment;

FIG. **8** illustrates a multi-GPU computing system, according to an embodiment;

FIG. **9**A-**9**B illustrate layers of exemplary deep neural networks;

FIG. **10** illustrates an exemplary recurrent neural network;

FIG. **11** illustrates training and deployment of a deep neural network;

FIG. **12** is a block diagram illustrating distributed learning;

FIG. **13** illustrates an exemplary inferencing system on a chip (SOC) suitable for performing inferencing using a trained model;

FIG. **14** is a block diagram of a multiprocessor unit, according to an embodiment;

FIG. **15** illustrates a mixed-precision processing system, according to an embodiment;

FIG. **16** illustrates an additional mixed-precision processing system, according to an embodiment;

FIG. **17** is a flow diagram of operational logic for a mixed-precision processing system, according to an embodiment;

FIG. **18** is a flow diagram of operational logic for another mixed-precision processing system, according to an embodiment;

FIG. **19** illustrates a machine learning system, according to an embodiment;

FIG. **20** illustrates logic operations of a machine learning system, according to an embodiment;

FIG. **21** is a block diagram of a processing system, according to an embodiment;

FIG. **22** is a block diagram of a processor according to an embodiment;

FIG. **23** is a block diagram of a graphics processor, according to an embodiment;

FIG. **24** is a block diagram of a graphics processing engine of a graphics processor in accordance with some embodiments;

FIG. **25** is a block diagram of a graphics processor provided by an additional embodiment;

FIG. **26** illustrates thread execution logic including an array of processing elements employed in some embodiments;

FIG. **27** is a block diagram illustrating graphics processor instruction formats according to some embodiments;

FIG. **28** is a block diagram of a graphics processor according to another embodiment.

FIG. **29**A-**29**B illustrate a graphics processor command format and command sequence, according to some embodiments;

FIG. **30** illustrates exemplary graphics software architecture for a data processing system according to some embodiments;

FIG. **31** is a block diagram illustrating an IP core development system, according to an embodiment;

FIG. **32** is a block diagram illustrating an exemplary system on a chip integrated circuit, according to an embodiment;

FIG. **33** is a block diagram illustrating an additional graphics processor, according to an embodiment; and

FIG. **34** is a block diagram illustrating an additional exemplary graphics processor of a system on a chip integrated circuit, according to an embodiment.

## DETAILED DESCRIPTION

In some embodiments, a graphics processing unit (GPU) is communicatively coupled to host/processor cores to accelerate graphics operations, machine-learning operations, pattern analysis operations, and various general-purpose GPU (GPGPU) functions. The GPU may be communicatively coupled to the host processor/cores over a bus or another interconnect (e.g., a high-speed interconnect such as PCIe or NVLink). In other embodiments, the GPU may be integrated on the same package or chip as the cores and communicatively coupled to the cores over an internal processor bus/interconnect (i.e., internal to the package or chip). Regardless of the manner in which the GPU is connected, the processor cores may allocate work to the GPU in the form of sequences of commands/instructions contained in a work descriptor. The GPU then uses dedicated circuitry/logic for efficiently processing these commands/instructions.

In the following description, numerous specific details are set forth to provide a more thorough understanding. However, it will be apparent to one of skill in the art that the embodiments described herein may be practiced without one or more of these specific details. In other instances, well-known features have not been described to avoid obscuring the details of the present embodiments.

System Overview

FIG. **1** is a block diagram illustrating a computing system **100** configured to implement one or more aspects of the embodiments described herein. The computing system **100** includes a processing subsystem **101** having one or more processor(s) **102** and a system memory **104** communicating via an interconnection path that may include a memory hub **105**. The memory hub **105** may be a separate component within a chipset component or may be integrated within the one or more processor(s) **102**. The memory hub **105** couples with an I/O subsystem **111** via a communication link **106**. The I/O subsystem **111** includes an I/O hub **107** that can enable the computing system **100** to receive input from one or more input device(s) **108**. Additionally, the I/O hub **107** can enable a display controller, which may be included in the one or more processor(s) **102**, to provide outputs to one or more display device(s) **110**A. In one embodiment the one or more display device(s) **110**A coupled with the I/O hub **107** can include a local, internal, or embedded display device.

In one embodiment the processing subsystem **101** includes one or more parallel processor(s) **112** coupled to memory hub **105** via a bus or other communication link **113**. The communication link **113** may be one of any number of standards-based communication link technologies or protocols, such as, but not limited to PCI Express, or may be a vendor specific communications interface or communications fabric. In one embodiment the one or more parallel processor(s) **112** form a computationally focused parallel or vector processing system that can include a large number of processing cores and/or processing clusters, such as a many integrated core (MIC) processor. In one embodiment the one or more parallel processor(s) **112** form a graphics processing subsystem that can output pixels to one of the one or more display device(s) **110**A coupled via the I/O hub **107**. The one or more parallel processor(s) **112** can also include a display controller and display interface (not shown) to enable a direct connection to one or more display device(s) **110**B.

Within the I/O subsystem **111**, a system storage unit **114** can connect to the I/O hub **107** to provide a storage mechanism for the computing system **100**. An I/O switch **116** can be used to provide an interface mechanism to enable connections between the I/O hub **107** and other components, such as a network adapter **118** and/or wireless network adapter **119** that may be integrated into the platform, and various other devices that can be added via one or more add-in device(s) **120**. The network adapter **118** can be an Ethernet adapter or another wired network adapter. The wireless network adapter **119** can include one or more of a Wi-Fi, Bluetooth, near field communication (NFC), or other network device that includes one or more wireless radios.

The computing system **100** can include other components not explicitly shown, including USB or other port connections, optical storage drives, video capture devices, and the like, may also be connected to the I/O hub **107**. Communication paths interconnecting the various components in FIG. **1** may be implemented using any suitable protocols, such as PCI (Peripheral Component Interconnect) based protocols (e.g., PCI-Express), or any other bus or point-to-point communication interfaces and/or protocol(s), such as the NV-Link high-speed interconnect, or interconnect protocols known in the art.

In one embodiment, the one or more parallel processor(s) **112** incorporate circuitry optimized for graphics and video processing, including, for example, video output circuitry, and constitutes a graphics processing unit (GPU). In another embodiment, the one or more parallel processor(s) **112** incorporate circuitry optimized for general-purpose processing, while preserving the underlying computational architecture, described in greater detail herein. In yet another embodiment, components of the computing system **100** may be integrated with one or more other system elements on a single integrated circuit. For example, the one or more parallel processor(s) **112**, memory hub **105**, processor(s) **102**, and I/O hub **107** can be integrated into a system on chip (SoC) integrated circuit. Alternatively, the components of the computing system **100** can be integrated into a single package to form a system in package (SIP) configuration. In one embodiment at least a portion of the components of the computing system **100** can be integrated into a multi-chip module (MCM), which can be interconnected with other multi-chip modules into a modular computing system.

It will be appreciated that the computing system **100** shown herein is illustrative and that variations and modifications are possible. The connection topology, including the number and arrangement of bridges, the number of processor(s) **102**, and the number of parallel processor(s) **112**, may be modified as desired. For instance, in some embodiments, system memory **104** is connected to the processor(s) **102** directly rather than through a bridge, while other devices communicate with system memory **104** via the memory hub **105** and the processor(s) **102**. In other alternative topologies, the parallel processor(s) **112** are connected to the I/O hub **107** or directly to one of the one or more processor(s) **102**, rather than to the memory hub **105**. In other embodiments, the I/O hub **107** and memory hub **105** may be integrated into a single chip. Some embodiments may include two or more sets of processor(s) **102** attached via multiple sockets, which can couple with two or more instances of the parallel processor(s) **112**.

Some of the particular components shown herein are optional and may not be included in all implementations of the computing system **100**. For example, any number of add-in cards or peripherals may be supported, or some components may be eliminated. Furthermore, some architectures may use different terminology for components similar to those illustrated in FIG. **1**. For example, the memory hub **105** may be referred to as a Northbridge in some architectures, while the I/O hub **107** may be referred to as a Southbridge.

FIG. **2**A illustrates a parallel processor **200**, according to an embodiment. The various components of the parallel processor **200** may be implemented using one or more integrated circuit devices, such as programmable processors, application specific integrated circuits (ASICs), or field programmable gate arrays (FPGA). The illustrated parallel processor **200** is a variant of the one or more parallel processor(s) **112** shown in FIG. **1**, according to an embodiment.

In one embodiment the parallel processor **200** includes a parallel processing unit **202**. The parallel processing unit includes an I/O unit **204** that enables communication with other devices, including other instances of the parallel processing unit **202**. The I/O unit **204** may be directly connected to other devices. In one embodiment the I/O unit **204** connects with other devices via the use of a hub or switch interface, such as memory hub **105**. The connections between the memory hub **105** and the I/O unit **204** form a communication link **113**. Within the parallel processing unit **202**, the I/O unit **204** connects with a host interface **206** and a memory crossbar **216**, where the host interface **206** receives commands directed to performing processing operations and the memory crossbar **216** receives commands directed to performing memory operations.

When the host interface **206** receives a command buffer via the I/O unit **204**, the host interface **206** can direct work operations to perform those commands to a front end **208**. In one embodiment the front end **208** couples with a scheduler **210**, which is configured to distribute commands or other work items to a processing cluster array **212**. In one embodiment the scheduler **210** ensures that the processing cluster array **212** is properly configured and in a valid state before tasks are distributed to the processing clusters of the processing cluster array **212**. In one embodiment the scheduler **210** is implemented via firmware logic executing on a microcontroller. The microcontroller implemented scheduler **210** is configurable to perform complex scheduling and work distribution operations at coarse and fine granularity, enabling rapid preemption and context switching of threads executing on the processing cluster array **212**. In one embodiment, the host software can provide workloads for scheduling on the processing cluster array **212** via one of multiple graphics processing doorbells. The workloads can then be automatically distributed across the processing cluster array **212** by the scheduler **210** logic within the scheduler microcontroller.

The processing cluster array **212** can include up to "N" processing clusters (e.g., cluster **214**A, cluster **214**B, through cluster **214**N). Each cluster **214**A-**214**N of the processing cluster array **212** can execute a large number of concurrent threads. The scheduler **210** can allocate work to the clusters **214**A-**214**N of the processing cluster array **212** using various scheduling and/or work distribution algorithms, which may vary depending on the workload arising for each type of program or computation. The scheduling can be handled dynamically by the scheduler **210**, or can be assisted in part by compiler logic during compilation of program logic configured for execution by the processing cluster array **212**. In one embodiment, different clusters **214**A-**214**N of the processing cluster array **212** can be allocated for processing different types of programs or for performing different types of computations.

The processing cluster array **212** can be configured to perform various types of parallel processing operations. In one embodiment the processing cluster array **212** is configured to perform general-purpose parallel compute operations. For example, the processing cluster array **212** can include logic to execute processing tasks including filtering of video and/or audio data, performing modeling operations, including physics operations, and performing data transformations.

In one embodiment the processing cluster array **212** is configured to perform parallel graphics processing operations. In embodiments in which the parallel processor **200** is configured to perform graphics processing operations, the processing cluster array **212** can include additional logic to support the execution of such graphics processing operations, including, but not limited to texture sampling logic to perform texture operations, as well as tessellation logic and other vertex processing logic. Additionally, the processing cluster array **212** can be configured to execute graphics processing related shader programs such as, but not limited to vertex shaders, tessellation shaders, geometry shaders, and pixel shaders. The parallel processing unit **202** can transfer data from system memory via the I/O unit **204** for processing. During processing the transferred data can be stored to on-chip memory (e.g., parallel processor memory **222**) during processing, then written back to system memory.

In one embodiment, when the parallel processing unit **202** is used to perform graphics processing, the scheduler **210** can be configured to divide the processing workload into approximately equal sized tasks, to better enable distribution of the graphics processing operations to multiple clusters **214**A-**214**N of the processing cluster array **212**. In some embodiments, portions of the processing cluster array **212** can be configured to perform different types of processing. For example a first portion may be configured to perform vertex shading and topology generation, a second portion may be configured to perform tessellation and geometry shading, and a third portion may be configured to perform pixel shading or other screen space operations, to produce a rendered image for display. Intermediate data produced by one or more of the clusters **214**A-**214**N may be stored in buffers to allow the intermediate data to be transmitted between clusters **214**A-**214**N for further processing.

During operation, the processing cluster array **212** can receive processing tasks to be executed via the scheduler **210**, which receives commands defining processing tasks from front end **208**. For graphics processing operations, processing tasks can include indices of data to be processed, e.g., surface (patch) data, primitive data, vertex data, and/or pixel data, as well as state parameters and commands defining how the data is to be processed (e.g., what program is to be executed). The scheduler **210** may be configured to fetch the indices corresponding to the tasks or may receive the indices from the front end **208**. The front end **208** can be configured to ensure the processing cluster array **212** is configured to a valid state before the workload specified by incoming command buffers (e.g., batch-buffers, push buffers, etc.) is initiated.

Each of the one or more instances of the parallel processing unit **202** can couple with parallel processor memory **222**. The parallel processor memory **222** can be accessed via the memory crossbar **216**, which can receive memory requests from the processing cluster array **212** as well as the I/O unit **204**. The memory crossbar **216** can access the parallel processor memory **222** via a memory interface **218**. The memory interface **218** can include multiple partition units (e.g., partition unit **220**A, partition unit **220**B, through partition unit **220**N) that can each couple to a portion (e.g., memory unit) of parallel processor memory **222**. In one implementation the number of partition units **220**A-**220**N is configured to be equal to the number of memory units, such

that a first partition unit **220**A has a corresponding first memory unit **224**A, a second partition unit **220**B has a corresponding memory unit **224**B, and an Nth partition unit **220**N has a corresponding Nth memory unit **224**N. In other embodiments, the number of partition units **220**A-**220**N may not be equal to the number of memory devices.

In various embodiments, the memory units **224**A-**224**N can include various types of memory devices, including dynamic random-access memory (DRAM) or graphics random-access memory, such as synchronous graphics random-access memory (SGRAM), including graphics double data rate (GDDR) memory. In one embodiment, the memory units **224**A-**224**N may also include 3D stacked memory, including but not limited to high bandwidth memory (HBM). Persons skilled in the art will appreciate that the specific implementation of the memory units **224**A-**224**N can vary, and can be selected from one of various conventional designs. Render targets, such as frame buffers or texture maps may be stored across the memory units **224**A-**224**N, allowing partition units **220**A-**220**N to write portions of each render target in parallel to efficiently use the available bandwidth of parallel processor memory **222**. In some embodiments, a local instance of the parallel processor memory **222** may be excluded in favor of a unified memory design that utilizes system memory in conjunction with local cache memory.

In one embodiment, any one of the clusters **214**A-**214**N of the processing cluster array **212** can process data that will be written to any of the memory units **224**A-**224**N within parallel processor memory **222**. The memory crossbar **216** can be configured to transfer the output of each cluster **214**A-**214**N to any partition unit **220**A-**220**N or to another cluster **214**A-**214**N, which can perform additional processing operations on the output. Each cluster **214**A-**214**N can communicate with the memory interface **218** through the memory crossbar **216** to read from or write to various external memory devices. In one embodiment the memory crossbar **216** has a connection to the memory interface **218** to communicate with the I/O unit **204**, as well as a connection to a local instance of the parallel processor memory **222**, enabling the processing units within the different processing clusters **214**A-**214**N to communicate with system memory or other memory that is not local to the parallel processing unit **202**. In one embodiment the memory crossbar **216** can use virtual channels to separate traffic streams between the clusters **214**A-**214**N and the partition units **220**A-**220**N.

While a single instance of the parallel processing unit **202** is illustrated within the parallel processor **200**, any number of instances of the parallel processing unit **202** can be included. For example, multiple instances of the parallel processing unit **202** can be provided on a single add-in card, or multiple add-in cards can be interconnected. The different instances of the parallel processing unit **202** can be configured to inter-operate even if the different instances have different numbers of processing cores, different amounts of local parallel processor memory, and/or other configuration differences. For example and in one embodiment, some instances of the parallel processing unit **202** can include higher precision floating-point units relative to other instances. Systems incorporating one or more instances of the parallel processing unit **202** or the parallel processor **200** can be implemented in a variety of configurations and form factors, including but not limited to desktop, laptop, or handheld personal computers, servers, workstations, game consoles, and/or embedded systems.

FIG. **2**B is a block diagram of a partition unit **220**, according to an embodiment. In one embodiment the partition unit **220** is an instance of one of the partition units **220**A-**220**N of FIG. **2**A. As illustrated, the partition unit **220** includes an L2 cache **221**, a frame buffer interface **225**, and a ROP **226** (raster operations unit). The L2 cache **221** is a read/write cache that is configured to perform load and store operations received from the memory crossbar **216** and ROP **226**. Read misses and urgent write-back requests are output by L2 cache **221** to frame buffer interface **225** for processing. Updates can also be sent to the frame buffer via the frame buffer interface **225** for processing. In one embodiment the frame buffer interface **225** interfaces with one of the memory units in parallel processor memory, such as the memory units **224**A-**224**N of FIG. **2**A (e.g., within parallel processor memory **222**).

In graphics applications, the ROP **226** is a processing unit that performs raster operations such as stencil, z test, blending, and the like. The ROP **226** then outputs processed graphics data that is stored in graphics memory. In some embodiments the ROP **226** includes compression logic to compress depth or color data that is written to memory and decompress depth or color data that is read from memory. The compression logic can be lossless compression logic that makes use of one or more of multiple compression algorithms. The type of compression that is performed by the ROP **226** can vary based on the statistical characteristics of the data to be compressed. For example, in one embodiment, delta color compression is performed on depth and color data on a per-tile basis.

In some embodiments, the ROP **226** is included within each processing cluster (e.g., cluster **214**A-**214**N of FIG. **2**A) instead of within the partition unit **220**. In such embodiment, read and write requests for pixel data are transmitted over the memory crossbar **216** instead of pixel fragment data. The processed graphics data may be displayed on a display device, such as one of the one or more display device(s) **110** of FIG. **1**, routed for further processing by the processor(s) **102**, or routed for further processing by one of the processing entities within the parallel processor **200** of FIG. **2**A.

FIG. **2**C is a block diagram of a processing cluster **214** within a parallel processing unit, according to an embodiment. In one embodiment the processing cluster is an instance of one of the processing clusters **214**A-**214**N of FIG. **2**A. The processing cluster **214** can be configured to execute many threads in parallel, where the term "thread" refers to an instance of a particular program executing on a particular set of input data. In some embodiments, single-instruction, multiple-data (SIMD) instruction issue techniques are used to support parallel execution of a large number of threads without providing multiple independent instruction units. In other embodiments, single-instruction, multiple-thread (SIMT) techniques are used to support parallel execution of a large number of generally synchronized threads, using a common instruction unit configured to issue instructions to a set of processing engines within each one of the processing clusters. Unlike a SIMD execution regime, where all processing engines typically execute identical instructions, SIMT execution allows different threads to more readily follow divergent execution paths through a given thread program. Persons skilled in the art will understand that a SIMD processing regime represents a functional subset of a SIMT processing regime.

Operation of the processing cluster **214** can be controlled via a pipeline manager **232** that distributes processing tasks to SIMT parallel processors. The pipeline manager **232** receives instructions from the scheduler **210** of FIG. **2**A and manages execution of those instructions via a graphics

multiprocessor **234** and/or a texture unit **236**. The illustrated graphics multiprocessor **234** is an exemplary instance of a SIMT parallel processor. However, various types of SIMT parallel processors of differing architectures may be included within the processing cluster **214**. One or more instances of the graphics multiprocessor **234** can be included within a processing cluster **214**. The graphics multiprocessor **234** can process data and a data crossbar **240** can be used to distribute the processed data to one of multiple possible destinations, including other shader units. The pipeline manager **232** can facilitate the distribution of processed data by specifying destinations for processed data to be distributed via the data crossbar **240**.

Each graphics multiprocessor **234** within the processing cluster **214** can include an identical set of functional execution logic (e.g., arithmetic logic units, load-store units, etc.). The functional execution logic can be configured in a pipelined manner in which new instructions can be issued before previous instructions are complete. The functional execution logic supports a variety of operations including integer and floating-point arithmetic, comparison operations, Boolean operations, bit-shifting, and computation of various algebraic functions. In one embodiment the same functional-unit hardware can be leveraged to perform different operations and any combination of functional units may be present.

The instructions transmitted to the processing cluster **214** constitutes a thread. A set of threads executing across the set of parallel processing engines is a thread group. A thread group executes the same program on different input data. Each thread within a thread group can be assigned to a different processing engine within a graphics multiprocessor **234**. A thread group may include fewer threads than the number of processing engines within the graphics multiprocessor **234**. When a thread group includes fewer threads than the number of processing engines, one or more of the processing engines may be idle during cycles in which that thread group is being processed. A thread group may also include more threads than the number of processing engines within the graphics multiprocessor **234**. When the thread group includes more threads than the number of processing engines within the graphics multiprocessor **234**, processing can be performed over consecutive clock cycles. In one embodiment multiple thread groups can be executed concurrently on a graphics multiprocessor **234**.

In one embodiment the graphics multiprocessor **234** includes an internal cache memory to perform load and store operations. In one embodiment, the graphics multiprocessor **234** can forego an internal cache and use a cache memory (e.g., L1 cache **248**) within the processing cluster **214**. Each graphics multiprocessor **234** also has access to L2 caches within the partition units (e.g., partition units **220**A-**220**N of FIG. **2**A) that are shared among all processing clusters **214** and may be used to transfer data between threads. The graphics multiprocessor **234** may also access off-chip global memory, which can include one or more of local parallel processor memory and/or system memory. Any memory external to the parallel processing unit **202** may be used as global memory. Embodiments in which the processing cluster **214** includes multiple instances of the graphics multiprocessor **234** can share common instructions and data, which may be stored in the L1 cache **248**.

Each processing cluster **214** may include an MMU **245** (memory management unit) that is configured to map virtual addresses into physical addresses. In other embodiments, one or more instances of the MMU **245** may reside within the memory interface **218** of FIG. **2**A. The MMU **245** includes a set of page table entries (PTEs) used to map a virtual address to a physical address of a tile and optionally a cache line index. The MMU **245** may include address translation lookaside buffers (TLB) or caches that may reside within the graphics multiprocessor **234** or the L1 cache or processing cluster **214**. The physical address is processed to distribute surface data access locality to allow efficient request interleaving among partition units. The cache line index may be used to determine whether a request for a cache line is a hit or miss.

In graphics and computing applications, a processing cluster **214** may be configured such that each graphics multiprocessor **234** is coupled to a texture unit **236** for performing texture mapping operations, e.g., determining texture sample positions, reading texture data, and filtering the texture data. Texture data is read from an internal texture L1 cache (not shown) or in some embodiments from the L1 cache within graphics multiprocessor **234** and is fetched from an L2 cache, local parallel processor memory, or system memory, as needed. Each graphics multiprocessor **234** outputs processed tasks to the data crossbar **240** to provide the processed task to another processing cluster **214** for further processing or to store the processed task in an L2 cache, local parallel processor memory, or system memory via the memory crossbar **216**. A preROP **242** (pre-raster operations unit) is configured to receive data from graphics multiprocessor **234**, direct data to ROP units, which may be located with partition units as described herein (e.g., partition units **220**A-**220**N of FIG. **2**A). The preROP **242** unit can perform optimizations for color blending, organize pixel color data, and perform address translations.

It will be appreciated that the core architecture described herein is illustrative and that variations and modifications are possible. Any number of processing units, e.g., graphics multiprocessor **234**, texture units **236**, preROPs **242**, etc., may be included within a processing cluster **214**. Further, while only one processing cluster **214** is shown, a parallel processing unit as described herein may include any number of instances of the processing cluster **214**. In one embodiment, each processing cluster **214** can be configured to operate independently of other processing clusters **214** using separate and distinct processing units, L1 caches, etc.

FIG. **2**D shows a graphics multiprocessor **234**, according to one embodiment. In such embodiment the graphics multiprocessor **234** couples with the pipeline manager **232** of the processing cluster **214**. The graphics multiprocessor **234** has an execution pipeline including but not limited to an instruction cache **252**, an instruction unit **254**, an address mapping unit **256**, a register file **258**, one or more general-purpose graphics processing unit (GPGPU) cores **262**, and one or more load/store units **266**. The GPGPU cores **262** and load/store units **266** are coupled with cache memory **272** and shared memory **270** via a memory and cache interconnect **268**.

In one embodiment, the instruction cache **252** receives a stream of instructions to execute from the pipeline manager **232**. The instructions are cached in the instruction cache **252** and dispatched for execution by the instruction unit **254**. The instruction unit **254** can dispatch instructions as thread groups (e.g., warps), with each thread of the thread group assigned to a different execution unit within GPGPU core **262**. An instruction can access any of a local, shared, or global address space by specifying an address within a unified address space. The address mapping unit **256** can be used to translate addresses in the unified address space into a distinct memory address that can be accessed by the load/store units **266**.

The register file **258** provides a set of registers for the functional units of the graphics multiprocessor **234**. The register file **258** provides temporary storage for operands connected to the data paths of the functional units (e.g., GPGPU cores **262**, load/store units **266**) of the graphics multiprocessor **234**. In one embodiment, the register file **258** is divided between each of the functional units such that each functional unit is allocated a dedicated portion of the register file **258**. In one embodiment, the register file **258** is divided between the different warps being executed by the graphics multiprocessor **234**.

The GPGPU cores **262** can each include floating-point units (FPUs) and/or integer arithmetic logic units (ALUs) that are used to execute instructions of the graphics multiprocessor **234**. The GPGPU cores **262** can be similar in architecture or can differ in architecture, according to embodiments. For example and in one embodiment, a first portion of the GPGPU cores **262** include a single precision FPU and an integer ALU while a second portion of the GPGPU cores include a double precision FPU. In one embodiment the FPUs can implement the IEEE 754-2008 standard for floating-point arithmetic or enable variable precision floating-point arithmetic. The graphics multiprocessor **234** can additionally include one or more fixed function or special function units to perform specific functions such as copy rectangle or pixel blending operations. In one embodiment one or more of the GPGPU cores can also include fixed or special function logic.

In one embodiment the GPGPU cores **262** include SIMD logic capable of performing a single instruction on multiple sets of data. In one embodiment GPGPU cores **262** can physically execute SIMD4, SIMD8, and SIMD16 instructions and logically execute SIMD1, SIMD2, and SIMD32 instructions. The SIMD instructions for the GPGPU cores can be generated at compile time by a shader compiler or automatically generated when executing programs written and compiled for single program multiple data (SPMD) or SIMT architectures. Multiple threads of a program configured for the SIMT execution model can be executed via a single SIMD instruction. For example and in one embodiment, eight SIMT threads that perform the same or similar operations can be executed in parallel via a single SIMD8 logic unit.

The memory and cache interconnect **268** is an interconnect network that connects each of the functional units of the graphics multiprocessor **234** to the register file **258** and to the shared memory **270**. In one embodiment, the memory and cache interconnect **268** is a crossbar interconnect that allows the load/store unit **266** to implement load and store operations between the shared memory **270** and the register file **258**. The register file **258** can operate at the same frequency as the GPGPU cores **262**, thus data transfer between the GPGPU cores **262** and the register file **258** is very low latency. The shared memory **270** can be used to enable communication between threads that execute on the functional units within the graphics multiprocessor **234**. The cache memory **272** can be used as a data cache for example, to cache texture data communicated between the functional units and the texture unit **236**. The shared memory **270** can also be used as a program managed cached. Threads executing on the GPGPU cores **262** can programmatically store data within the shared memory in addition to the automatically cached data that is stored within the cache memory **272**.

FIG. **3**A-**3**B illustrate additional graphics multiprocessors, according to embodiments. The illustrated graphics multiprocessors **325**, **350** are variants of the graphics multiprocessor **234** of

FIG. **2**C. The illustrated graphics multiprocessors **325**, **350** can be configured as a streaming multiprocessor (SM) capable of simultaneous execution of a large number of execution threads.

FIG. **3**A shows a graphics multiprocessor **325** according to an additional embodiment. The graphics multiprocessor **325** includes multiple additional instances of execution resource units relative to the graphics multiprocessor **234** of FIG. **2**D. For example, the graphics multiprocessor **325** can include multiple instances of the instruction unit **332**A-**332**B, register file **334**A-**334**B, and texture unit(s) **344**A-**344**B. The graphics multiprocessor **325** also includes multiple sets of graphics or compute execution units (e.g., GPGPU core **336**A-**336**B, GPGPU core **337**A-**337**B, GPGPU core **338**A-**338**B) and multiple sets of load/store units **340**A-**340**B. In one embodiment the execution resource units have a common instruction cache **330**, texture and/or data cache memory **342**, and shared memory **346**.

The various components can communicate via an interconnect fabric **327**. In one embodiment the interconnect fabric **327** includes one or more crossbar switches to enable communication between the various components of the graphics multiprocessor **325**. In one embodiment the interconnect fabric **327** is a separate, high-speed network fabric layer upon which each component of the graphics multiprocessor **325** is stacked. The components of the graphics multiprocessor **325** communicate with remote components via the interconnect fabric **327**. For example, the GPGPU cores **336**A-**336**B, **337**A-**337**B, and **3378**A-**338**B can each communicate with shared memory **346** via the interconnect fabric **327**. The interconnect fabric **327** can arbitrate communication within the graphics multiprocessor **325** to ensure a fair bandwidth allocation between components.

FIG. **3**B shows a graphics multiprocessor **350** according to an additional embodiment. The graphics processor includes multiple sets of execution resources **356**A-**356**D, where each set of execution resource includes multiple instruction units, register files, GPGPU cores, and load store units, as illustrated in FIG. **2**D and FIG. **3**A. The execution resources **356**A-**356**D can work in concert with texture unit(s) **360**A-**360**D for texture operations, while sharing an instruction cache **354**, and shared memory **362**. In one embodiment the execution resources **356**A-**356**D can share an instruction cache **354** and shared memory **362**, as well as multiple instances of a texture and/or data cache memory **358**A-**358**B. The various components can communicate via an interconnect fabric **352** similar to the interconnect fabric **327** of FIG. **3**A.

Persons skilled in the art will understand that the architecture described in FIGS. **1**, **2**A-**2**D, and **3**A-**3**B are descriptive and not limiting as to the scope of the present embodiments. Thus, the techniques described herein may be implemented on any properly configured processing unit, including, without limitation, one or more mobile application processors, one or more desktop or server central processing units (CPUs) including multi-core CPUs, one or more parallel processing units, such as the parallel processing unit **202** of FIG. **2**A, as well as one or more graphics processors or special purpose processing units, without departure from the scope of the embodiments described herein.

In some embodiments a parallel processor or GPGPU as described herein is communicatively coupled to host/pro-

cessor cores to accelerate graphics operations, machine-learning operations, pattern analysis operations, and various general-purpose GPU (GPGPU) functions. The GPU may be communicatively coupled to the host processor/cores over a bus or other interconnect (e.g., a high-speed interconnect such as PCIe or NVLink). In other embodiments, the GPU may be integrated on the same package or chip as the cores and communicatively coupled to the cores over an internal processor bus/interconnect (i.e., internal to the package or chip). Regardless of the manner in which the GPU is connected, the processor cores may allocate work to the GPU in the form of sequences of commands/instructions contained in a work descriptor. The GPU then uses dedicated circuitry/logic for efficiently processing these commands/instructions.

Techniques for GPU to Host Processor Interconnection

FIG. **4**A illustrates an exemplary architecture in which a plurality of GPUs **410-413** is communicatively coupled to a plurality of multi-core processors **405-406** over high-speed links **440**A-**440**D (e.g., buses, point-to-point interconnects, etc.). In one embodiment, the high-speed links **440**A-**440**D support a communication throughput of 4 GB/s, 30 GB/s, 80 GB/s or higher, depending on the implementation. Various interconnect protocols may be used including, but not limited to, PCIe 4.0 or 5.0 and NVLink 2.0. However, the underlying principles of the invention are not limited to any particular communication protocol or throughput.

In addition, in one embodiment, two or more of the GPUs **410-413** are interconnected over high-speed links **442**A-**442**B, which may be implemented using the same or different protocols/links than those used for high-speed links **440**A-**440**D. Similarly, two or more of the multi-core processors **405-406** may be connected over high speed link **443** which may be symmetric multi-processor (SMP) buses operating at 20 GB/s, 30 GB/s, 120 GB/s or higher. Alternatively, all communication between the various system components shown in FIG. **4**A may be accomplished using the same protocols/links (e.g., over a common interconnection fabric). As mentioned, however, the underlying principles of the invention are not limited to any particular type of interconnect technology.

In one embodiment, each multi-core processor **405-406** is communicatively coupled to a processor memory **401-402**, via memory interconnects **430**A-**430**B, respectively, and each GPU **410-413** is communicatively coupled to GPU memory **420-423** over GPU memory interconnects **450**A-**450**D, respectively. The memory interconnects **430**A-**430**B and **450**A-**450**D may utilize the same or different memory access technologies. By way of example, and not limitation, the processor memories **401-402** and GPU memories **420-423** may be volatile memories such as dynamic random-access memories (DRAMs) (including stacked DRAMs), Graphics DDR SDRAM (GDDR) (e.g., GDDR5, GDDR6), or High Bandwidth Memory (HBM) and/or may be non-volatile memories such as 3D XPoint or Nano-Ram. In one embodiment, some portion of the memories may be volatile memory and another portion may be non-volatile memory (e.g., using a two-level memory (2 LM) hierarchy).

As described below, although the various processors **405-406** and GPUs **410-413** may be physically coupled to a particular memory **401-402**, **420-423**, respectively, a unified memory architecture may be implemented in which the same virtual system address space (also referred to as the "effective address" space) is distributed among all of the various physical memories. For example, processor memories **401-402** may each comprise 64 GB of the system memory address space and GPU memories **420-423** may each comprise 32 GB of the system memory address space (resulting in a total of 256 GB addressable memory in this example).

FIG. **4**B illustrates additional details for an interconnection between a multi-core processor **407** and a graphics acceleration module **446** in accordance with one embodiment. The graphics acceleration module **446** may include one or more GPU chips integrated on a line card which is coupled to the processor **407** via the high-speed link **440**. Alternatively, the graphics acceleration module **446** may be integrated on the same package or chip as the processor **407**.

The illustrated processor **407** includes a plurality of cores **460**A-**460**D, each with a translation lookaside buffer **461**A-**461**D and one or more caches **462**A-**462**D. The cores may include various other components for executing instructions and processing data which are not illustrated to avoid obscuring the underlying principles of the invention (e.g., instruction fetch units, branch prediction units, decoders, execution units, reorder buffers, etc.). The caches **462**A-**462**D may comprise level 1 (L1) and level 2 (L2) caches. In addition, one or more shared caches **456** may be included in the caching hierarchy and shared by sets of the cores **460**A-**460**D. For example, one embodiment of the processor **407** includes 24 cores, each with its own L1 cache, twelve shared L2 caches, and twelve shared L3 caches. In this embodiment, one of the L2 and L3 caches are shared by two adjacent cores. The processor **407** and the graphics accelerator integration module **446** connect with system memory **441**, which may include processor memories **401-402**.

Coherency is maintained for data and instructions stored in the various caches **462**A-**462**D, **456** and system memory **441** via inter-core communication over a coherence bus **464**. For example, each cache may have cache coherency logic/circuitry associated therewith to communicate to over the coherence bus **464** in response to detected reads or writes to particular cache lines. In one implementation, a cache snooping protocol is implemented over the coherence bus **464** to snoop cache accesses. Cache snooping/coherency techniques are well understood by those of skill in the art and will not be described in detail here to avoid obscuring the underlying principles of the invention.

In one embodiment, a proxy circuit **425** communicatively couples the graphics acceleration module **446** to the coherence bus **464**, allowing the graphics acceleration module **446** to participate in the cache coherence protocol as a peer of the cores. In particular, an interface **435** provides connectivity to the proxy circuit **425** over high-speed link **440** (e.g., a PCIe bus, NVLink, etc.) and an interface **437** connects the graphics acceleration module **446** to the high-speed link **440**.

In one implementation, an accelerator integration circuit **436** provides cache management, memory access, context management, and interrupt management services on behalf of a plurality of graphics processing engines **431**, **432**, N of the graphics acceleration module **446**. The graphics processing engines **431**, **432**, N may each comprise a separate graphics processing unit (GPU). Alternatively, the graphics processing engines **431**, **432**, N may comprise different types of graphics processing engines within a GPU such as graphics execution units, media processing engines (e.g., video encoders/decoders), samplers, and blit engines. In other words, the graphics acceleration module may be a GPU with a plurality of graphics processing engines **431-432**, N or the graphics processing engines **431-432**, N may be individual GPUs integrated on a common package, line card, or chip.

In one embodiment, the accelerator integration circuit **436** includes a memory management unit (MMU) **439** for performing various memory management functions such as virtual-to-physical memory translations (also referred to as effective-to-real memory translations) and memory access protocols for accessing system memory **441**. The MMU **439** may also include a translation lookaside buffer (TLB) (not shown) for caching the virtual/effective to physical/real address translations. In one implementation, a cache **438** stores commands and data for efficient access by the graphics processing engines **431-432**, N. In one embodiment, the data stored in cache **438** and graphics memories **433-434**, M is kept coherent with the core caches **462**A-**462**D, **456** and system memory **411**. As mentioned, this may be accomplished via proxy circuit **425** which takes part in the cache coherency mechanism on behalf of cache **438** and memories **433-434**, M (e.g., sending updates to the cache **438** related to modifications/accesses of cache lines on processor caches **462**A-**462**D, **456** and receiving updates from the cache **438**).

A set of registers **445** store context data for threads executed by the graphics processing engines **431-432**, N and a context management circuit **448** manages the thread contexts. For example, the context management circuit **448** may perform save and restore operations to save and restore contexts of the various threads during contexts switches (e.g., where a first thread is saved and a second thread is stored so that the second thread can be execute by a graphics processing engine). For example, on a context switch, the context management circuit **448** may store current register values to a designated region in memory (e.g., identified by a context pointer). It may then restore the register values when returning to the context. In one embodiment, an interrupt management circuit **447** receives and processes interrupts received from system devices.

In one implementation, virtual/effective addresses from a graphics processing engine **431** are translated to real/physical addresses in system memory **411** by the MMU **439**. One embodiment of the accelerator integration circuit **436** supports multiple (e.g., 4, 8, 16) graphics accelerator modules **446** and/or other accelerator devices. The graphics accelerator module **446** may be dedicated to a single application executed on the processor **407** or may be shared between multiple applications. In one embodiment, a virtualized graphics execution environment is presented in which the resources of the graphics processing engines **431-432**, N are shared with multiple applications or virtual machines (VMs). The resources may be subdivided into "slices" which are allocated to different VMs and/or applications based on the processing requirements and priorities associated with the VMs and/or applications.

Thus, the accelerator integration circuit acts as a bridge to the system for the graphics acceleration module **446** and provides address translation and system memory cache services. In addition, the accelerator integration circuit **436** may provide virtualization facilities for the host processor to manage virtualization of the graphics processing engines, interrupts, and memory management.

Because hardware resources of the graphics processing engines **431-432**, N are mapped explicitly to the real address space seen by the host processor **407**, any host processor can address these resources directly using an effective address value. One function of the accelerator integration circuit **436**, in one embodiment, is the physical separation of the graphics processing engines **431-432**, N so that they appear to the system as independent units.

As mentioned, in the illustrated embodiment, one or more graphics memories **433-434**, M are coupled to each of the graphics processing engines **431-432**, N, respectively. The graphics memories **433-434**, M store instructions and data being processed by each of the graphics processing engines **431-432**, N. The graphics memories **433-434**, M may be volatile memories such as DRAMs (including stacked DRAMs), GDDR memory (e.g., GDDR5, GDDR6), or HBM, and/or may be non-volatile memories such as 3D XPoint or Nano-Ram.

In one embodiment, to reduce data traffic over the high-speed link **440**, biasing techniques are used to ensure that the data stored in graphics memories **433-434**, M is data which will be used most frequently by the graphics processing engines **431-432**, N and preferably not used by the cores **460**A-**460**D (at least not frequently). Similarly, the biasing mechanism attempts to keep data needed by the cores (and preferably not the graphics processing engines **431-432**, N) within the caches **462**A-**462**D, **456** of the cores and system memory **411**.

FIG. **4**C illustrates another embodiment in which the accelerator integration circuit **436** is integrated within the processor **407**. In this embodiment, the graphics processing engines **431-432**, N communicate directly over the high-speed link **440** to the accelerator integration circuit **436** via interface **437** and interface **435** (which, again, may be utilize any form of bus or interface protocol). The accelerator integration circuit **436** may perform the same operations as those described with respect to FIG. **4**B, but potentially at a higher throughput given its close proximity to the coherence bus **464** and caches **462**A-**462**D, **456**.

One embodiment supports different programming models including a dedicated-process programming model (no graphics acceleration module virtualization) and shared programming models (with virtualization). The latter may include programming models which are controlled by the accelerator integration circuit **436** and programming models which are controlled by the graphics acceleration module **446**.

In one embodiment of the dedicated process model, graphics processing engines **431-432**, N are dedicated to a single application or process under a single operating system. The single application can funnel other application requests to the graphics engines **431-432**, N, providing virtualization within a VM/partition.

In the dedicated-process programming models, the graphics processing engines **431-432**, N, may be shared by multiple VM/application partitions. The shared models require a system hypervisor to virtualize the graphics processing engines **431-432**, N to allow access by each operating system. For single-partition systems without a hypervisor, the graphics processing engines **431-432**, N are owned by the operating system. In both cases, the operating system can virtualize the graphics processing engines **431-432**, N to provide access to each process or application.

For the shared programming model, the graphics acceleration module **446** or an individual graphics processing engine **431-432**, N selects a process element using a process handle. In one embodiment, process elements are stored in system memory **411** and are addressable using the effective address to real address translation techniques described herein. The process handle may be an implementation-specific value provided to the host process when registering its context with the graphics processing engine **431-432**, N (that is, calling system software to add the process element to the process element linked list). The lower 16-bits of the process handle may be the offset of the process element within the process element linked list.

FIG. 4D illustrates an exemplary accelerator integration slice **490**. As used herein, a "slice" comprises a specified portion of the processing resources of the accelerator integration circuit **436**. Application effective address space **482** within system memory **411** stores process elements **483**. In one embodiment, the process elements **483** are stored in response to GPU invocations **481** from applications **480** executed on the processor **407**. A process element **483** contains the process state for the corresponding application **480**. A work descriptor (WD) **484** contained in the process element **483** can be a single job requested by an application or may contain a pointer to a queue of jobs. In the latter case, the WD **484** is a pointer to the job request queue in the application's address space **482**.

The graphics acceleration module **446** and/or the individual graphics processing engines **431-432**, N can be shared by all or a subset of the processes in the system. Embodiments of the invention include an infrastructure for setting up the process state and sending a WD **484** to a graphics acceleration module **446** to start a job in a virtualized environment.

In one implementation, the dedicated-process programming model is implementation-specific. In this model, a single process owns the graphics acceleration module **446** or an individual graphics processing engine **431**. Because the graphics acceleration module **446** is owned by a single process, the hypervisor initializes the accelerator integration circuit **436** for the owning partition and the operating system initializes the accelerator integration circuit **436** for the owning process at the time when the graphics acceleration module **446** is assigned.

In operation, a WD fetch unit **491** in the accelerator integration slice **490** fetches the next WD **484** which includes an indication of the work to be done by one of the graphics processing engines of the graphics acceleration module **446**. Data from the WD **484** may be stored in registers **445** and used by the MMU **439**, interrupt management circuit **447** and/or context management circuit **448** as illustrated. For example, one embodiment of the MMU **439** includes segment/page walk circuitry for accessing segment/page tables **486** within the OS virtual address space **485**. The interrupt management circuit **447** may process interrupt events **492** received from the graphics acceleration module **446**. When performing graphics operations, an effective address **493** generated by a graphics processing engine **431-432**, N is translated to a real address by the MMU **439**.

In one embodiment, the same set of registers **445** are duplicated for each graphics processing engine **431-432**, N and/or graphics acceleration module **446** and may be initialized by the hypervisor or operating system. Each of these duplicated registers may be included in an accelerator integration slice **490**. Exemplary registers that may be initialized by the hypervisor are shown in Table 1.

TABLE 1

| Hypervisor Initialized Registers | |
|---|---|
| 1 | Slice Control Register |
| 2 | Real Address (RA) Scheduled Processes Area Pointer |
| 3 | Authority Mask Override Register |
| 4 | Interrupt Vector Table Entry Offset |
| 5 | Interrupt Vector Table Entry Limit |
| 6 | State Register |
| 7 | Logical Partition ID |
| 8 | Real address (RA) Hypervisor Accelerator Utilization Record Pointer |
| 9 | Storage Description Register |

Exemplary registers that may be initialized by the operating system are shown in Table 2.

TABLE 2

| Operating System Initialized Registers | |
|---|---|
| 1 | Process and Thread Identification |
| 2 | Effective Address (EA) Context Save/Restore Pointer |
| 3 | Virtual Address (VA) Accelerator Utilization Record Pointer |
| 4 | Virtual Address (VA) Storage Segment Table Pointer |
| 5 | Authority Mask |
| 6 | Work descriptor |

In one embodiment, each WD **484** is specific to a particular graphics acceleration module **446** and/or graphics processing engine **431-432**, N. It contains all the information a graphics processing engine **431-432**, N requires to do its work or it can be a pointer to a memory location where the application has set up a command queue of work to be completed.

FIG. 4E illustrates additional details for one embodiment of a shared model. This embodiment includes a hypervisor real address space **498** in which a process element list **499** is stored. The hypervisor real address space **498** is accessible via a hypervisor **496** which virtualizes the graphics acceleration module engines for the operating system **495**.

The shared programming models allow for all or a subset of processes from all or a subset of partitions in the system to use a graphics acceleration module **446**. There are two programming models where the graphics acceleration module **446** is shared by multiple processes and partitions: time-sliced shared and graphics directed shared.

In this model, the system hypervisor **496** owns the graphics acceleration module **446** and makes its function available to all operating systems **495**. For a graphics acceleration module **446** to support virtualization by the system hypervisor **496**, the graphics acceleration module **446** may adhere to the following requirements: 1) An application's job request must be autonomous (that is, the state does not need to be maintained between jobs), or the graphics acceleration module **446** must provide a context save and restore mechanism. 2) An application's job request is guaranteed by the graphics acceleration module **446** to complete in a specified amount of time, including any translation faults, or the graphics acceleration module **446** provides the ability to preempt the processing of the job. 3) The graphics acceleration module **446** must be guaranteed fairness between processes when operating in the directed shared programming model.

In one embodiment, for the shared model, the application **480** is required to make an operating system **495** system call with a graphics acceleration module **446** type, a work descriptor (WD), an authority mask register (AMR) value, and a context save/restore area pointer (CSRP). The graphics acceleration module **446** type describes the targeted acceleration function for the system call. The graphics acceleration module **446** type may be a system-specific value. The WD is formatted specifically for the graphics acceleration module **446** and can be in the form of a graphics acceleration module **446** command, an effective address pointer to a user-defined structure, an effective address pointer to a queue of commands, or any other data structure to describe the work to be done by the graphics acceleration module **446**. In one embodiment, the AMR value is the AMR state to use for the current process. The value passed to the operating system is similar to an application setting the AMR. If the accelerator integration circuit **436** and graphics

acceleration module **446** implementations do not support a User Authority Mask Override Register (UAMOR), the operating system may apply the current UAMOR value to the AMR value before passing the AMR in the hypervisor call. The hypervisor **496** may optionally apply the current Authority Mask Override Register (AMOR) value before placing the AMR into the process element **483**. In one embodiment, the CSRP is one of the registers **445** containing the effective address of an area in the application's address space **482** for the graphics acceleration module **446** to save and restore the context state. This pointer is optional if no state is required to be saved between jobs or when a job is preempted. The context save/restore area may be pinned system memory.

Upon receiving the system call, the operating system **495** may verify that the application **480** has registered and been given the authority to use the graphics acceleration module **446**. The operating system **495** then calls the hypervisor **496** with the information shown in Table 3.

TABLE 3

| OS to Hypervisor Call Parameters | |
|---|---|
| 1 | A work descriptor (WD) |
| 2 | An Authority Mask Register (AMR) value (potentially masked). |
| 3 | An effective address (EA) Context Save/Restore Area Pointer (CSRP) |
| 4 | A process ID (PID) and optional thread ID (TID) |
| 5 | A virtual address (VA) accelerator utilization record pointer (AURP) |
| 6 | The virtual address of the storage segment table pointer (SSTP) |
| 7 | A logical interrupt service number (LISN) |

Upon receiving the hypervisor call, the hypervisor **496** verifies that the operating system **495** has registered and been given the authority to use the graphics acceleration module **446**. The hypervisor **496** then puts the process element **483** into the process element linked list for the corresponding graphics acceleration module **446** type. The process element may include the information shown in Table 4.

TABLE 4

| Process Element Information | |
|---|---|
| 1 | A work descriptor (WD) |
| 2 | An Authority Mask Register (AMR) value (potentially masked). |
| 3 | An effective address (EA) Context Save/Restore Area Pointer (CSRP) |
| 4 | A process ID (PID) and optional thread ID (TID) |
| 5 | A virtual address (VA) accelerator utilization record pointer (AURP) |
| 6 | The virtual address of the storage segment table pointer (SSTP) |
| 7 | A logical interrupt service number (LISN) |
| 8 | Interrupt vector table, derived from the hypervisor call parameters. |
| 9 | A state register (SR) value |
| 10 | A logical partition ID (LPID) |
| 11 | A real address (RA) hypervisor accelerator utilization record pointer |
| 12 | The Storage Descriptor Register (SDR) |

In one embodiment, the hypervisor initializes a plurality of accelerator integration slice **490** registers **445**.

As illustrated in FIG. **4**F, one embodiment of the invention employs a unified memory addressable via a common virtual memory address space used to access the physical processor memories **401-402** and GPU memories **420-423**. In this implementation, operations executed on the GPUs **410-413** utilize the same virtual/effective memory address space to access the processors memories **401-402** and vice versa, thereby simplifying programmability. In one embodiment, a first portion of the virtual/effective address space is allocated to the processor memory **401**, a second portion to the second processor memory **402**, a third portion to the GPU memory **420**, and so on. The entire virtual/effective memory space (sometimes referred to as the effective address space) is thereby distributed across each of the processor memories **401-402** and GPU memories **420-423**, allowing any processor or GPU to access any physical memory with a virtual address mapped to that memory.

In one embodiment, bias/coherence management circuitry **494**A-**494**E within one or more of the MMUs **439**A-**439**E ensures cache coherence between the caches of the host processors (e.g., **405**) and the GPUs **410-413** and implements biasing techniques indicating the physical memories in which certain types of data should be stored. While multiple instances of bias/coherence management circuitry **494**A-**494**E are illustrated in FIG. **4**F, the bias/coherence circuitry may be implemented within the MMU of one or more host processors **405** and/or within the accelerator integration circuit **436**.

One embodiment allows GPU-attached memory **420-423** to be mapped as part of system memory, and accessed using shared virtual memory (SVM) technology, but without suffering the typical performance drawbacks associated with full system cache coherence. The ability to GPU-attached memory **420-423** to be accessed as system memory without onerous cache coherence overhead provides a beneficial operating environment for GPU offload. This arrangement allows the host processor **405** software to setup operands and access computation results, without the overhead of tradition I/O DMA data copies. Such traditional copies involve driver calls, interrupts and memory mapped I/O (MMIO) accesses that are all inefficient relative to simple memory accesses. At the same time, the ability to access GPU attached memory **420-423** without cache coherence overheads can be critical to the execution time of an offloaded computation. In cases with substantial streaming write memory traffic, for example, cache coherence overhead can significantly reduce the effective write bandwidth seen by a GPU **410-413**. The efficiency of operand setup, the efficiency of results access, and the efficiency of GPU computation all play a role in determining the effectiveness of GPU offload.

In one implementation, the selection of between GPU bias and host processor bias is driven by a bias tracker data structure. A bias table may be used, for example, which may be a page-granular structure (i.e., controlled at the granularity of a memory page) that includes 1 or 2 bits per GPU-attached memory page. The bias table may be implemented in a stolen memory range of one or more GPU-attached memories **420-423**, with or without a bias cache in the GPU **410-413** (e.g., to cache frequently/recently used entries of the bias table). Alternatively, the entire bias table may be maintained within the GPU.

In one implementation, the bias table entry associated with each access to the GPU-attached memory **420-423** is accessed prior the actual access to the GPU memory, causing the following operations. First, local requests from the GPU **410-413** that find their page in GPU bias are forwarded directly to a corresponding GPU memory **420-423**. Local requests from the GPU that find their page in host bias are forwarded to the processor **405** (e.g., over a high-speed link as discussed above). In one embodiment, requests from the processor **405** that find the requested page in host processor bias complete the request like a normal memory read. Alternatively, requests directed to a GPU-biased page may

be forwarded to the GPU **410-413**. The GPU may then transition the page to a host processor bias if it is not currently using the page.

The bias state of a page can be changed either by a software-based mechanism, a hardware-assisted software-based mechanism, or, for a limited set of cases, a purely hardware-based mechanism.

One mechanism for changing the bias state employs an API call (e.g. OpenCL), which, in turn, calls the GPU's device driver which, in turn, sends a message (or enqueues a command descriptor) to the GPU directing it to change the bias state and, for some transitions, perform a cache flushing operation in the host. The cache flushing operation is required for a transition from host processor **405** bias to GPU bias, but is not required for the opposite transition.

In one embodiment, cache coherency is maintained by temporarily rendering GPU-biased pages uncacheable by the host processor **405**. To access these pages, the processor **405** may request access from the GPU **410** which may or may not grant access right away, depending on the implementation. Thus, to reduce communication between the processor **405** and GPU **410** it is beneficial to ensure that GPU-biased pages are those which are required by the GPU but not the host processor **405** and vice versa.

Graphics Processing Pipeline

FIG. **5** illustrates a graphics processing pipeline **500**, according to an embodiment. In one embodiment a graphics processor can implement the illustrated graphics processing pipeline **500**. The graphics processor can be included within the parallel processing subsystems as described herein, such as the parallel processor **200** of FIG. **2**A, which, in one embodiment, is a variant of the parallel processor(s) **112** of FIG. **1**. The various parallel processing systems can implement the graphics processing pipeline **500** via one or more instances of the parallel processing unit (e.g., parallel processing unit **202** of FIG. **2**A) as described herein. For example, a shader unit (e.g., graphics multiprocessor **234** of FIG. **2**C) may be configured to perform the functions of one or more of a vertex processing unit **504**, a tessellation control processing unit **508**, a tessellation evaluation processing unit **512**, a geometry processing unit **516**, and a fragment/pixel processing unit **524**. The functions of data assembler **502**, primitive assemblers **506**, **514**, **518**, tessellation unit **510**, rasterizer **522**, and raster operations unit **526** may also be performed by other processing engines within a processing cluster (e.g., processing cluster **214** of FIG. **2**A) and a corresponding partition unit (e.g., partition unit **220**A-**220**N of FIG. **2**A). The graphics processing pipeline **500** may also be implemented using dedicated processing units for one or more functions. In one embodiment, one or more portions of the graphics processing pipeline **500** can be performed by parallel processing logic within a general-purpose processor (e.g., CPU). In one embodiment, one or more portions of the graphics processing pipeline **500** can access on-chip memory (e.g., parallel processor memory **222** as in FIG. **2**A) via a memory interface **528**, which may be an instance of the memory interface **218** of FIG. **2**A.

In one embodiment the data assembler **502** is a processing unit that collects vertex data for surfaces and primitives. The data assembler **502** then outputs the vertex data, including the vertex attributes, to the vertex processing unit **504**. The vertex processing unit **504** is a programmable execution unit that executes vertex shader programs, lighting and transforming vertex data as specified by the vertex shader programs. The vertex processing unit **504** reads data that is stored in cache, local or system memory for use in processing the vertex data and may be programmed to transform the vertex data from an object-based coordinate representation to a world space coordinate space or a normalized device coordinate space.

A first instance of a primitive assembler **506** receives vertex attributes from the vertex processing unit **504**. The primitive assembler **506** readings stored vertex attributes as needed and constructs graphics primitives for processing by tessellation control processing unit **508**. The graphics primitives include triangles, line segments, points, patches, and so forth, as supported by various graphics processing application programming interfaces (APIs).

The tessellation control processing unit **508** treats the input vertices as control points for a geometric patch. The control points are transformed from an input representation from the patch (e.g., the patch's bases) to a representation that is suitable for use in surface evaluation by the tessellation evaluation processing unit **512**. The tessellation control processing unit **508** can also compute tessellation factors for edges of geometric patches. A tessellation factor applies to a single edge and quantifies a view-dependent level of detail associated with the edge. A tessellation unit **510** is configured to receive the tessellation factors for edges of a patch and to tessellate the patch into multiple geometric primitives such as line, triangle, or quadrilateral primitives, which are transmitted to a tessellation evaluation processing unit **512**. The tessellation evaluation processing unit **512** operates on parameterized coordinates of the subdivided patch to generate a surface representation and vertex attributes for each vertex associated with the geometric primitives.

A second instance of a primitive assembler **514** receives vertex attributes from the tessellation evaluation processing unit **512**, reading stored vertex attributes as needed, and constructs graphics primitives for processing by the geometry processing unit **516**. The geometry processing unit **516** is a programmable execution unit that executes geometry shader programs to transform graphics primitives received from primitive assembler **514** as specified by the geometry shader programs. In one embodiment the geometry processing unit **516** is programmed to subdivide the graphics primitives into one or more new graphics primitives and calculate parameters used to rasterize the new graphics primitives.

In some embodiments the geometry processing unit **516** can add or delete elements in the geometry stream. The geometry processing unit **516** outputs the parameters and vertices specifying new graphics primitives to primitive assembler **518**. The primitive assembler **518** receives the parameters and vertices from the geometry processing unit **516** and constructs graphics primitives for processing by a viewport scale, cull, and clip unit **520**. The geometry processing unit **516** reads data that is stored in parallel processor memory or system memory for use in processing the geometry data. The viewport scale, cull, and clip unit **520** performs clipping, culling, and viewport scaling and outputs processed graphics primitives to a rasterizer **522**.

The rasterizer **522** can perform depth culling and other depth-based optimizations. The rasterizer **522** also performs scan conversion on the new graphics primitives to generate fragments and output those fragments and associated coverage data to the fragment/pixel processing unit **524**. The fragment/pixel processing unit **524** is a programmable execution unit that is configured to execute fragment shader programs or pixel shader programs. The fragment/pixel processing unit **524** transforming fragments or pixels received from rasterizer **522**, as specified by the fragment or pixel shader programs. For example, the fragment/pixel

processing unit **524** may be programmed to perform operations included but not limited to texture mapping, shading, blending, texture correction and perspective correction to produce shaded fragments or pixels that are output to a raster operations unit **526**. The fragment/pixel processing unit **524** can read data that is stored in either the parallel processor memory or the system memory for use when processing the fragment data. Fragment or pixel shader programs may be configured to shade at sample, pixel, tile, or other granularities depending on the sampling rate configured for the processing units.

The raster operations unit **526** is a processing unit that performs raster operations including, but not limited to stencil, z test, blending, and the like, and outputs pixel data as processed graphics data to be stored in graphics memory (e.g., parallel processor memory **222** as in FIG. **2**A, and/or system memory **104** as in FIG. **1**), to be displayed on the one or more display device(s) **110** or for further processing by one of the one or more processor(s) **102** or parallel processor (s) **112**. In some embodiments the raster operations unit **526** is configured to compress z or color data that is written to memory and decompress z or color data that is read from memory.

Machine Learning Overview

A machine learning algorithm is an algorithm that can learn based on a set of data. Embodiments of machine learning algorithms can be designed to model high-level abstractions within a data set. For example, image recognition algorithms can be used to determine which of several categories to which a given input belong; regression algorithms can output a numerical value given an input; and pattern recognition algorithms can be used to generate translated text or perform text to speech and/or speech recognition.

An exemplary type of machine learning algorithm is a neural network. There are many types of neural networks; a simple type of neural network is a feedforward network. A feedforward network may be implemented as an acyclic graph in which the nodes are arranged in layers. Typically, a feedforward network topology includes an input layer and an output layer that are separated by at least one hidden layer. The hidden layer transforms input received by the input layer into a representation that is useful for generating output in the output layer. The network nodes are fully connected via edges to the nodes in adjacent layers, but there are no edges between nodes within each layer. Data received at the nodes of an input layer of a feedforward network are propagated (i.e., "fed forward") to the nodes of the output layer via an activation function that calculates the states of the nodes of each successive layer in the network based on coefficients ("weights") respectively associated with each of the edges connecting the layers. Depending on the specific model being represented by the algorithm being executed, the output from the neural network algorithm can take various forms.

Before a machine learning algorithm can be used to model a particular problem, the algorithm is trained using a training data set. Training a neural network involves selecting a network topology, using a set of training data representing a problem being modeled by the network, and adjusting the weights until the network model performs with a minimal error for all instances of the training data set. For example, during a supervised learning training process for a neural network, the output produced by the network in response to the input representing an instance in a training data set is compared to the "correct" labeled output for that instance, an error signal representing the difference between the output and the labeled output is calculated, and the weights associated with the connections are adjusted to minimize that error as the error signal is backward propagated through the layers of the network. The network is considered "trained" when the errors for each of the outputs generated from the instances of the training data set are minimized.

The accuracy of a machine learning algorithm can be affected significantly by the quality of the data set used to train the algorithm. The training process can be computationally intensive and may require a significant amount of time on a conventional general-purpose processor. Accordingly, parallel processing hardware is used to train many types of machine learning algorithms. This is particularly useful for optimizing the training of neural networks, as the computations performed in adjusting the coefficients in neural networks lend themselves naturally to parallel implementations. Specifically, many machine learning algorithms and software applications have been adapted to make use of the parallel processing hardware within general-purpose graphics processing devices.

FIG. **6** is a generalized diagram of a machine learning software stack **600**. A machine learning application **602** can be configured to train a neural network using a training dataset or to use a trained deep neural network to implement machine intelligence. The machine learning application **602** can include training and inference functionality for a neural network and/or specialized software that can be used to train a neural network before deployment. The machine learning application **602** can implement any type of machine intelligence including but not limited to image recognition, mapping and localization, autonomous navigation, speech synthesis, medical imaging, or language translation.

Hardware acceleration for the machine learning application **602** can be enabled via a machine learning framework **604**. The machine learning framework **604** can provide a library of machine learning primitives. Machine learning primitives are basic operations that are commonly performed by machine learning algorithms. Without the machine learning framework **604**, developers of machine learning algorithms would be required to create and optimize the main computational logic associated with the machine learning algorithm, then re-optimize the computational logic as new parallel processors are developed. Instead, the machine learning application can be configured to perform the necessary computations using the primitives provided by the machine learning framework **604**. Exemplary primitives include tensor convolutions, activation functions, and pooling, which are computational operations that are performed while training a convolutional neural network (CNN). The machine learning framework **604** can also provide primitives to implement basic linear algebra subprograms performed by many machine-learning algorithms, such as matrix and vector operations.

The machine learning framework **604** can process input data received from the machine learning application **602** and generate the appropriate input to a compute framework **606**. The compute framework **606** can abstract the underlying instructions provided to the GPGPU driver **608** to enable the machine learning framework **604** to take advantage of hardware acceleration via the GPGPU hardware **610** without requiring the machine learning framework **604** to have intimate knowledge of the architecture of the GPGPU hardware **610**. Additionally, the compute framework **606** can enable hardware acceleration for the machine learning framework **604** across a variety of types and generations of the GPGPU hardware **610**.

GPGPU Machine Learning Acceleration

FIG. **7** illustrates a highly-parallel general-purpose graphics processing unit **700**, according to an embodiment. In one embodiment the general-purpose processing unit (GPGPU) **700** can be configured to be particularly efficient in processing the type of computational workloads associated with training deep neural networks. Additionally, the GPGPU **700** can be linked directly to other instances of the GPGPU to create a multi-GPU cluster to improve training speed for particularly deep neural networks.

The GPGPU **700** includes a host interface **702** to enable a connection with a host processor. In one embodiment the host interface **702** is a PCI Express interface. However, the host interface can also be a vendor specific communications interface or communications fabric. The GPGPU **700** receives commands from the host processor and uses a global scheduler **704** to distribute execution threads associated with those commands to a set of compute clusters **706**A-**706**H. The compute clusters **706**A-**706**H share a cache memory **708**. The cache memory **708** can serve as a higher-level cache for cache memories within the compute clusters **706**A-**706**H.

The GPGPU **700** includes memory **714**A-**714**B coupled with the compute clusters **706**A-H via a set of memory controllers **712**A-**712**B. In various embodiments, the memory **714**A-**714**B can include various types of memory devices including dynamic random-access memory (DRAM) or graphics random-access memory, such as synchronous graphics random-access memory (SGRAM), including graphics double data rate (GDDR) memory or 3D stacked memory, including but not limited to high bandwidth memory (HBM).

In one embodiment each compute cluster **706**A-**706**H includes a set of graphics multiprocessors, such as the graphics multiprocessor **234** of FIG. **2**D, the graphics multiprocessor **325** of FIG. **3**A, or the graphics multiprocessor **350** of FIG. **3**B. The graphics multiprocessors of the compute cluster multiple types of integer and floating-point logic units that can perform computational operations at a range of precisions including precisions suited for machine learning computations. For example and in one embodiment at least a subset of the floating-point units in each of the compute clusters **706**A-**706**H can be configured to perform 16-bit or 32-bit floating-point operations, while a different subset of the floating-point units can be configured to perform 64-bit floating-point operations.

Multiple instances of the GPGPU **700** can be configured to operate as a compute cluster. The communication mechanism used by the compute cluster for synchronization and data exchange varies across embodiments. In one embodiment the multiple instances of the GPGPU **700** communicate over the host interface **702**. In one embodiment the GPGPU **700** includes an I/O hub **709** that couples the GPGPU **700** with a GPU link **710** that enables a direct connection to other instances of the GPGPU. In one embodiment the GPU link **710** is coupled to a dedicated GPU-to-GPU bridge that enables communication and synchronization between multiple instances of the GPGPU **700**. In one embodiment the GPU link **710** couples with a high-speed interconnect to transmit and receive data to other GPGPUs or parallel processors. In one embodiment the multiple instances of the GPGPU **700** are located in separate data processing systems and communicate via a network device that is accessible via the host interface **702**. In one embodiment the GPU link **710** can be configured to enable a connection to a host processor in addition to or as an alternative to the host interface **702**.

While the illustrated configuration of the GPGPU **700** can be configured to train neural networks, one embodiment provides alternate configuration of the GPGPU **700** that can be configured for deployment within a high performance or low power inferencing platform. In an inferencing configuration the GPGPU **700** includes fewer of the compute clusters **706**A-**706**H relative to the training configuration. Additionally memory technology associated with the memory **714**A-**714**B may differ between inferencing and training configurations. In one embodiment the inferencing configuration of the GPGPU **700** can support inferencing specific instructions. For example, an inferencing configuration can provide support for one or more 8-bit integer dot product instructions, which are commonly used during inferencing operations for deployed neural networks.

FIG. **8** illustrates a multi-GPU computing system **800**, according to an embodiment. The multi-GPU computing system **800** can include a processor **802** coupled to multiple GPGPUs **806**A-**806**D via a host interface switch **804**. The host interface switch **804**, in one embodiment, is a PCI express switch device that couples the processor **802** to a PCI express bus over which the processor **802** can communicate with the set of GPGPUs **806**A-**806**D. Each of the multiple GPGPUs **806**A-**806**D can be an instance of the GPGPU **700** of FIG. **7**. The GPGPUs **806**A-**806**D can interconnect via a set of high-speed point-to-point GPU to GPU links **816**. The high-speed GPU to GPU links can connect to each of the GPGPUs **806**A-**806**D via a dedicated GPU link, such as the GPU link **710** as in FIG. **7**. The P2P GPU links **816** enable direct communication between each of the GPGPUs **806**A-**806**D without requiring communication over the host interface bus to which the processor **802** is connected. With GPU-to-GPU traffic directed to the P2P GPU links, the host interface bus remains available for system memory access or to communicate with other instances of the multi-GPU computing system **800**, for example, via one or more network devices. While in the illustrated embodiment the GPGPUs **806**A-**806**D connect to the processor **802** via the host interface switch **804**, in one embodiment the processor **802** includes direct support for the P2P GPU links **816** and can connect directly to the GPGPUs **806**A-**806**D.

Machine Learning Neural Network Implementations

The computing architecture provided by embodiments described herein can be configured to perform the types of parallel processing that is particularly suited for training and deploying neural networks for machine learning. A neural network can be generalized as a network of functions having a graph relationship. As is well-known in the art, there are a variety of types of neural network implementations used in machine learning. One exemplary type of neural network is the feedforward network, as previously described.

A second exemplary type of neural network is the Convolutional Neural Network (CNN). A CNN is a specialized feedforward neural network for processing data having a known, grid-like topology, such as image data. Accordingly, CNNs are commonly used for compute vision and image recognition applications, but they also may be used for other types of pattern recognition such as speech and language processing. The nodes in the CNN input layer are organized into a set of "filters" (feature detectors inspired by the receptive fields found in the retina), and the output of each set of filters is propagated to nodes in successive layers of the network. The computations for a CNN include applying the convolution mathematical operation to each filter to produce the output of that filter. Convolution is a specialized kind of mathematical operation performed by two functions

to produce a third function that is a modified version of one of the two original functions. In convolutional network terminology, the first function to the convolution can be referred to as the input, while the second function can be referred to as the convolution kernel. The output may be referred to as the feature map. For example, the input to a convolution layer can be a multidimensional array of data that defines the various color components of an input image. The convolution kernel can be a multidimensional array of parameters, where the parameters are adapted by the training process for the neural network.

Recurrent neural networks (RNNs) are a family of feedforward neural networks that include feedback connections between layers. RNNs enable modeling of sequential data by sharing parameter data across different parts of the neural network. The architecture for an RNN includes cycles. The cycles represent the influence of a present value of a variable on its own value at a future time, as at least a portion of the output data from the RNN is used as feedback for processing subsequent input in a sequence. This feature makes RNNs particularly useful for language processing due to the variable nature in which language data can be composed.

The figures described below present exemplary feedforward, CNN, and RNN networks, as well as describe a general process for respectively training and deploying each of those types of networks. It will be understood that these descriptions are exemplary and non-limiting as to any specific embodiment described herein and the concepts illustrated can be applied generally to deep neural networks and machine learning techniques in general.

The exemplary neural networks described above can be used to perform deep learning. Deep learning is machine learning using deep neural networks. The deep neural networks used in deep learning are artificial neural networks composed of multiple hidden layers, as opposed to shallow neural networks that include only a single hidden layer. Deeper neural networks are generally more computationally intensive to train. However, the additional hidden layers of the network enable multistep pattern recognition that results in reduced output error relative to shallow machine learning techniques.

Deep neural networks used in deep learning typically include a front-end network to perform feature recognition coupled to a back-end network which represents a mathematical model that can perform operations (e.g., object classification, speech recognition, etc.) based on the feature representation provided to the model. Deep learning enables machine learning to be performed without requiring hand crafted feature engineering to be performed for the model. Instead, deep neural networks can learn features based on statistical structure or correlation within the input data. The learned features can be provided to a mathematical model that can map detected features to an output. The mathematical model used by the network is generally specialized for the specific task to be performed, and different models will be used to perform different task.

Once the neural network is structured, a learning model can be applied to the network to train the network to perform specific tasks. The learning model describes how to adjust the weights within the model to reduce the output error of the network. Backpropagation of errors is a common method used to train neural networks. An input vector is presented to the network for processing. The output of the network is compared to the desired output using a loss function and an error value is calculated for each of the neurons in the output layer. The error values are then propagated backwards until each neuron has an associated error value which roughly represents its contribution to the original output. The network can then learn from those errors using an algorithm, such as the stochastic gradient descent algorithm, to update the weights of the of the neural network.

FIG. **9**A-**9**B illustrate an exemplary convolutional neural network. FIG. **9**A illustrates various layers within a CNN. As shown in FIG. **9**A, an exemplary CNN used to model image processing can receive input **902** describing the red, green, and blue (RGB) components of an input image. The input **902** can be processed by multiple convolutional layers (e.g., convolutional layer **904**, convolutional layer **906**). The output from the multiple convolutional layers may optionally be processed by a set of fully connected layers **908**. Neurons in a fully connected layer have full connections to all activations in the previous layer, as previously described for a feedforward network. The output from the fully connected layers **908** can be used to generate an output result from the network. The activations within the fully connected layers **908** can be computed using matrix multiplication instead of convolution. Not all CNN implementations are configured to make use of fully connected layers **908**. For example, in some implementations the convolutional layer **906** can generate output for the CNN.

The convolutional layers are sparsely connected, which differs from traditional neural network configuration found in the fully connected layers **908**. Traditional neural network layers are fully connected, such that every output unit interacts with every input unit. However, the convolutional layers are sparsely connected because the output of the convolution of a field is input (instead of the respective state value of each of the nodes in the field) to the nodes of the subsequent layer, as illustrated. The kernels associated with the convolutional layers perform convolution operations, the output of which is sent to the next layer. The dimensionality reduction performed within the convolutional layers is one aspect that enables the CNN to scale to process large images.

FIG. **9**B illustrates exemplary computation stages within a convolutional layer of a CNN. Input to a convolutional layer **912** of a CNN can be processed in three stages of a convolutional layer **914**. The three stages can include a convolution stage **916**, a detector stage **918**, and a pooling stage **920**. The convolutional layer **914** can then output data to a successive convolutional layer. The final convolutional layer of the network can generate output feature map data or provide input to a fully connected layer, for example, to generate a classification value for the input to the CNN.

In the convolution stage **916** performs several convolutions in parallel to produce a set of linear activations. The convolution stage **916** can include an affine transformation, which is any transformation that can be specified as a linear transformation plus a translation. Affine transformations include rotations, translations, scaling, and combinations of these transformations. The convolution stage computes the output of functions (e.g., neurons) that are connected to specific regions in the input, which can be determined as the local region associated with the neuron. The neurons compute a dot product between the weights of the neurons and the region in the local input to which the neurons are connected. The output from the convolution stage **916** defines a set of linear activations that are processed by successive stages of the convolutional layer **914**.

The linear activations can be processed by a detector stage **918**. In the detector stage **918**, each linear activation is processed by a non-linear activation function. The non-linear activation function increases the nonlinear properties of the overall network without affecting the receptive fields of the convolution layer. Several types of non-linear activa-

tion functions may be used. One particular type is the rectified linear unit (ReLU), which uses an activation function defined as $f(x)=\max(0, x)$, such that the activation is thresholded at zero.

The pooling stage **920** uses a pooling function that replaces the output of the convolutional layer **906** with a summary statistic of the nearby outputs. The pooling function can be used to introduce translation invariance into the neural network, such that small translations to the input do not change the pooled outputs. Invariance to local translation can be useful in scenarios where the presence of a feature in the input data is more important than the precise location of the feature. Various types of pooling functions can be used during the pooling stage **920**, including max pooling, average pooling, and l2-norm pooling. Additionally, some CNN implementations do not include a pooling stage. Instead, such implementations substitute and additional convolution stage having an increased stride relative to previous convolution stages.

The output from the convolutional layer **914** can then be processed by the next layer **922**. The next layer **922** can be an additional convolutional layer or one of the fully connected layers **908**. For example, the first convolutional layer **904** of FIG. **9**A can output to the second convolutional layer **906**, while the second convolutional layer can output to a first layer of the fully connected layers **908**.

FIG. **10** illustrates an exemplary recurrent neural network **1000**. In a recurrent neural network (RNN), the previous state of the network influences the output of the current state of the network. RNNs can be built in a variety of ways using a variety of functions. The use of RNNs generally revolves around using mathematical models to predict the future based on a prior sequence of inputs. For example, an RNN may be used to perform statistical language modeling to predict an upcoming word given a previous sequence of words. The illustrated RNN **1000** can be described has having an input layer **1002** that receives an input vector, hidden layers **1004** to implement a recurrent function, a feedback mechanism **1005** to enable a 'memory' of previous states, and an output layer **1006** to output a result. The RNN **1000** operates based on time-steps. The state of the RNN at a given time step is influenced based on the previous time step via the feedback mechanism **1005**. For a given time step, the state of the hidden layers **1004** is defined by the previous state and the input at the current time step. An initial input ($x_1$) at a first time step can be processed by the hidden layer **1004**. A second input ($x_2$) can be processed by the hidden layer **1004** using state information that is determined during the processing of the initial input ($x_1$). A given state can be computed as $s_t=f(Ux_t++Ws_{t-1})$, where U and W are parameter matrices. The function f is generally a nonlinearity, such as the hyperbolic tangent function (Tanh) or a variant of the rectifier function $f(x)=\max(0, x)$. However, the specific mathematical function used in the hidden layers **1004** can vary depending on the specific implementation details of the RNN **1000**.

In addition to the basic CNN and RNN networks described, variations on those networks may be enabled. One example RNN variant is the long short term memory (LSTM) RNN. LSTM RNNs are capable of learning long-term dependencies that may be necessary for processing longer sequences of language. A variant on the CNN is a convolutional deep belief network, which has a structure similar to a CNN and is trained in a manner similar to a deep belief network. A deep belief network (DBN) is a generative neural network that is composed of multiple layers of stochastic (random) variables. DBNs can be trained layer-by-layer using greedy unsupervised learning. The learned weights of the DBN can then be used to provide pre-train neural networks by determining an optimal initial set of weights for the neural network.

FIG. **11** illustrates training and deployment of a deep neural network. Once a given network has been structured for a task the neural network is trained using a training dataset **1102**. Various training frameworks **1104** have been developed to enable hardware acceleration of the training process. For example, the machine learning framework **604** of FIG. **6** may be configured as a training framework **1104**. The training framework **1104** can hook into an untrained neural network **1106** and enable the untrained neural net to be trained using the parallel processing resources described herein to generate a trained neural network **1108**.

To start the training process the initial weights may be chosen randomly or by pre-training using a deep belief network. The training cycle then be performed in either a supervised or unsupervised manner.

Supervised learning is a learning method in which training is performed as a mediated operation, such as when the training dataset **1102** includes input paired with the desired output for the input, or where the training dataset includes input having known output and the output of the neural network is manually graded. The network processes the inputs and compares the resulting outputs against a set of expected or desired outputs. Errors are then propagated back through the system. The training framework **1104** can adjust to adjust the weights that control the untrained neural network **1106**. The training framework **1104** can provide tools to monitor how well the untrained neural network **1106** is converging towards a model suitable to generating correct answers based on known input data. The training process occurs repeatedly as the weights of the network are adjusted to refine the output generated by the neural network. The training process can continue until the neural network reaches a statistically desired accuracy associated with a trained neural network **1108**. The trained neural network **1108** can then be deployed to implement any number of machine learning operations to generate an inference result **1114** based on input of new data **1112**.

Unsupervised learning is a learning method in which the network attempts to train itself using unlabeled data. Thus, for unsupervised learning the training dataset **1102** will include input data without any associated output data. The untrained neural network **1106** can learn groupings within the unlabeled input and can determine how individual inputs are related to the overall dataset. Unsupervised training can be used to generate a self-organizing map, which is a type of trained neural network **1108** capable of performing operations useful in reducing the dimensionality of data. Unsupervised training can also be used to perform anomaly detection, which allows the identification of data points in an input dataset that deviate from the normal patterns of the data.

Variations on supervised and unsupervised training may also be employed. Semi-supervised learning is a technique in which in the training dataset **1102** includes a mix of labeled and unlabeled data of the same distribution. Incremental learning is a variant of supervised learning in which input data is continuously used to further train the model. Incremental learning enables the trained neural network **1108** to adapt to the new data **1112** without forgetting the knowledge instilled within the network during initial training.

Whether supervised or unsupervised, the training process for particularly deep neural networks may be too computa-

tionally intensive for a single compute node. Instead of using a single compute node, a distributed network of computational nodes can be used to accelerate the training process.

FIG. **12** is a block diagram illustrating distributed learning. Distributed learning is a training model that uses multiple distributed computing nodes to perform supervised or unsupervised training of a neural network. The distributed computational nodes can each include one or more host processors and one or more of the general-purpose processing nodes, such as the highly-parallel general-purpose graphics processing unit **700** as in FIG. **7**. As illustrated, distributed learning can be performed model parallelism **1202**, data parallelism **1204**, or a combination of model and data parallelism **1204**.

In model parallelism **1202**, different computational nodes in a distributed system can perform training computations for different parts of a single network. For example, each layer of a neural network can be trained by a different processing node of the distributed system. The benefits of model parallelism include the ability to scale to particularly large models. Splitting the computations associated with different layers of the neural network enables the training of very large neural networks in which the weights of all layers would not fit into the memory of a single computational node. In some instances, model parallelism can be particularly useful in performing unsupervised training of large neural networks.

In data parallelism **1204**, the different nodes of the distributed network have a complete instance of the model and each node receives a different portion of the data. The results from the different nodes are then combined. While different approaches to data parallelism are possible, data parallel training approaches all require a technique of combining results and synchronizing the model parameters between each node. Exemplary approaches to combining data include parameter averaging and update based data parallelism. Parameter averaging trains each node on a subset of the training data and sets the global parameters (e.g., weights, biases) to the average of the parameters from each node. Parameter averaging uses a central parameter server that maintains the parameter data. Update based data parallelism is similar to parameter averaging except that instead of transferring parameters from the nodes to the parameter server, the updates to the model are transferred. Additionally, update based data parallelism can be performed in a decentralized manner, where the updates are compressed and transferred between nodes.

Combined model and data parallelism **1206** can be implemented, for example, in a distributed system in which each computational node includes multiple GPUs. Each node can have a complete instance of the model with separate GPUs within each node are used to train different portions of the model.

Distributed training has increased overhead relative to training on a single machine. However, the parallel processors and GPGPUs described herein can each implement various techniques to reduce the overhead of distributed training, including techniques to enable high bandwidth GPU-to-GPU data transfer and accelerated remote data synchronization.

Exemplary Machine Learning Applications

Machine learning can be applied to solve a variety of technological problems, including but not limited to computer vision, autonomous driving and navigation, speech recognition, and language processing. Computer vision has traditionally been one of the most active research areas for machine learning applications. Applications of computer vision range from reproducing human visual abilities, such as recognizing faces, to creating new categories of visual abilities. For example, computer vision applications can be configured to recognize sound waves from the vibrations induced in objects visible in a video. Parallel processor accelerated machine learning enables computer vision applications to be trained using significantly larger training dataset than previously feasible and enables inferencing systems to be deployed using low power parallel processors.

Parallel processor accelerated machine learning has autonomous driving applications including lane and road sign recognition, obstacle avoidance, navigation, and driving control. Accelerated machine learning techniques can be used to train driving models based on datasets that define the appropriate responses to specific training input. The parallel processors described herein can enable rapid training of the increasingly complex neural networks used for autonomous driving solutions and enables the deployment of low power inferencing processors in a mobile platform suitable for integration into autonomous vehicles.

Parallel processor accelerated deep neural networks have enabled machine learning approaches to automatic speech recognition (ASR). ASR includes the creation of a function that computes the most probable linguistic sequence given an input acoustic sequence. Accelerated machine learning using deep neural networks have enabled the replacement of the hidden Markov models (HMMs) and Gaussian mixture models (GMMs) previously used for ASR.

Parallel processor accelerated machine learning can also be used to accelerate natural language processing. Automatic learning procedures can make use of statistical inference algorithms to produce models that are robust to erroneous or unfamiliar input. Exemplary natural language processor applications include automatic machine translation between human languages.

The parallel processing platforms used for machine learning can be divided into training platforms and deployment platforms. Training platforms are generally highly parallel and include optimizations to accelerate multi-GPU single node training and multi-node, multi-GPU training. Exemplary parallel processors suited for training include the highly-parallel general-purpose graphics processing unit **700** of FIG. **7** and the multi-GPU computing system **800** of FIG. **8**. On the contrary, deployed machine learning platforms generally include lower power parallel processors suitable for use in products such as cameras, autonomous robots, and autonomous vehicles.

FIG. **13** illustrates an exemplary inferencing system on a chip (SOC) **1300** suitable for performing inferencing using a trained model. The SOC **1300** can integrate processing components including a media processor **1302**, a vision processor **1304**, a GPGPU **1306** and a multi-core processor **1308**. The SOC **1300** can additionally include on-chip memory **1305** that can enable a shared on-chip data pool that is accessible by each of the processing components. The processing components can be optimized for low power operation to enable deployment to a variety of machine learning platforms, including autonomous vehicles and autonomous robots. For example, one implementation of the SOC **1300** can be used as a portion of the main control system for an autonomous vehicle. Where the SOC **1300** is configured for use in autonomous vehicles the SOC is designed and configured for compliance with the relevant functional safety standards of the deployment jurisdiction.

During operation, the media processor **1302** and vision processor **1304** can work in concert to accelerate computer vision operations. The media processor **1302** can enable low

latency decode of multiple high-resolution (e.g., 4K, 8K) video streams. The decoded video streams can be written to a buffer in the on-chip memory **1305**. The vision processor **1304** can then parse the decoded video and perform preliminary processing operations on the frames of the decoded video in preparation of processing the frames using a trained image recognition model. For example, the vision processor **1304** can accelerate convolution operations for a CNN that is used to perform image recognition on the high-resolution video data, while back end model computations are performed by the GPGPU **1306**.

The multi-core processor **1308** can include control logic to assist with sequencing and synchronization of data transfers and shared memory operations performed by the media processor **1302** and the vision processor **1304**. The multi-core processor **1308** can also function as an application processor to execute software applications that can make use of the inferencing compute capability of the GPGPU **1306**. For example, at least a portion of the navigation and driving logic can be implemented in software executing on the multi-core processor **1308**. Such software can directly issue computational workloads to the GPGPU **1306** or the computational workloads can be issued to the multi-core processor **1308**, which can offload at least a portion of those operations to the GPGPU **1306**.

The GPGPU **1306** can include compute clusters such as a low power configuration of the compute clusters **706**A-**706**H within the highly-parallel general-purpose graphics processing unit **700**. The compute clusters within the GPGPU **1306** can support instruction that are specifically optimized to perform inferencing computations on a trained neural network. For example, the GPGPU **1306** can support instructions to perform low precision computations such as 8-bit and 4-bit integer vector operations.

Mixed Inference Using Both Low and High Precision

GPGPU accelerated computing enables offload of parallel portions of an application to the GPGPU while the remainder of the program code executes on a host processor (e.g., CPU). As described herein, GPGPUs include compute units with the capability of preforming integer and floating-point operations. Generally those operations are exclusive in that a compute unit that is assigned to perform an integer operation will power gate or otherwise disable computational elements responsible for floating-point operations. The opposite may also be true in that the compute unit can disable integer components when performing floating-point operations. Such configuration enables a reduction in the operational power consumption of each compute unit. Alternatively, some compute units can be configured to perform computations selectively at one of multiple precisions. For example, a compute unit can be configured to perform an FP32 operation or a dual FP16 operation. A compute unit configured to perform 32-bit integer operations can perform four simultaneous 8-bit integer operations. Selective multi-precision or multi-data type compute units can enable compute units having robust capabilities that are also power efficient, in that a variety of operations can be performed by a single compute unit while idle logic units within each compute unit are disabled to reduce operational power consumption.

However, it is also possible to enable power efficient compute unit operation by utilizing the otherwise idle logic units within a compute unit. In embodiments described herein, variable precision and/or variable data type logic components within a compute unit can be operated simultaneously, such that compute units that are not used to service an initial operation can be enabled to process one or more additional operations. For example and in one embodiment, a compute unit having floating-point and integer logic units can process integer and floating-point operations simultaneously. In one embodiment a compute unit configured to selectively perform a 32-bit operation or a dual 16-bit operation can be configured to perform a 32-bit operation and a dual 16-bit operation or a 32-bit operation and multiple, independent 16-bit operations. In one embodiment such operations are enabled by allowing multiple instructions of different types to be issued to a single compute unit. In one embodiment, mixed data type instructions are enabled to allow single instruction multiple thread operations to accept mixed data-type and/or mixed precision operands.

FIG. **14** is a block diagram of a multiprocessor unit **1400**, according to an embodiment. The multiprocessor unit **1400** can be a variant of a graphics multiprocessor **234** of FIG. **2**D. The multiprocessor unit **1400** includes a fetch and decode unit **1402**, a branch unit **1404**, a register file **1406**, a thread manager **1407**, a single instruction multiple thread unit (SIMT unit **1410**), and a voltage and frequency manager **1420**. The fetch and decode unit **1402** can fetch an instruction for execution by the multiprocessor unit **1400**. The branch unit **1404** can compute instruction pointer adjustments based on an executed jump instruction. The register file **1406** can store general-purpose and architectural registers used by the SIMT unit **1410**. The thread manager **1407** can distribute and re-distribute threads among the compute units of the SIMT unit **1410**. In one embodiment, the SIMT unit **1410** is configured to execute a single instruction as multiple threads, with each thread of the instruction executed by a separate compute unit. In one embodiment compute unit **1411** through compute unit **1418** each includes an integer ALU (e.g., ALU **1411**A-**1418**A) and a floating-point unit (e.g., FPU **1411**B-**1418**B). The voltage and frequency of each compute unit **1411**-**1418** within the SIMT unit **1410** can be dynamically managed by the voltage and frequency manager **1420**, which can increase or decrease the voltage and clock frequency supplied to the various compute units as components of the compute units are enabled and disabled.

In some previously enabled configurations, each compute unit can execute a single thread of either an integer instruction or a floating-point instruction. If any of ALU **1411**A-**1418**A is tasked to execute a thread of an integer instruction, respective FPU **1411**B-FPU **1418**B is unavailable for use to execute a thread of a floating-point instruction and may be power gated during the operation of the corresponding ALU **1411**A-ALU **1418**A. For example, while ALU **1411**A may execute a thread of an integer instruction while FPU **1413**B executes a thread of a floating-point instruction, FPU **1411**B is power gated while ALU **1411**A is active. Embodiments described herein overcome such limitations by enabling, for example, ALU **1411**A to execute thread of an instruction while FPU **1411**B executes a thread of a different instruction. Furthermore, one embodiment provides support for mixed precision or mixed data-type operands, such that a single instruction can simultaneously perform operations for an instruction having floating-point and integer operands and/or operands having different precisions.

Embodiments described herein enable increased operational throughput for a cluster of compute units by making all logic units within each compute unit available to perform computations. In such embodiments, logic units within a compute unit that are designed to perform computations selectively at one of multiple precisions or multiple data types can be configured to perform multiple simultaneous

operations for each precision or data type supported by the compute unit. For a given compute unit **1411**-**1418**, ALUs **1411**A-**1418**A can perform integer operations while FPU **1411**B-**1418**B perform floating-point operations. These operations can be performed for a single instruction or for multiple instructions. In one embodiment a new class of mixed-precision instruction is enabled in which one or more operands are of one data type or precision while one or more different operands are of a different data type or precision. For example, an instruction can accept two or more multiple-element operands that include floating-point and integer data types and a single instruction is performed on a per-data type or per-precision basis.

FIG. **15** illustrates a mixed-precision processing system **1500**, according to an embodiment. The mixed-precision processing system **1500** includes a compute unit **1509** including an FPU **1508**A and an ALU **1508**B. In embodiments described herein the compute unit **1509** can execute a mixed precision/mixed data-type instruction. For example and in one embodiment an instruction can be processed to perform a single operation or multiple fused operations on multiple operands including multiple data elements. In one embodiment the data elements within an operand can be mixed precision or mixed data-type elements. For example, a first input register **1501** can store a first operand including a floating-point element **1502**A and multiple integer elements, including integer element **1504**A and integer element **1506**A. A second input register **1503** can store a second operand including a floating-point element **1502**B and multiple integer elements, including integer element **1504**B and integer element **1506**B. A third input register **1505** can store a third operand including a floating-point element **1502**C and multiple integer elements, including integer element **1504**C and integer element **1506**C.

The elements can be used as input to perform a single operation defined by a single opcode (e.g., opcode **1510**). For example, the opcode **1510** can specify a multi-element fused multiply-add operation in which the FPU **1508**A multiplies and adds floating-point elements. The FPU **1508**A can multiply floating-point elements (FP element **1502**A and FP element **1502**B) and add the product of the multiplication to a third floating-point element (FP element **1502**C). In parallel, the ALU **1508**B can perform a dual integer fused multiply-add operation in which a first set of integer elements (INT element **1504**A and INT element **1504**B) and a second set of integer elements (INT element **1506**A and INT element **1506**B) are multiplied and the product of each multiplication is added to a third integer element (INT element **1504**C and INT element **1506**C). Separate status flags **1512**A-**1512**C can be set based on each operation. The separate status flags **1512**A-**1512**C can be set to indicate status output known in the art, including but not limited to carry, negative, zero, and overflow. In one embodiment the separate status flags **1512**A-**1512**C can be output as a status vector, with each element of the status vector associated with each operation. Multiple results (e.g., result **1522**A, result **1522**B, result **1522**C) can be generated, with a first result **1522**A being a floating-point result and the second and third results **1522**B-**1522**C being integer results.

In one embodiment the precision of the elements can also be mixed. For example and in one embodiment input register **1501**, input register **1503**, and input register **1505** are 32-bit registers. FP element **1502**A-**1502**C can be 16-bit floating-point elements (e.g., FP16), while INT elements **1504**A-**1504**C and INT element **1506**A-**1506**C can each be 8-bit integer elements (e.g., INT8). The output register **1520** may also be a 32-bit register, where result **1522**A is 16-bits, while result **1522**B and result **1522**C are each 8-bits. In various embodiments, different register sizes can be used, including 64-bit, 128-bit, 256-bit, and 512-bit registers, allowing input elements ranging between 8-bits and 64-bits.

Operational logic **1700** for the mixed-precision processing system **1500** is shown in FIG. **17**, and can be implemented via the multiprocessor unit **1400** of FIG. **14**. With additional reference to FIG. **14**, the fetch and decode unit **1402** of the multiprocessor unit **1400** can fetch and decode a single instruction including multiple operands, the multiple operands referencing multiple data elements having different precisions, as shown at block **1702** of FIG. **17**. The thread manager **1407** can dispatch multiple threads of the single instruction for execution within a compute unit (e.g., compute unit **1509** as in FIG. **15**) of a GPGPU, as shown at block **1704**. In parallel, the compute unit can perform an instruction operation on a first set of operands having a first precision via a first logic unit within a compute unit, as shown at block **1706**, and perform the instruction operation on a second set of operands having a second precision via a second logic unit within the compute unit, as shown at block **1708**. The logic units can output multiple results of the operation, as shown at block **1710**. The differing precision operands can be of the same data type (e.g., floating-point, fixed-point, integer) having different precisions (e.g., 8-bit, 16-bit, 32-bit, etc.), same precision operands of different data types (e.g., FP-16 and INT-16), or different data types of different precision (e.g., dual INT-8 and FP16). Such instructions can be particularly useful when performing processing operations for mixed precision or mixed data-type neural networks in which input data for a layer is of a differing precision or data type than the weights applied to the input data.

While an integer ALU **150**B is described in FIG. **15** throughout, embodiments are not limited specifically to the use of integer only arithmetic logic units. In one embodiment the integer ALUs described herein may be floating-point units configured to perform integer operations.

FIG. **16** illustrates an additional mixed-precision processing system **1600**, according to an embodiment. The illustrated mixed-precision processing system **1600** is configured to enable parallel execution of floating-point workloads (e.g., FP workload **1602**) and integer workloads (e.g., INT workload **1604**) across multiple compute units (e.g., compute unit **1607** and compute unit **1609**). Compute unit **1607** and compute unit **1609** each include a set of floating-point units (FPUs **1606**A, FPUs **1608**A) and a set of integer arithmetic logic units (e.g., ALUs **1606**B, ALUs **1608**B). In previous implementations, for example a thread of FP workload **1602** would execute, for example, on FPUs **1606**A of compute unit **1607**, while a thread of INT workload **1604** would execute in parallel on ALUs **1608**B of compute unit **1609**. Embodiments described herein enable the FPUs and ALUs of the compute units to operate in parallel.

Operational logic **1800** for the mixed-precision processing system **1600** is shown in FIG. **18**. In one embodiment the operational logic **1800** can fetch and decode an integer instruction to be executed across multiple threads within a GPGPU, as shown at block **1802**. The logic **1800** can also fetch and decode a floating-point instruction to be executed across multiple threads within the GPGPU, as shown at block **1804**. The operational logic **1800** can enable parallel execution of the integer and floating-point instructions and execute the integer instruction via integer units of a first compute unit and a second compute unit at block **1805** while executing the floating instruction via floating-point units of the first compute unit and second compute unit, as shown at

block **1806**. The logic **1800** can cause the integer instruction to output integer results at block **1807**, while causing the floating-point operation to output floating-point results at block **1808**.

Specialized Inference System for a Neural Network

Performing inference operations can be made more efficient in deployed machine learning systems if the parallel processor or GPGPU used to perform the inferencing operations is specialized for the type of computations performed during inferencing for a neural network. In one embodiment, specialized inferencing logic can perform computations using compressed and/or encoded weight data in memory. Adaptive encoding can be enabled based on a profile generated for a neural network. The profile can be generated based on a determination of a set of commonly used weight values or common patterns that appear within the weight values. Byte-reduced encodings for common weight values can be stored in memory, reducing power requirements, or allowing larger networks to be stored in memory.

FIG. **19** illustrates a machine learning system **1900**, according to an embodiment. In one embodiment the machine learning system **1900** is a specialized inference system for a neural network that supports encoding of weight data for a neural network. The encoded weight data can enable a reduced size representation of the weight information for a deployed neural network. The reduced size of the weight data can enable power efficient inferencing or can enable processing of larger neural networks for a given memory size.

In one embodiment the machine learning system **1900** begins with an untrained neural network **1902** that can be processed by a training system **1903**. The training system **1903** can generate a trained neural network **1904**. A weight data profiling system **1905** can be used to profile the weight data of the trained neural network **1904**. The weight data profiling system **1905** can generate frequency-encoded weight data **1906** as well as an encoding profile **1910** that was used to generate the frequency-encoded weight data **1906**. The frequency-encoded weight data **1906** and encoding profile **1910** can be stored in GPGPU memory **1908**.

The frequency-encoded weight data **1906** and encoding profile **1910** stored in the GPGPU memory **1908** can be used to perform neural network layer computations on a GPGPU compute unit **1914**. In one embodiment the encoding profile **1910** can be read from the GPGPU memory **1908** and used to configure a GPGPU weight decoder **1912**. The GPGPU weight decoder **1912** can decode the frequency-encoded weight data **1906** to provide decoded weight data **1913** to the GPGPU compute unit **1914**. The input data **1911** for a neural network layer can also be read from GPGPU memory **1908**. The input data **1911** and decoded weight data **1913** can be processed by the GPGPU compute unit **1914** to generate GPGPU compute results **1915**.

In one embodiment the GPGPU compute unit **1914** can be configured to include the GPGPU weight decoder **1912**, such that the encoding profile **1910** and frequency-encoded weight data **1906** can be provided directly to the compute unit **1914** along with input data **1911** to generate GPGPU compute results **1915**.

Logic operations **2000** of the machine learning system **1900** are illustrated by the flow diagram of FIG. **20**. In one embodiment the logic operations **2000** can configure a GPGPU to profile weights of a trained neural network to general a weight profile for the weight data of the neural network, as shown at block **2002**. The logic operations **2000** can then cause the GPGPU to encode the weights of the neural network using the weight profile, as shown at block **2004**. The logic operations **2000** can then cause the GPGPU to store encoded weights and the weight profile to GPGPU memory, as shown at block **2006**. While performing computations for a neural network, the logic operations **2000** can cause the GPGPU to read encoded weights from the GPGPU memory during neural network processing, as shown at block **2008**. In one embodiment the logic operations **2000** can cause the GPGPU to decode the encoded weights based on a weight profile, as shown at block **2010**, before the GPGPU performs computations for the neural network at block **2012**. The encoded weights can be decoded using a GPGPU weight decoder **1912** as in FIG. **19**. The decode process can be skipped when the GPGPU is configured to directly accept encoded weights from GPGPU memory. In such embodiment, the logic operations **2000** can be configured such that the GPGPU will perform computations for the neural network at block **2012** without pre-decoding the weight data.

Additional Exemplary Graphics Processing System

Details of the embodiments described above can be incorporated within graphics processing systems and devices described below. The graphics processing system and devices of FIG. **21-34** illustrate alternative systems and graphics processing hardware that can implement any and all of the techniques described above.

Additional Exemplary Graphics Processing System Overview

FIG. **21** is a block diagram of a processing system **2100**, according to an embodiment. In various embodiments the system **2100** includes one or more processors **2102** and one or more graphics processors **2108**, and may be a single processor desktop system, a multiprocessor workstation system, or a server system having a large number of processors **2102** or processor cores **2107**. In one embodiment, the system **2100** is a processing platform incorporated within a system-on-a-chip (SoC) integrated circuit for use in mobile, handheld, or embedded devices.

An embodiment of system **2100** can include, or be incorporated within a server-based gaming platform, a game console, including a game and media console, a mobile gaming console, a handheld game console, or an online game console. In some embodiments system **2100** is a mobile phone, smart phone, tablet computing device or mobile Internet device. Data processing system **2100** can also include, couple with, or be integrated within a wearable device, such as a smart watch wearable device, smart eyewear device, augmented reality device, or virtual reality device. In some embodiments, data processing system **2100** is a television or set top box device having one or more processors **2102** and a graphical interface generated by one or more graphics processors **2108**.

In some embodiments, the one or more processors **2102** each include one or more processor cores **2107** to process instructions which, when executed, perform operations for system and user software. In some embodiments, each of the one or more processor cores **2107** is configured to process a specific instruction set **2109**. In some embodiments, instruction set **2109** may facilitate Complex Instruction Set Computing (CISC), Reduced Instruction Set Computing (RISC), or computing via a Very Long Instruction Word (VLIW). Multiple processor cores **2107** may each process a different instruction set **2109**, which may include instructions to facilitate the emulation of other instruction sets. Processor core **2107** may also include other processing devices, such a Digital Signal Processor (DSP).

In some embodiments, the processor **2102** includes cache memory **2104**. Depending on the architecture, the processor

2102 can have a single internal cache or multiple levels of internal cache. In some embodiments, the cache memory is shared among various components of the processor 2102. In some embodiments, the processor 2102 also uses an external cache (e.g., a Level-3 (L3) cache or Last Level Cache (LLC)) (not shown), which may be shared among processor cores 2107 using known cache coherency techniques. A register file 2106 is additionally included in processor 2102 which may include different types of registers for storing different types of data (e.g., integer registers, floating-point registers, status registers, and an instruction pointer register). Some registers may be general-purpose registers, while other registers may be specific to the design of the processor 2102.

In some embodiments, processor 2102 is coupled with a processor bus 2110 to transmit communication signals such as address, data, or control signals between processor 2102 and other components in system 2100. In one embodiment the system 2100 uses an exemplary 'hub' system architecture, including a memory controller hub 2116 and an Input Output (I/O) controller hub 2130. A memory controller hub 2116 facilitates communication between a memory device and other components of system 2100, while an I/O Controller Hub (ICH) 2130 provides connections to I/O devices via a local I/O bus. In one embodiment, the logic of the memory controller hub 2116 is integrated within the processor.

Memory device 2120 can be a dynamic random-access memory (DRAM) device, a static random-access memory (SRAM) device, flash memory device, phase-change memory device, or some other memory device having suitable performance to serve as process memory. In one embodiment the memory device 2120 can operate as system memory for the system 2100, to store data 2122 and instructions 2121 for use when the one or more processors 2102 executes an application or process. Memory controller hub 2116 also couples with an optional external graphics processor 2112, which may communicate with the one or more graphics processors 2108 in processors 2102 to perform graphics and media operations.

In some embodiments, ICH 2130 enables peripherals to connect to memory device 2120 and processor 2102 via a high-speed I/O bus. The I/O peripherals include, but are not limited to, an audio controller 2146, a firmware interface 2128, a wireless transceiver 2126 (e.g., Wi-Fi, Bluetooth), a data storage device 2124 (e.g., hard disk drive, flash memory, etc.), and a legacy I/O controller 2140 for coupling legacy (e.g., Personal System 2 (PS/2)) devices to the system. One or more Universal Serial Bus (USB) controllers 2142 connect input devices, such as keyboard and mouse 2144 combinations. A network controller 2134 may also couple with ICH 2130. In some embodiments, a high-performance network controller (not shown) couples with processor bus 2110. It will be appreciated that the system 2100 shown is exemplary and not limiting, as other types of data processing systems that are differently configured may also be used. For example, the I/O controller hub 2130 may be integrated within the one or more processor 2102, or the memory controller hub 2116 and I/O controller hub 2130 may be integrated into a discreet external graphics processor, such as the external graphics processor 2112.

FIG. 22 is a block diagram of an embodiment of a processor 2200 having one or more processor cores 2202A-2202N, an integrated memory controller 2214, and an integrated graphics processor 2208. Those elements of FIG. 22 having the same reference numbers (or names) as the elements of any other figure herein can operate or function in any manner similar to that described elsewhere herein, but are not limited to such. Processor 2200 can include additional cores up to and including additional core 2202N represented by the dashed lined boxes. Each of processor cores 2202A-2202N includes one or more internal cache units 2204A-2204N. In some embodiments each processor core also has access to one or more shared cached units 2206.

The internal cache units 2204A-2204N and shared cache units 2206 represent a cache memory hierarchy within the processor 2200. The cache memory hierarchy may include at least one level of instruction and data cache within each processor core and one or more levels of shared mid-level cache, such as a Level 2 (L2), Level 3 (L3), Level 4 (L4), or other levels of cache, where the highest level of cache before external memory is classified as the LLC. In some embodiments, cache coherency logic maintains coherency between the various cache units 2206 and 2204A-2204N.

In some embodiments, processor 2200 may also include a set of one or more bus controller units 2216 and a system agent core 2210. The one or more bus controller units 2216 manage a set of peripheral buses, such as one or more Peripheral Component Interconnect buses (e.g., PCI, PCI Express). System agent core 2210 provides management functionality for the various processor components. In some embodiments, system agent core 2210 includes one or more integrated memory controllers 2214 to manage access to various external memory devices (not shown).

In some embodiments, one or more of the processor cores 2202A-2202N include support for simultaneous multi-threading. In such embodiment, the system agent core 2210 includes components for coordinating and operating cores 2202A-2202N during multi-threaded processing. System agent core 2210 may additionally include a power control unit (PCU), which includes logic and components to regulate the power state of processor cores 2202A-2202N and graphics processor 2208.

In some embodiments, processor 2200 additionally includes graphics processor 2208 to execute graphics processing operations. In some embodiments, the graphics processor 2208 couples with the set of shared cache units 2206, and the system agent core 2210, including the one or more integrated memory controllers 2214. In some embodiments, a display controller 2211 is coupled with the graphics processor 2208 to drive graphics processor output to one or more coupled displays. In some embodiments, display controller 2211 may be a separate module coupled with the graphics processor via at least one interconnect, or may be integrated within the graphics processor 2208 or system agent core 2210.

In some embodiments, a ring-based interconnect 2212 is used to couple the internal components of the processor 2200. However, an alternative interconnect unit may be used, such as a point-to-point interconnect, a switched interconnect, or other techniques, including techniques well known in the art. In some embodiments, graphics processor 2208 couples with the ring-based interconnect 2212 via an I/O link 2213.

The exemplary I/O link 2213 represents at least one of multiple varieties of I/O interconnects, including an on package I/O interconnect which facilitates communication between various processor components and a high-performance embedded memory module 2218, such as an eDRAM module. In some embodiments, each of the processor cores 2202A-2202N and graphics processor 2208 use embedded memory modules 2218 as a shared Last Level Cache.

In some embodiments, processor cores **2202**A-**2202**N are homogenous cores executing the same instruction set architecture. In another embodiment, processor cores **2202**A-**2202**N are heterogeneous in terms of instruction set architecture (ISA), where one or more of processor cores **2202**A-**2202**N execute a first instruction set, while at least one of the other cores executes a subset of the first instruction set or a different instruction set. In one embodiment processor cores **2202**A-**2202**N are heterogeneous in terms of microarchitecture, where one or more cores having a relatively higher power consumption couple with one or more power cores having a lower power consumption. Additionally, processor **2200** can be implemented on one or more chips or as an SoC integrated circuit having the illustrated components, in addition to other components.

FIG. **23** is a block diagram of a graphics processor **2300**, which may be a discrete graphics processing unit, or may be a graphics processor integrated with a plurality of processing cores. In some embodiments, the graphics processor communicates via a memory mapped I/O interface to registers on the graphics processor and with commands placed into the processor memory. In some embodiments, graphics processor **2300** includes a memory interface **2314** to access memory. Memory interface **2314** can be an interface to local memory, one or more internal caches, one or more shared external caches, and/or to system memory.

In some embodiments, graphics processor **2300** also includes a display controller **2302** to drive display output data to a display device **2320**. Display controller **2302** includes hardware for one or more overlay planes for the display and composition of multiple layers of video or user interface elements. In some embodiments, graphics processor **2300** includes a video codec engine **2306** to encode, decode, or transcode media to, from, or between one or more media encoding formats, including, but not limited to Moving Picture Experts Group (MPEG) formats such as MPEG-2, Advanced Video Coding (AVC) formats such as H.264/MPEG-4 AVC, as well as the Society of Motion Picture & Television Engineers (SMPTE) 421M/VC-1, and Joint Photographic Experts Group (JPEG) formats such as JPEG, and Motion JPEG (MJPEG) formats.

In some embodiments, graphics processor **2300** includes a block image transfer (BLIT) engine **2304** to perform two-dimensional (2D) rasterizer operations including, for example, bit-boundary block transfers. However, in one embodiment, 2D graphics operations are performed using one or more components of graphics processing engine (GPE) **2310**. In some embodiments, GPE **2310** is a compute engine for performing graphics operations, including three-dimensional (3D) graphics operations and media operations.

In some embodiments, GPE **310** includes a 3D pipeline **2312** for performing 3D operations, such as rendering three-dimensional images and scenes using processing functions that act upon 3D primitive shapes (e.g., rectangle, triangle, etc.). The 3D pipeline **2312** includes programmable and fixed function elements that perform various tasks within the element and/or spawn execution threads to a 3D/Media sub-system **2315**. While 3D pipeline **2312** can be used to perform media operations, an embodiment of GPE **2310** also includes a media pipeline **2316** that is specifically used to perform media operations, such as video post-processing and image enhancement.

In some embodiments, media pipeline **2316** includes fixed function or programmable logic units to perform one or more specialized media operations, such as video decode acceleration, video de-interlacing, and video encode acceleration in place of, or on behalf of video codec engine **2306**. In some embodiments, media pipeline **2316** additionally includes a thread spawning unit to spawn threads for execution on 3D/Media sub-system **2315**. The spawned threads perform computations for the media operations on one or more graphics execution units included in 3D/Media sub-system **2315**.

In some embodiments, 3D/Media subsystem **2315** includes logic for executing threads spawned by 3D pipeline **2312** and media pipeline **2316**. In one embodiment, the pipelines send thread execution requests to 3D/Media sub-system **2315**, which includes thread dispatch logic for arbitrating and dispatching the various requests to available thread execution resources. The execution resources include an array of graphics execution units to process the 3D and media threads. In some embodiments, 3D/Media subsystem **2315** includes one or more internal caches for thread instructions and data. In some embodiments, the subsystem also includes shared memory, including registers and addressable memory, to share data between threads and to store output data.

Additional Exemplary Graphics Processing Engine

FIG. **24** is a block diagram of a graphics processing engine **2410** of a graphics processor in accordance with some embodiments. In one embodiment, the graphics processing engine (GPE) **2410** is a version of the GPE **2310** shown in FIG. **23**. Elements of FIG. **24** having the same reference numbers (or names) as the elements of any other figure herein can operate or function in any manner similar to that described elsewhere herein, but are not limited to such. For example, the 3D pipeline **2312** and media pipeline **2316** of FIG. **23** are illustrated. The media pipeline **2316** is optional in some embodiments of the GPE **2410** and may not be explicitly included within the GPE **2410**. For example and in at least one embodiment, a separate media and/or image processor is coupled to the GPE **2410**.

In some embodiments, GPE **2410** couples with or includes a command streamer **2403**, which provides a command stream to the 3D pipeline **2312** and/or media pipelines **2316**. In some embodiments, command streamer **2403** is coupled with memory, which can be system memory, or one or more of internal cache memory and shared cache memory. In some embodiments, command streamer **2403** receives commands from the memory and sends the commands to 3D pipeline **2312** and/or media pipeline **2316**. The commands are directives fetched from a ring buffer, which stores commands for the 3D pipeline **2312** and media pipeline **2316**. In one embodiment, the ring buffer can additionally include batch command buffers storing batches of multiple commands. The commands for the 3D pipeline **2312** can also include references to data stored in memory, such as but not limited to vertex and geometry data for the 3D pipeline **2312** and/or image data and memory objects for the media pipeline **2316**. The 3D pipeline **2312** and media pipeline **2316** process the commands and data by performing operations via logic within the respective pipelines or by dispatching one or more execution threads to a graphics core array **2414**.

In various embodiments the 3D pipeline **2312** can execute one or more shader programs, such as vertex shaders, geometry shaders, pixel shaders, fragment shaders, compute shaders, or other shader programs, by processing the instructions and dispatching execution threads to the graphics core array **2414**. The graphics core array **2414** provides a unified block of execution resources. Multi-purpose execution logic (e.g., execution units) within the graphics core array **2414**

includes support for various 3D API shader languages and can execute multiple simultaneous execution threads associated with multiple shaders.

In some embodiments the graphics core array **2414** also includes execution logic to perform media functions, such as video and/or image processing. In one embodiment, the execution units additionally include general-purpose logic that is programmable to perform parallel general-purpose computational operations, in addition to graphics processing operations. The general-purpose logic can perform processing operations in parallel or in conjunction with general-purpose logic within the processor core(s) **2107** of FIG. **21** or core **2202**A-**2202**N as in FIG. **22**.

Output data generated by threads executing on the graphics core array **2414** can output data to memory in a unified return buffer (URB) **2418**. The URB **2418** can store data for multiple threads. In some embodiments the URB **2418** may be used to send data between different threads executing on the graphics core array **2414**. In some embodiments the URB **2418** may additionally be used for synchronization between threads on the graphics core array and fixed function logic within the shared function logic **2420**.

In some embodiments, graphics core array **2414** is scalable, such that the array includes a variable number of graphics cores, each having a variable number of execution units based on the target power and performance level of GPE **2410**. In one embodiment the execution resources are dynamically scalable, such that execution resources may be enabled or disabled as needed.

The graphics core array **2414** couples with shared function logic **2420** that includes multiple resources that are shared between the graphics cores in the graphics core array. The shared functions within the shared function logic **2420** are hardware logic units that provide specialized supplemental functionality to the graphics core array **2414**. In various embodiments, shared function logic **2420** includes but is not limited to sampler **2421**, math **2422**, and inter-thread communication (ITC) **2423** logic. Additionally, some embodiments implement one or more cache(s) **2425** within the shared function logic **2420**. A shared function is implemented where the demand for a given specialized function is insufficient for inclusion within the graphics core array **2414**. Instead a single instantiation of that specialized function is implemented as a stand-alone entity in the shared function logic **2420** and shared among the execution resources within the graphics core array **2414**. The precise set of functions that are shared between the graphics core array **2414** and included within the graphics core array **2414** varies between embodiments.

FIG. **25** is a block diagram of another embodiment of a graphics processor **2500**.

Elements of FIG. **25** having the same reference numbers (or names) as the elements of any other figure herein can operate or function in any manner similar to that described elsewhere herein, but are not limited to such.

In some embodiments, graphics processor **2500** includes a ring interconnect **2502**, a pipeline front-end **2504**, a media engine **2537**, and graphics cores **2580**A-**2580**N. In some embodiments, ring interconnect **2502** couples the graphics processor to other processing units, including other graphics processors or one or more general-purpose processor cores. In some embodiments, the graphics processor is one of many processors integrated within a multi-core processing system.

In some embodiments, graphics processor **2500** receives batches of commands via ring interconnect **2502**. The incoming commands are interpreted by a command streamer **2503** in the pipeline front-end **2504**. In some embodiments, graphics processor **2500** includes scalable execution logic to perform 3D geometry processing and media processing via the graphics core(s) **2580**A-**2580**N. For 3D geometry processing commands, command streamer **2503** supplies commands to geometry pipeline **2536**. For at least some media processing commands, command streamer **2503** supplies the commands to a video front end **2534**, which couples with a media engine **2537**. In some embodiments, media engine **2537** includes a Video Quality Engine (VQE) **2530** for video and image post-processing and a multi-format encode/decode (MFX) **2533** engine to provide hardware-accelerated media data encode and decode. In some embodiments, geometry pipeline **2536** and media engine **2537** each generate execution threads for the thread execution resources provided by at least one graphics core **2580**A.

In some embodiments, graphics processor **2500** includes scalable thread execution resources featuring modular cores **2580**A-**2580**N (sometimes referred to as core slices), each having multiple sub-cores **2550**A-**550**N, **2560**A-**2560**N (sometimes referred to as core sub-slices). In some embodiments, graphics processor **2500** can have any number of graphics cores **2580**A through **2580**N. In some embodiments, graphics processor **2500** includes a graphics core **2580**A having at least a first sub-core **2550**A and a second sub-core **2560**A. In other embodiments, the graphics processor is a low power processor with a single sub-core (e.g., **2550**A). In some embodiments, graphics processor **2500** includes multiple graphics cores **2580**A-**2580**N, each including a set of first sub-cores **2550**A-**2550**N and a set of second sub-cores **2560**A-**2560**N. Each sub-core in the set of first sub-cores **2550**A-**2550**N includes at least a first set of execution units **2552**A-**2552**N and media/texture samplers **2554**A-**2554**N. Each sub-core in the set of second sub-cores **2560**A-**2560**N includes at least a second set of execution units **2562**A-**2562**N and samplers **2564**A-**2564**N. In some embodiments, each sub-core **2550**A-**2550**N, **2560**A-**2560**N shares a set of shared resources **2570**A-**2570**N. In some embodiments, the shared resources include shared cache memory and pixel operation logic. Other shared resources may also be included in the various embodiments of the graphics processor.

Additional Exemplary Execution Units

FIG. **26** illustrates thread execution logic **2600** including an array of processing elements employed in some embodiments of a GPE. Elements of FIG. **26** having the same reference numbers (or names) as the elements of any other figure herein can operate or function in any manner similar to that described elsewhere herein, but are not limited to such.

In some embodiments, thread execution logic **2600** includes a shader processor **2602**, a thread dispatcher **2604**, instruction cache **2606**, a scalable execution unit array including a plurality of execution units **2608**A-**2608**N, a sampler **2610**, a data cache **2612**, and a data port **2614**. In one embodiment the scalable execution unit array can dynamically scale by enabling or disabling one or more execution units (e.g., any of execution unit **2608**A, **2608**B, **2608**C, **2608**D, through **2608**N-**1** and **2608**N) based on the computational requirements of a workload. In one embodiment the included components are interconnected via an interconnect fabric that links to each of the components. In some embodiments, thread execution logic **2600** includes one or more connections to memory, such as system memory or cache memory, through one or more of instruction cache **2606**, data port **2614**, sampler **2610**, and execution units **2608**A-**2608**N. In some embodiments, each execution unit (e.g. **2608**A) is a stand-alone programmable general-pur-

pose computational unit that is capable of executing multiple simultaneous hardware threads while processing multiple data elements in parallel for each thread. In various embodiments, the array of execution units **2608**A-**2608**N is scalable to include any number individual execution units.

In some embodiments, the execution units **2608**A-**2608**N are primarily used to execute shader programs. A shader processor **2602** can process the various shader programs and dispatch execution threads associated with the shader programs via a thread dispatcher **2604**. In one embodiment the thread dispatcher includes logic to arbitrate thread initiation requests from the graphics and media pipelines and instantiate the requested threads on one or more execution unit in the execution units **2608**A-**2608**N. For example, the geometry pipeline (e.g., **2536** of FIG. **25**) can dispatch vertex, tessellation, or geometry shaders to the thread execution logic **2600** (FIG. **26**) for processing. In some embodiments, thread dispatcher **2604** can also process runtime thread spawning requests from the executing shader programs.

In some embodiments, the execution units **2608**A-**2608**N support an instruction set that includes native support for many standard 3D graphics shader instructions, such that shader programs from graphics libraries (e.g., Direct 3D and OpenGL) are executed with a minimal translation. The execution units support vertex and geometry processing (e.g., vertex programs, geometry programs, vertex shaders), pixel processing (e.g., pixel shaders, fragment shaders) and general-purpose processing (e.g., compute and media shaders). Each of the execution units **2608**A-**2608**N is capable of multi-issue single instruction multiple data (SIMD) execution and multi-threaded operation enables an efficient execution environment in the face of higher latency memory accesses. Each hardware thread within each execution unit has a dedicated high-bandwidth register file and associated independent thread-state. Execution is multi-issue per clock to pipelines capable of integer, single and double precision floating-point operations, SIMD branch capability, logical operations, transcendental operations, and other miscellaneous operations. While waiting for data from memory or one of the shared functions, dependency logic within the execution units **2608**A-**2608**N causes a waiting thread to sleep until the requested data has been returned. While the waiting thread is sleeping, hardware resources may be devoted to processing other threads. For example, during a delay associated with a vertex shader operation, an execution unit can perform operations for a pixel shader, fragment shader, or another type of shader program, including a different vertex shader.

Each execution unit in execution units **2608**A-**2608**N operates on arrays of data elements. The number of data elements is the "execution size," or the number of channels for the instruction. An execution channel is a logical unit of execution for data element access, masking, and flow control within instructions. The number of channels may be independent of the number of physical Arithmetic Logic Units (ALUs) or Floating-Point Units (FPUs) for a particular graphics processor. In some embodiments, execution units **2608**A-**2608**N support integer and floating-point data types.

The execution unit instruction set includes SIMD instructions. The various data elements can be stored as a packed data type in a register and the execution unit will process the various elements based on the data size of the elements. For example, when operating on a 256-bit wide vector, the 256 bits of the vector are stored in a register and the execution unit operates on the vector as four separate 64-bit packed data elements (Quad-Word (QW) size data elements), eight separate 32-bit packed data elements (Double Word (DW) size data elements), sixteen separate 16-bit packed data elements (Word (W) size data elements), or thirty-two separate 8-bit data elements (byte (B) size data elements). However, different vector widths and register sizes are possible.

One or more internal instruction caches (e.g., **2606**) are included in the thread execution logic **2600** to cache thread instructions for the execution units. In some embodiments, one or more data caches (e.g., **2612**) are included to cache thread data during thread execution. In some embodiments, a sampler **2610** is included to provide texture sampling for 3D operations and media sampling for media operations. In some embodiments, sampler **2610** includes specialized texture or media sampling functionality to process texture or media data during the sampling process before providing the sampled data to an execution unit.

During execution, the graphics and media pipelines send thread initiation requests to thread execution logic **2600** via thread spawning and dispatch logic. Once a group of geometric objects has been processed and rasterized into pixel data, pixel processor logic (e.g., pixel shader logic, fragment shader logic, etc.) within the shader processor **2602** is invoked to further compute output information and cause results to be written to output surfaces (e.g., color buffers, depth buffers, stencil buffers, etc.). In some embodiments, a pixel shader or fragment shader calculates the values of the various vertex attributes that are to be interpolated across the rasterized object. In some embodiments, pixel processor logic within the shader processor **2602** then executes an application programming interface (API)-supplied pixel or fragment shader program. To execute the shader program, the shader processor **2602** dispatches threads to an execution unit (e.g., **2608**A) via thread dispatcher **2604**. In some embodiments, shader processor **2602** uses texture sampling logic in the sampler **2610** to access texture data in texture maps stored in memory. Arithmetic operations on the texture data and the input geometry data compute pixel color data for each geometric fragment or discards one or more pixels from further processing.

In some embodiments, the data port **2614** provides a memory access mechanism for the thread execution logic **2600** output processed data to memory for processing on a graphics processor output pipeline. In some embodiments, the data port **2614** includes or couples to one or more cache memories (e.g., data cache **2612**) to cache data for memory access via the data port.

FIG. **27** is a block diagram illustrating graphics processor instruction formats **2700** according to some embodiments. In one or more embodiment, the graphics processor execution units support an instruction set having instructions in multiple formats. The solid lined boxes illustrate the components that are generally included in an execution unit instruction, while the dashed lines include components that are optional or that are only included in a sub-set of the instructions. In some embodiments, the graphics processor instruction formats **2700** described and illustrated are macro-instructions, in that they are instructions supplied to the execution unit, as opposed to micro-operations resulting from instruction decode once the instruction is processed.

In some embodiments, the graphics processor execution units natively support instructions in a 128-bit instruction format **2710**. A 64-bit compacted instruction format **2730** is available for some instructions based on the selected instruction, instruction options, and number of operands. The native 128-bit instruction format **2710** provides access to all instruction options, while some options and operations are restricted in the 64-bit format **2730**. The native instructions

available in the 64-bit format **2730** vary by embodiment. In some embodiments, the instruction is compacted in part using a set of index values in an index field **2713**. The execution unit hardware references a set of compaction tables based on the index values and uses the compaction table outputs to reconstruct a native instruction in the 128-bit instruction format **2710**.

For each format, instruction opcode **2712** defines the operation that the execution unit is to perform. The execution units execute each instruction in parallel across the multiple data elements of each operand. For example, in response to an add instruction the execution unit performs a simultaneous add operation across each color channel representing a texture element or picture element. By default, the execution unit performs each instruction across all data channels of the operands. In some embodiments, instruction control field **2714** enables control over certain execution options, such as channels selection (e.g., predication) and data channel order (e.g., swizzle). For instructions in the 128-bit instruction format **2710** an exec-size field **2716** limits the number of data channels that will be executed in parallel. In some embodiments, exec-size field **2716** is not available for use in the 64-bit compact instruction format **2730**.

Some execution unit instructions have up to three operands including two source operands, src0 **2720**, src1 **2722**, and one destination **2718**. In some embodiments, the execution units support dual destination instructions, where one of the destinations is implied. Data manipulation instructions can have a third source operand (e.g., SRC2 **2724**), where the instruction opcode **2712** determines the number of source operands. An instruction's last source operand can be an immediate (e.g., hard-coded) value passed with the instruction.

In some embodiments, the 128-bit instruction format **2710** includes an access/address mode field **2726** specifying, for example, whether direct register addressing mode or indirect register addressing mode is used. When direct register addressing mode is used, the register address of one or more operands is directly provided by bits in the instruction.

In some embodiments, the 128-bit instruction format **2710** includes an access/address mode field **2726**, which specifies an address mode and/or an access mode for the instruction. In one embodiment the access mode is used to define a data access alignment for the instruction. Some embodiments support access modes including a 16-byte aligned access mode and a 1-byte aligned access mode, where the byte alignment of the access mode determines the access alignment of the instruction operands. For example, when in a first mode, the instruction may use byte-aligned addressing for source and destination operands and when in a second mode, the instruction may use 16-byte-aligned addressing for all source and destination operands.

In one embodiment, the address mode portion of the access/address mode field **2726** determines whether the instruction is to use direct or indirect addressing. When direct register addressing mode is used bits in the instruction directly provide the register address of one or more operands. When indirect register addressing mode is used, the register address of one or more operands may be computed based on an address register value and an address immediate field in the instruction.

In some embodiments instructions are grouped based on opcode **2712** bit-fields to simplify Opcode decode **2740**. For an 8-bit opcode, bits 4, 5, and 6 allow the execution unit to determine the type of opcode. The precise opcode grouping shown is merely an example. In some embodiments, a move and logic opcode group **2742** includes data movement and logic instructions (e.g., move (mov), compare (cmp)). In some embodiments, move and logic group **2742** shares the five most significant bits (MSB), where move (mov) instructions are in the form of 0000xxxxb and logic instructions are in the form of 0001xxxxb. A flow control instruction group **2744** (e.g., call, jump (jmp)) includes instructions in the form of 0010xxxxb (e.g., 0x20). A miscellaneous instruction group **2746** includes a mix of instructions, including synchronization instructions (e.g., wait, send) in the form of 0011xxxxb (e.g., 0x30). A parallel math instruction group **2748** includes component-wise arithmetic instructions (e.g., add, multiply (mul)) in the form of 0100xxxxb (e.g., 0x40). The parallel math group **2748** performs the arithmetic operations in parallel across data channels. The vector math group **2750** includes arithmetic instructions (e.g., dp4) in the form of 0101xxxxb (e.g., 0x50). The vector math group performs arithmetic such as dot product calculations on vector operands.

Additional Exemplary Graphics Pipeline

FIG. **28** is a block diagram of another embodiment of a graphics processor **2800**. Elements of FIG. **28** having the same reference numbers (or names) as the elements of any other figure herein can operate or function in any manner similar to that described elsewhere herein, but are not limited to such.

In some embodiments, graphics processor **2800** includes a graphics pipeline **2820**, a media pipeline **2830**, a display engine **2840**, thread execution logic **2850**, and a render output pipeline **2870**. In some embodiments, graphics processor **2800** is a graphics processor within a multi-core processing system that includes one or more general-purpose processing cores. The graphics processor is controlled by register writes to one or more control registers (not shown) or via commands issued to graphics processor **2800** via a ring interconnect **2802**. In some embodiments, ring interconnect **2802** couples graphics processor **2800** to other processing components, such as other graphics processors or general-purpose processors. Commands from ring interconnect **2802** are interpreted by a command streamer **2803**, which supplies instructions to individual components of graphics pipeline **2820** or media pipeline **2830**.

In some embodiments, command streamer **2803** directs the operation of a vertex fetcher **2805** that reads vertex data from memory and executes vertex-processing commands provided by command streamer **2803**. In some embodiments, vertex fetcher **2805** provides vertex data to a vertex shader **2807**, which performs coordinate space transformation and lighting operations to each vertex. In some embodiments, vertex fetcher **2805** and vertex shader **2807** execute vertex-processing instructions by dispatching execution threads to execution units **2852**A-**2852**B via a thread dispatcher **2831**.

In some embodiments, execution units **2852**A-**2852**B are an array of vector processors having an instruction set for performing graphics and media operations. In some embodiments, execution units **2852**A-**2852**B have an attached L1 cache **2851** that is specific for each array or shared between the arrays. The cache can be configured as a data cache, an instruction cache, or a single cache that is partitioned to contain data and instructions in different partitions.

In some embodiments, graphics pipeline **2820** includes tessellation components to perform hardware-accelerated tessellation of 3D objects. In some embodiments, a programmable hull shader **811** configures the tessellation operations. A programmable domain shader **817** provides back-

end evaluation of tessellation output. A tessellator **2813** operates at the direction of hull shader **2811** and contains special purpose logic to generate a set of detailed geometric objects based on a coarse geometric model that is provided as input to graphics pipeline **2820**. In some embodiments, if tessellation is not used, tessellation components (e.g., hull shader **2811**, tessellator **2813**, and domain shader **2817**) can be bypassed.

In some embodiments, complete geometric objects can be processed by a geometry shader **2819** via one or more threads dispatched to execution units **2852**A-**2852**B, or can proceed directly to the clipper **2829**. In some embodiments, the geometry shader operates on entire geometric objects, rather than vertices or patches of vertices as in previous stages of the graphics pipeline. If the tessellation is disabled the geometry shader **2819** receives input from the vertex shader **2807**. In some embodiments, geometry shader **2819** is programmable by a geometry shader program to perform geometry tessellation if the tessellation units are disabled.

Before rasterization, a clipper **2829** processes vertex data. The clipper **2829** may be a fixed function clipper or a programmable clipper having clipping and geometry shader functions. In some embodiments, a rasterizer and depth test component **2873** in the render output pipeline **2870** dispatches pixel shaders to convert the geometric objects into their per pixel representations. In some embodiments, pixel shader logic is included in thread execution logic **2850**. In some embodiments, an application can bypass the rasterizer and depth test component **2873** and access un-rasterized vertex data via a stream out unit **2823**.

The graphics processor **2800** has an interconnect bus, interconnect fabric, or some other interconnect mechanism that allows data and message passing amongst the major components of the processor. In some embodiments, execution units **2852**A-**2852**B and associated cache(s) **2851**, texture and media sampler **2854**, and texture/sampler cache **2858** interconnect via a data port **2856** to perform memory access and communicate with render output pipeline components of the processor. In some embodiments, sampler **2854**, caches **2851**, **2858** and execution units **2852**A-**2852**B each have separate memory access paths.

In some embodiments, render output pipeline **2870** contains a rasterizer and depth test component **2873** that converts vertex-based objects into an associated pixel-based representation. In some embodiments, the rasterizer logic includes a windower/masker unit to perform fixed function triangle and line rasterization. An associated render cache **2878** and depth cache **2879** are also available in some embodiments. A pixel operations component **2877** performs pixel-based operations on the data, though in some instances, pixel operations associated with 2D operations (e.g. bit block image transfers with blending) are performed by the 2D engine **2841** or substituted at display time by the display controller **2843** using overlay display planes. In some embodiments, a shared L3 cache **2875** is available to all graphics components, allowing the sharing of data without the use of main system memory.

In some embodiments, graphics processor media pipeline **2830** includes a media engine **2837** and a video front-end **2834**. In some embodiments, video front-end **2834** receives pipeline commands from the command streamer **2803**. In some embodiments, media pipeline **2830** includes a separate command streamer. In some embodiments, video front-end **2834** processes media commands before sending the command to the media engine **2837**. In some embodiments, media engine **2837** includes thread spawning functionality to spawn threads for dispatch to thread execution logic **2850** via thread dispatcher **2831**.

In some embodiments, graphics processor **2800** includes a display engine **2840**. In some embodiments, display engine **2840** is external to processor **2800** and couples with the graphics processor via the ring interconnect **2802**, or some other interconnect bus or fabric. In some embodiments, display engine **2840** includes a 2D engine **2841** and a display controller **2843**. In some embodiments, display engine **2840** contains special purpose logic capable of operating independently of the 3D pipeline. In some embodiments, display controller **2843** couples with a display device (not shown), which may be a system integrated display device, as in a laptop computer, or an external display device attached via a display device connector.

In some embodiments, graphics pipeline **2820** and media pipeline **2830** are configurable to perform operations based on multiple graphics and media programming interfaces and are not specific to any one application programming interface (API). In some embodiments, driver software for the graphics processor translates API calls that are specific to a particular graphics or media library into commands that can be processed by the graphics processor. In some embodiments, support is provided for the Open Graphics Library (OpenGL), Open Computing Language (OpenCL), and/or Vulkan graphics and compute API, all from the Khronos Group. In some embodiments, support may also be provided for the Direct3D library from the Microsoft Corporation. In some embodiments, a combination of these libraries may be supported. Support may also be provided for the Open Source Computer Vision Library (OpenCV). A future API with a compatible 3D pipeline would also be supported if a mapping can be made from the pipeline of the future API to the pipeline of the graphics processor.

Graphics Pipeline Programming

FIG. **29**A is a block diagram illustrating a graphics processor command format **2900** according to some embodiments. FIG. **29**B is a block diagram illustrating a graphics processor command sequence **2910** according to an embodiment. The solid lined boxes in FIG. **29**A illustrate the components that are generally included in a graphics command while the dashed lines include components that are optional or that are only included in a sub-set of the graphics commands. The exemplary graphics processor command format **2900** of FIG. **29**A includes data fields to identify a target client **2902** of the command, a command operation code (opcode) **2904**, and the relevant data **2906** for the command. A sub-opcode **2905** and a command size **2908** are also included in some commands.

In some embodiments, client **2902** specifies the client unit of the graphics device that processes the command data. In some embodiments, a graphics processor command parser examines the client field of each command to condition the further processing of the command and route the command data to the appropriate client unit. In some embodiments, the graphics processor client units include a memory interface unit, a render unit, a 2D unit, a 3D unit, and a media unit. Each client unit has a corresponding processing pipeline that processes the commands. Once the command is received by the client unit, the client unit reads the opcode **2904** and, if present, sub-opcode **2905** to determine the operation to perform. The client unit performs the command using information in data field **2906**. For some commands an explicit command size **2908** is expected to specify the size of the command. In some embodiments, the command parser automatically determines the size of at least some of the com-

mands based on the command opcode. In some embodiments commands are aligned via multiples of a double word.

The flow diagram in FIG. **29**B shows an exemplary graphics processor command sequence **2910**. In some embodiments, software or firmware of a data processing system that features an embodiment of a graphics processor uses a version of the command sequence shown to set up, execute, and terminate a set of graphics operations. A sample command sequence is shown and described for purposes of example only as embodiments are not limited to these specific commands or to this command sequence. Moreover, the commands may be issued as batch of commands in a command sequence, such that the graphics processor will process the sequence of commands in at least partially concurrence.

In some embodiments, the graphics processor command sequence **2910** may begin with a pipeline flush command **2912** to cause any active graphics pipeline to complete the currently pending commands for the pipeline. In some embodiments, the 3D pipeline **2922** and the media pipeline **2924** do not operate concurrently. The pipeline flush is performed to cause the active graphics pipeline to complete any pending commands. In response to a pipeline flush, the command parser for the graphics processor will pause command processing until the active drawing engines complete pending operations and the relevant read caches are invalidated. Optionally, any data in the render cache that is marked 'dirty' can be flushed to memory. In some embodiments, pipeline flush command **2912** can be used for pipeline synchronization or before placing the graphics processor into a low power state.

In some embodiments, a pipeline select command **2913** is used when a command sequence requires the graphics processor to explicitly switch between pipelines. In some embodiments, a pipeline select command **2913** is required only once within an execution context before issuing pipeline commands unless the context is to issue commands for both pipelines. In some embodiments, a pipeline flush command **2912** is required immediately before a pipeline switch via the pipeline select command **2913**.

In some embodiments, a pipeline control command **2914** configures a graphics pipeline for operation and is used to program the 3D pipeline **2922** and the media pipeline **2924**. In some embodiments, pipeline control command **2914** configures the pipeline state for the active pipeline. In one embodiment, the pipeline control command **2914** is used for pipeline synchronization and to clear data from one or more cache memories within the active pipeline before processing a batch of commands.

In some embodiments, return buffer state commands **2916** are used to configure a set of return buffers for the respective pipelines to write data. Some pipeline operations require the allocation, selection, or configuration of one or more return buffers into which the operations write intermediate data during processing. In some embodiments, the graphics processor also uses one or more return buffers to store output data and to perform cross thread communication. In some embodiments, the return buffer state **2916** includes selecting the size and number of return buffers to use for a set of pipeline operations.

The remaining commands in the command sequence differ based on the active pipeline for operations. Based on a pipeline determination **2920**, the command sequence is tailored to the 3D pipeline **2922** beginning with the 3D pipeline state **2930** or the media pipeline **2924** beginning at the media pipeline state **2940**.

The commands to configure the 3D pipeline state **2930** include 3D state setting commands for vertex buffer state, vertex element state, constant color state, depth buffer state, and other state variables that are to be configured before 3D primitive commands are processed. The values of these commands are determined at least in part based on the particular 3D API in use. In some embodiments, 3D pipeline state **2930** commands are also able to selectively disable or bypass certain pipeline elements if those elements will not be used.

In some embodiments, 3D primitive **2932** command is used to submit 3D primitives to be processed by the 3D pipeline. Commands and associated parameters that are passed to the graphics processor via the 3D primitive **2932** command are forwarded to the vertex fetch function in the graphics pipeline. The vertex fetch function uses the 3D primitive **2932** command data to generate vertex data structures. The vertex data structures are stored in one or more return buffers. In some embodiments, 3D primitive **2932** command is used to perform vertex operations on 3D primitives via vertex shaders. To process vertex shaders, 3D pipeline **2922** dispatches shader execution threads to graphics processor execution units.

In some embodiments, 3D pipeline **2922** is triggered via an execute **2934** command or event. In some embodiments, a register write triggers command execution. In some embodiments execution is triggered via a 'go' or 'kick' command in the command sequence. In one embodiment, command execution is triggered using a pipeline synchronization command to flush the command sequence through the graphics pipeline. The 3D pipeline will perform geometry processing for the 3D primitives. Once operations are complete, the resulting geometric objects are rasterized and the pixel engine colors the resulting pixels. Additional commands to control pixel shading and pixel back end operations may also be included for those operations.

In some embodiments, the graphics processor command sequence **2910** follows the media pipeline **2924** path when performing media operations. In general, the specific use and manner of programming for the media pipeline **2924** depends on the media or compute operations to be performed. Specific media decode operations may be offloaded to the media pipeline during media decode. In some embodiments, the media pipeline can also be bypassed and media decode can be performed in whole or in part using resources provided by one or more general-purpose processing cores. In one embodiment, the media pipeline also includes elements for general-purpose graphics processor unit (GPGPU) operations, where the graphics processor is used to perform SIMD vector operations using computational shader programs that are not explicitly related to the rendering of graphics primitives.

In some embodiments, media pipeline **2924** is configured in a similar manner as the 3D pipeline **2922**. A set of commands to configure the media pipeline state **2940** are dispatched or placed into a command queue before the media object commands **2942**. In some embodiments, media pipeline state commands **2940** include data to configure the media pipeline elements that will be used to process the media objects. This includes data to configure the video decode and video encode logic within the media pipeline, such as encode or decode format. In some embodiments, media pipeline state commands **2940** also support the use of one or more pointers to "indirect" state elements that contain a batch of state settings.

In some embodiments, media object commands **2942** supply pointers to media objects for processing by the media

pipeline. The media objects include memory buffers containing video data to be processed. In some embodiments, all media pipeline states must be valid before issuing a media object command **2942**. Once the pipeline state is configured and media object commands **2942** are queued, the media pipeline **2924** is triggered via an execute command **2944** or an equivalent execute event (e.g., register write). Output from media pipeline **2924** may then be post processed by operations provided by the 3D pipeline **2922** or the media pipeline **2924**. In some embodiments, GPGPU operations are configured and executed in a similar manner as media operations.

Graphics Software Architecture

FIG. **30** illustrates exemplary graphics software architecture for a data processing system **3000** according to some embodiments. In some embodiments, software architecture includes a 3D graphics application **3010**, an operating system **3020**, and at least one processor **3030**. In some embodiments, processor **3030** includes a graphics processor **3032** and one or more general-purpose processor core(s) **3034**. The graphics application **3010** and operating system **3020** each execute in the system memory **3050** of the data processing system.

In some embodiments, 3D graphics application **3010** contains one or more shader programs including shader instructions **3012**. The shader language instructions may be in a high-level shader language, such as the High-Level Shader Language (HLSL) or the OpenGL Shader Language (GLSL). The application also includes executable instructions **3014** in a machine language suitable for execution by the general-purpose processor core **3034**. The application also includes graphics objects **3016** defined by vertex data.

In some embodiments, operating system **3020** is a Microsoft® Windows® operating system from the Microsoft Corporation, a proprietary UNIX-like operating system, or an open source UNIX-like operating system using a variant of the Linux kernel. The operating system **3020** can support a graphics API **3022** such as the Direct3D API, the OpenGL API, or the Vulkan API. When the Direct3D API is in use, the operating system **3020** uses a front-end shader compiler **3024** to compile any shader instructions **3012** in HLSL into a lower-level shader language. The compilation may be a just-in-time (JIT) compilation or the application can perform shader pre-compilation. In some embodiments, high-level shaders are compiled into low-level shaders during the compilation of the 3D graphics application **3010**. In some embodiments, the shader instructions **3012** are provided in an intermediate form, such as a version of the Standard Portable Intermediate Representation (SPIR) used by the Vulkan API.

In some embodiments, user mode graphics driver **3026** contains a back-end shader compiler **3027** to convert the shader instructions **3012** into a hardware specific representation. When the OpenGL API is in use, shader instructions **3012** in the GLSL high-level language are passed to a user mode graphics driver **3026** for compilation. In some embodiments, user mode graphics driver **3026** uses operating system kernel mode functions **3028** to communicate with a kernel mode graphics driver **3029**. In some embodiments, kernel mode graphics driver **3029** communicates with graphics processor **3032** to dispatch commands and instructions.

IP Core Implementations

One or more aspects of at least one embodiment may be implemented by representative code stored on a machine-readable medium which represents and/or defines logic within an integrated circuit such as a processor. For example, the machine-readable medium may include instructions which represent various logic within the processor. When read by a machine, the instructions may cause the machine to fabricate the logic to perform the techniques described herein. Such representations, known as "IP cores," are reusable units of logic for an integrated circuit that may be stored on a tangible, machine-readable medium as a hardware model that describes the structure of the integrated circuit. The hardware model may be supplied to various customers or manufacturing facilities, which load the hardware model on fabrication machines that manufacture the integrated circuit. The integrated circuit may be fabricated such that the circuit performs operations described in association with any of the embodiments described herein.

FIG. **31** is a block diagram illustrating an IP core development system **3100** that may be used to manufacture an integrated circuit to perform operations according to an embodiment. The IP core development system **3100** may be used to generate modular, re-usable designs that can be incorporated into a larger design or used to construct an entire integrated circuit (e.g., an SOC integrated circuit). A design facility **3130** can generate a software simulation **3110** of an IP core design in a high level programming language (e.g., C/C++). The software simulation **3110** can be used to design, test, and verify the behavior of the IP core using a simulation model **3112**. The simulation model **3112** may include functional, behavioral, and/or timing simulations. A register transfer level (RTL) design **3115** can then be created or synthesized from the simulation model **3112**. The RTL design **3115** is an abstraction of the behavior of the integrated circuit that models the flow of digital signals between hardware registers, including the associated logic performed using the modeled digital signals. In addition to an RTL design **3115**, lower-level designs at the logic level or transistor level may also be created, designed, or synthesized. Thus, the particular details of the initial design and simulation may vary.

The RTL design **3115** or equivalent may be further synthesized by the design facility into a hardware model **3120**, which may be in a hardware description language (HDL), or some other representation of physical design data. The HDL may be further simulated or tested to verify the IP core design. The IP core design can be stored for delivery to a 3rd party fabrication facility **3165** using non-volatile memory **3140** (e.g., hard disk, flash memory, or any non-volatile storage medium). Alternatively, the IP core design may be transmitted (e.g., via the Internet) over a wired connection **3150** or wireless connection **3160**. The fabrication facility **3165** may then fabricate an integrated circuit that is based at least in part on the IP core design. The fabricated integrated circuit can be configured to perform operations in accordance with at least one embodiment described herein.

Exemplary System on a Chip Integrated Circuit

FIG. **32-34** illustrated exemplary integrated circuits and associated graphics processors that may be fabricated using one or more IP cores, according to various embodiments described herein. In addition to what is illustrated, other logic and circuits may be included, including additional graphics processors/cores, peripheral interface controllers, or general-purpose processor cores.

FIG. **32** is a block diagram illustrating an exemplary system on a chip integrated circuit **3200** that may be fabricated using one or more IP cores, according to an embodiment. Exemplary integrated circuit **3200** includes one or more application processor(s) **3205** (e.g., CPUs), at least one graphics processor **3210**, and may additionally

include an image processor **3215** and/or a video processor **3220**, any of which may be a modular IP core from the same or multiple different design facilities. Integrated circuit **3200** includes peripheral or bus logic including a USB controller **3225**, UART controller **3230**, an SPI/SDIO controller **3235**, and an $I^2S/I^2C$ controller **3240**. Additionally, the integrated circuit can include a display device **3245** coupled to one or more of a high-definition multimedia interface (HDMI) controller **3250** and a mobile industry processor interface (MIPI) display interface **3255**. Storage may be provided by a flash memory subsystem **3260** including flash memory and a flash memory controller. Memory interface may be provided via a memory controller **3265** for access to SDRAM or SRAM memory devices. Some integrated circuits additionally include an embedded security engine **3270**.

FIG. **33** is a block diagram illustrating an exemplary graphics processor **3310** of a system on a chip integrated circuit that may be fabricated using one or more IP cores, according to an embodiment. Graphics processor **3310** can be a variant of the graphics processor **3210** of FIG. **32**. Graphics processor **3310** includes a vertex processor **3305** and one or more fragment processor(s) **3315**A-**3315**N (e.g., **3315**A, **3315**B, **3315**C, **3315**D, through **3315**N-**1**, and **3315**N). Graphics processor **3310** can execute different shader programs via separate logic, such that the vertex processor **3305** is optimized to execute operations for vertex shader programs, while the one or more fragment processor(s) **3315**A-**3315**N execute fragment (e.g., pixel) shading operations for fragment or pixel shader programs. The vertex processor **3305** performs the vertex processing stage of the 3D graphics pipeline and generates primitives and vertex data. The fragment processor(s) **3315**A-**3315**N use the primitive and vertex data generated by the vertex processor **3305** to produce a framebuffer that is displayed on a display device. In one embodiment, the fragment processor(s) **3315**A-**3315**N are optimized to execute fragment shader programs as provided for in the OpenGL API, which may be used to perform similar operations as a pixel shader program as provided for in the Direct 3D API.

Graphics processor **3310** additionally includes one or more memory management units (MMUs) **3320**A-**3320**B, cache(s) **3325**A-**3325**B, and circuit interconnect(s) **3330**A-**3330**B. The one or more MMU(s) **3320**A-**3320**B provide for virtual to physical address mapping for graphics processor **3310**, including for the vertex processor **3305** and/or fragment processor(s) **3315**A-**3315**N, which may reference vertex or image/texture data stored in memory, in addition to vertex or image/texture data stored in the one or more cache(s) **3325**A-**3325**B. In one embodiment the one or more MMU(s) **3325**A-**3325**B may be synchronized with other MMUs within the system, including one or more MMUs associated with the one or more application processor(s) **3205**, image processor **3215**, and/or video processor **3220** of FIG. **32**, such that each processor **3205**-**3220** can participate in a shared or unified virtual memory system. The one or more circuit interconnect(s) **3330**A-**3330**B enable graphics processor **3310** to interface with other IP cores within the SoC, either via an internal bus of the SoC or via a direct connection, according to embodiments.

FIG. **34** is a block diagram illustrating an additional exemplary graphics processor **3410** of a system on a chip integrated circuit that may be fabricated using one or more IP cores, according to an embodiment. Graphics processor **3410** can be a variant of the graphics processor **3210** of FIG. **32**. Graphics processor **3410** includes the one or more MMU(s) **3320**A-**3320**B, caches **3325**A-**3325**B, and circuit interconnects **3330**A-**3330**B of the integrated circuit **3300** of FIG. **33**.

Graphics processor **3410** includes one or more shader core(s) **3415**A-**3415**N (e.g., **3415**A, **3415**B, **3415**C, **3415**D, **3415**E, **3415**F, through **3415**N-**1**, and **3415**N), which provides for a unified shader core architecture in which a single core or type or core can execute all types of programmable shader code, including shader program code to implement vertex shaders, fragment shaders, and/or compute shaders. The exact number of shader cores present can vary among embodiments and implementations. Additionally, graphics processor **3410** includes an inter-core task manager **3405**, which acts as a thread dispatcher to dispatch execution threads to one or more shader cores **3415**A-**3415**N and a tiling unit **3418** to accelerate tiling operations for tile-based rendering, in which rendering operations for a scene are subdivided in image space, for example to exploit local spatial coherence within a scene or to optimize use of internal caches.

The following clauses and/or examples pertain to specific embodiments or examples thereof. Specifics in the examples may be used anywhere in one or more embodiments. The various features of the different embodiments or examples may be variously combined with some features included and others excluded to suit a variety of different applications. Examples may include subject matter such as a method, means for performing acts of the method, at least one machine-readable medium including instructions that, when performed by a machine cause the machine to perform acts of the method, or of an apparatus or system according to embodiments and examples described herein. Various components can be a means for performing the operations or functions described.

One embodiment provides for a compute apparatus to perform machine learning operations, the compute apparatus comprising instruction decode logic to decode a single instruction including multiple operands into a single decoded instruction, the multiple operands having differing precisions and a general-purpose graphics compute unit including a first logic unit and a second logic unit, the general-purpose graphics compute unit to execute the single decoded instruction, wherein to execute the single decoded instruction includes to perform a first instruction operation on a first set of operands of the multiple operands at a first precision and a simultaneously perform second instruction operation on a second set of operands of the multiple operands at a second precision.

One embodiment provides for a method of performing machine learning operations, the method comprising fetching and decoding a single instruction including multiple operands, the multiple operands referencing multiple data elements having different precision, performing a first instruction operation on a first set of the multiple data elements via a first logic unit within a compute unit, the first set of the multiple data elements having a first precision, performing a second instruction operation on a second set of the multiple data elements via a second logic unit within the compute unit in parallel with performing the first instruction operand via the first logic unit, the second set of the multiple data elements having a second precision, and outputting results of the first instruction operation and the second instruction operation.

One embodiment provides for a data processing system comprising a non-transitory machine-readable medium to store instructions for execution by one or more processors of the data processing system and a general-purpose graphics

processing unit including instruction decode logic to decode a single instruction including multiple operands into a single decoded instruction, the multiple operands having differing precisions and a compute unit including a first logic unit and a second logic unit, the compute unit to execute the single decoded instruction, wherein to execute the single decoded instruction includes to perform a first instruction operation on a first set of operands of the multiple operands at a first precision and a simultaneously perform second instruction operation on a second set of operands of the multiple operands at a second precision.

One embodiment provides for a graphics processing unit (GPU) to accelerate machine learning operations, the GPU comprising an instruction cache to store a first instruction and a second instruction, the first instruction to cause the GPU to perform a floating-point operation, including a multi-dimensional floating-point operation, and the second instruction to cause the GPU to perform an integer operation; and a general-purpose graphics compute unit having a single instruction, multiple thread (SIMT) architecture, the general-purpose graphics compute unit to simultaneously execute the first instruction and the second instruction, wherein the integer operation corresponds to a memory address calculation. In one embodiment, the multi-dimensional floating-point operation is a two-dimensional matrix multiply operation. The two-dimensional matrix multiply operation additionally includes an add operation. The two-dimensional matrix multiply operation, in one embodiment, causes the general-purpose graphics compute unit to compute a 32-bit product from two or more 16-bit operands. In one embodiment the GPU additionally includes a scheduler to schedule at least one thread of the first instruction and at least one thread of the second instruction to the general-purpose graphics compute unit. The scheduler can independently schedule multiple threads of each of the first instruction and the second instruction. The threads of the first instruction and the second instruction can have independent thread state.

One embodiment provides a data processing system comprising a non-transitory machine-readable medium to store instructions for execution; a graphics processing unit (GPU) to accelerate machine learning operations, the GPU including an instruction cache to store a first instruction and a second instruction, the first instruction to cause the GPU to perform a floating-point operation, including a multi-dimensional floating-point operation, and the second instruction to cause the GPU to perform an integer operation; and a general-purpose graphics compute unit included within the GPU, the general-purpose graphics compute unit having a single instruction, multiple thread (SIMT) architecture, the general-purpose graphics compute unit to simultaneously execute the first instruction and the second instruction, where the integer operation corresponds to a memory address calculation.

One embodiment provides a method of accelerating a machine-learning operation, the method comprising decoding a single instruction on a graphics processing unit (GPU), the GPU having a single instruction, multiple thread (SIMT) architecture; and simultaneously executing a first instruction and a second instruction on a multiprocessor of the GPU, wherein executing the first instruction includes performing a floating-point operation, including a multi-dimensional floating-point operation, wherein executing the second instruction includes performing an integer operation, and wherein the integer operation corresponds to a memory address calculation. In one embodiment the multi-dimensional floating-point operation is a two-dimensional matrix multiply operation, wherein the two-dimensional matrix multiply operation includes an add operation. In one embodiment the two-dimensional matrix multiply operation includes computing a 32-bit product from two or more 16-bit operands. In one embodiment, the method additionally includes scheduling at least one thread of the first instruction and at least one thread of the second instruction via a scheduler within the GPU. In one embodiment the method additionally comprises independently scheduling multiple threads of each of the first instruction and the second instruction, the multiple threads of each instruction having independent thread state.

One embodiment provides for a general-purpose graphics processing unit comprising a streaming multiprocessor having a single instruction, multiple thread (SIMT) architecture including hardware multithreading, wherein the streaming multiprocessor comprises a first processing block including a first processing core having a first floating-point data path and a second processing core having a first integer data path, the first integer data path independent of the first floating-point data path, wherein the first integer data path is to enable execution of a first instruction and the first floating-point data path is to enable execution of a second instruction, the first instruction to be executed concurrently with the second instruction; a second processing block including a third processing core having a second floating-point data path and a fourth processing core having a second integer data path, the second integer data path independent of the second floating-point data path, wherein the second integer data path is to enable execution of a third instruction and the second floating-point data path is to enable execution of a fourth instruction, the third instruction to be executed concurrently with the fourth instruction; and a memory coupled with the first processing block and the second processing block.

One embodiment provides for a graphics processor system comprising a high-bandwidth graphics double data rate memory and a general-purpose graphics processor coupled with the high-bandwidth graphics double data rate memory via two or more memory controllers. The general-purpose graphics processor comprises a hardware multithreading compute unit having a single instruction, multiple thread (SIMT) architecture. The hardware multithreading compute unit comprises a first register file and a first hardware scheduler, the first register file and the first hardware scheduler each coupled with a first processing core and a second processing core, the first processing core having a first floating-point data path and the second processing core having a first integer data path, the first integer data path independent of the first floating-point data path, wherein the first integer data path is to enable execution of a first instruction and the first floating-point data path is to enable execution of a second instruction, the first instruction to be executed concurrently with the second instruction; a second register file and a second hardware scheduler, the second register file and the second hardware scheduler each coupled with a third processing core and a fourth processing core, the third processing core having a second floating-point data path and the fourth processing core having a second integer data path, the second integer data path independent of the second floating-point data path, wherein the second integer data path is to enable execution of a third instruction and the second floating-point data path is to enable execution of a fourth instruction, the third instruction to be executed concurrently with the fourth instruction; and an internal memory

coupled with the first processing core, the second processing core, the third processing core, and the fourth processing core.

One embodiment provides for a method executed on a general-purpose graphics processing unit, the method comprising scheduling a first group of threads to a first processing core and a third processing core, the first group of threads to perform a set of floating-point operations, the first processing core and the third processing core included within a floating-point data path of a first processing block of the general-purpose graphics processing unit, the general-purpose graphics processing unit comprising a hardware multithreading compute unit having a single instruction, multiple thread (SIMT) architecture, wherein the first group of threads is associated with a first instruction; scheduling a second group of threads to a second processing core and a fourth processing core, the second group of threads to perform a set of integer operations, the second processing core and the fourth processing core included within an integer data path of a second processing block of the general-purpose graphics processing unit, wherein the second group of threads is associated with a second instruction; executing a first thread of the first instruction using processing cores of the floating-point data path; and executing a second thread of the second instruction using processing cores of the integer data path, the second thread executed independently and contiguously with the first thread.

Other embodiments provide for a non-transitory machine-readable medium storing instructions to cause one or more processors to perform any method described herein.

One embodiment provides a general-purpose graphics processing unit including a multiprocessor having a single instruction, multiple thread, SIMT, architecture. The multiprocessor includes multiple sets of compute units, each compute unit in each set of compute units having a first logic unit configured to perform floating-point operations and a second logic unit configured to perform integer operations and a memory coupled with the multiple sets of compute units, wherein in a compute unit of the multiple sets of compute units, the first logic unit is to execute a thread of a floating-point instruction to perform one or more of the floating-point operations and the second logic unit is to execute a thread of an integer instruction to perform one or more of the integer operations, the thread of the floating-point instruction being executed in parallel with the thread of the integer instruction.

One embodiment provides a general-purpose graphics processing unit including a multiprocessor having a single instruction, multiple thread, SIMT, architecture. The multiprocessor includes multiple sets of compute units, each compute unit in each set of compute units having a first logic unit configured to perform floating-point operations and a second logic unit configured to perform integer operations and a memory coupled with the multiple sets of compute units. In a compute unit of the multiple sets of compute units, the first logic unit is to execute a thread of a floating-point instruction to perform one or more of the floating-point operations and the second logic unit is to execute a thread of an integer instruction to perform one or more of the integer operations, the thread of the floating-point instruction being executed in parallel with the thread of the integer instruction. The general-purpose graphics processing unit additionally includes a geometry processing unit coupled to the multiprocessor, the geometry processing unit to execute geometry shader program code to transform graphics primitives and a rasterizer coupled to the geometry processing unit to perform triangle rasterization to generate pixel data.

The embodiments described herein refer to specific configurations of hardware, such as application specific integrated circuits (ASICs), configured to perform certain operations or having a predetermined functionality. Such electronic devices typically include a set of one or more processors coupled to one or more other components, such as one or more storage devices (non-transitory machine-readable storage media), user input/output devices (e.g., a keyboard, a touchscreen, and/or a display), and network connections. The coupling of the set of processors and other components is typically through one or more busses and bridges (also termed as bus controllers). The storage device and signals carrying the network traffic respectively represent one or more machine-readable storage media and machine-readable communication media. Thus, the storage devices of a given electronic device typically store code and/or data for execution on the set of one or more processors of that electronic device.

Of course, one or more parts of an embodiment may be implemented using different combinations of software, firmware, and/or hardware. Throughout this detailed description, for the purposes of explanation, numerous specific details were set forth in order to provide a thorough understanding of the present invention. It will be apparent, however, to one skilled in the art that the embodiments may be practiced without some of these specific details. In certain instances, well-known structures and functions were not described in elaborate detail to avoid obscuring the inventive subject matter of the embodiments. Accordingly, the scope and spirit of the invention should be judged in terms of the claims that follow.

What is claimed is:

**1**. A general-purpose graphics processing unit including:

a processing cluster including a plurality of multiprocessors interconnected via a data crossbar configured to enable exchange of data between the plurality of multiprocessors directly via the data crossbar, from a first multiprocessor of the plurality of multiprocessors to a second multiprocessor of the plurality of multiprocessors, the plurality of multiprocessors having a single instruction, multiple thread, SIMT, architecture, wherein a multiprocessor of the plurality of multiprocessors comprises:

multiple sets of compute units, each compute unit in each set of compute units having a first logic unit configured to perform floating-point operations and a second logic unit configured to perform integer operations; and

a memory coupled with the multiple sets of compute units, wherein in a compute unit of the multiple sets of compute units, the first logic unit is to execute a thread of a floating-point instruction to perform one or more of the floating-point operations, the one or more of the floating-point operations include a 64-bit floating-point operation, and the second logic unit is to execute a thread of an integer instruction to perform one or more of the integer operations, the thread of the floating-point instruction being executed in parallel with the thread of the integer instruction.

**2**. The general-purpose graphics processing unit as in claim **1**, wherein the memory is shared between the multiple sets of compute units.

**3**. The general-purpose graphics processing unit as in claim **2**, wherein the memory includes a data cache accessible by the multiple sets of compute units.

4. The general-purpose graphics processing unit as in claim 1, wherein the floating-point instruction is to indicate to the multiprocessor to perform multiple cycles of a multi-dimensional floating-point operation in response to the floating-point instruction.

5. The general-purpose graphics processing unit as in claim 4, wherein the multi-dimensional floating-point operation is a two-dimensional matrix multiply operation.

6. The general-purpose graphics processing unit as in claim 5, wherein the two-dimensional matrix multiply operation additionally includes an add operation.

7. The general-purpose graphics processing unit as in claim 1, additionally including a scheduler to schedule at least one thread of the floating-point instruction and at least one thread of the integer instruction to the compute unit of the multiple sets of compute units.

8. The general-purpose graphics processing unit as in claim 7, the scheduler to independently schedule multiple threads of each of the floating-point instruction and the integer instruction.

9. The general-purpose graphics processing unit as in claim 8, wherein threads of the floating-point instruction and the integer instruction have independent thread state.

10. The general-purpose graphics processing unit as in claim 1 further comprising: a geometry processing unit to execute geometry shader program code to transform graphics primitives.

11. The general-purpose graphics processing unit as in claim 10 wherein the geometry processing unit is to subdivide the graphics primitives into one or more new graphics primitives.

12. The general-purpose graphics processing unit as in claim 11 further comprising: a rasterizer to perform fixed-function triangle rasterization to generate pixels.

13. A general-purpose graphics processing unit including:

a processing cluster including a plurality of multiprocessors interconnected via a data crossbar configured to enable exchange of data between the plurality of multiprocessors directly via the data crossbar, from a first multiprocessor of the plurality of multiprocessors to a second multiprocessor of the plurality of multiprocessors, the plurality of multiprocessors having a single instruction, multiple thread, SIMT, architecture, wherein a multiprocessor of the plurality of multiprocessors comprises:

multiple sets of compute units, each compute unit in each set of compute units having a first logic unit configured to perform floating-point operations and a second logic unit configured to perform integer operations;

a memory coupled with the multiple sets of compute units, wherein in a compute unit of the multiple sets of compute units, the first logic unit is to execute a thread of a floating-point instruction to perform one or more of the floating-point operations, the one or more of the floating-point operations include a 64-bit floating-point operation, and the second logic unit is to execute a thread of an integer instruction to perform one or more of the integer operations, the thread of the floating-point instruction being executed in parallel with the thread of the integer instruction;

a geometry processing unit coupled to the multiprocessor, the geometry processing unit to execute geometry shader program code to transform graphics primitives; and

a rasterizer coupled to the geometry processing unit to perform triangle rasterization to generate pixel data.

14. The general-purpose graphics processing unit as in claim 13, wherein the memory is shared between the multiple sets of compute units.

15. The general-purpose graphics processing unit as in claim 14, wherein the memory includes a data cache accessible by the multiple sets of compute units.

16. The general-purpose graphics processing unit as in claim 13, wherein the floating-point instruction is to indicate to the multiprocessor to perform multiple cycles of a multi-dimensional floating-point operation in response to the floating-point instruction.

17. The general-purpose graphics processing unit as in claim 16, wherein the multi-dimensional floating-point operation is a two-dimensional matrix multiply operation.

18. The general-purpose graphics processing unit as in claim 17, wherein the two-dimensional matrix multiply operation additionally includes an add operation.

19. The general-purpose graphics processing unit as in claim 13, additionally including a scheduler to schedule at least one thread of the floating-point instruction and at least one thread of the integer instruction to the compute unit of the multiple sets of compute units.

20. The general-purpose graphics processing unit as in claim 19, the scheduler to independently schedule multiple threads of each of the floating-point instruction and the integer instruction.

21. The general-purpose graphics processing unit as in claim 20, wherein threads of the floating-point instruction and the integer instruction have independent thread state.

* * * * *